CODERHOUSE

MATERIAL DE LECTURA

JavaScript

Guía de lectura del curso

Contenido

MÓDULO 1 · Introducción a JavaScript 4

JavaScript y su entorno de ejecución 4

Variables modernas: let y const 6

Tipos primitivos y operadores aritméticos básicos 8

PRE-ENTREGA **Pre-entrega 1: Primeros pasos con JavaScript** 13

MÓDULO 2 · Control de ciclos y lógica de flujo 16

Decisiones con condicionales: if, else if, else y switch 16

Comparaciones y condiciones compuestas 20

Repetición con for, while y do...while 21

PRE-ENTREGA **Pre-entrega 2: Lógica de Programación y Control de Flujo** 23

MÓDULO 3 · Funciones 27

Declaración y expresión de funciones 27

Parámetros, argumentos y retorno de valores 29

Scope: ámbito global y local 31

PRE-ENTREGA **Pre-entrega 3: Funciones e Integración de Lógica** 35

MÓDULO 4 · Arrays 38

Arrays: Lista de elementos 38

Acceso y modificación de elementos por posición 40

Recorrer arrays con for y for...of 44

PRE-ENTREGA **Pre-entrega 4: Uso e interacción con Arrays** 46

MÓDULO 5 · Objetos 49

Objetos literales: propiedades y métodos 49

Constructores y operador new 52

Clases en ES6 y prototipos: la forma moderna de crear objetos 54

PRE-ENTREGA **Pre-entrega 5: Instanciando Objetos** 57

MÓDULO 6 · Funciones de Orden Superior 60

Funciones como valores y abstracción 60

Búsqueda inteligente con find, filter y some 62

Map y reduce para transformar colecciones 64

PRE-ENTREGA Pre-entrega 6: Interactuando con Funciones de Orden Superior 67

MÓDULO 7 · DOM y Eventos: Interactividad en la Web 70

Nodos y selectores: encontrar elementos en el DOM 70

Cambiar texto y elementos para actualizar la interfaz 73

Interactividad: addEventListener y objeto Event 75

PRE-ENTREGA Pre-entrega 7: Interfaz dinámica con DOM y eventos 79

MÓDULO 8 · Storage, Operadores Avanzados y Workshop Integrador 83

Web Storage: persistencia local con localStorage y sessionStorage 83

Operadores modernos para escribir menos y evitar errores 87

Desestructuración 91

PRE-ENTREGA Pre-entrega 8: Sincronización de estado entre DOM y Storage 95

MÓDULO 9 · Asincronismo, Promesas y Flujo No Bloqueante 97

El Event Loop y por qué JavaScript no se bloquea 97

Callbacks y Promesas: de una función diferida a un objeto con estado 100

Async/Await: El Flujo Secuencial Asíncrono 102

PRE-ENTREGA Pre-entrega 9: Asincronismo y Promesas 104

MÓDULO 10 · APIs, Peticiones y Librerías 106

Cómo funciona Fetch API y cómo leer respuestas JSON 106

Librerías externas para mejorar la experiencia de usuario 109

PRE-ENTREGA Pre-entrega 10: APIs, Peticiones y Librerías 113

MÓDULO 11 · Ecosistema Moderno: Node.js, Frameworks y Cierre 116

Node.js y npm: ejecutar JavaScript fuera del navegador 116

React, Vue y Angular: cómo pensar en frameworks modernos 118

MÓDULO 12 · Proyecto Final 122

ENTREGA FINAL Entrega final: aplicación web interactiva 123

ANEXO · Material complementario 127

Palabras clave 127

Análisis profundo 128

Preguntas frecuentes 129

Conclusiones 131

MÓDULO 1

Introducción a JavaScript

Dominar los fundamentos de la sintaxis, declaración de variables con let y const y el uso de tipos de datos primitivos en JavaScript.

JavaScript y su entorno de ejecución

Aprende la diferencia fundamental entre el lenguaje JavaScript y el entorno donde se ejecuta), sus APIs específicas y cómo elegir el adecuado según tus objetivos.

¡Bienvenido al inicio de tu camino como desarrollador! Si estás aquí, probablemente ya sepas que JavaScript es el lenguaje que hace que las páginas web sean interactivas. Pero, ¿alguna vez te has preguntado dónde vive realmente ese código? ¿Cómo es que una línea de texto escrita en un editor se convierte en un botón que cambia de color o en un servidor que procesa los pagos de una tienda online?

Para entender esto, debemos hacer una distinción fundamental: una cosa es el **lenguaje** (las reglas de cómo escribimos) y otra es el **entorno de ejecución** (el lugar donde ese código cobra vida).

1. ¿Qué es un entorno de ejecución?

Imagina que JavaScript es una **receta de cocina**. La receta te dice qué ingredientes usar y qué pasos seguir, pero la receta por sí sola no puede cocinar nada. Necesitas una **cocina** equipada con herramientas: fuego, sartenes, horno y agua.

En programación, el **entorno de ejecución** es esa cocina. Es el software que permite que el código JavaScript se ejecute y, lo más importante, le proporciona "herramientas" adicionales para interactuar con el mundo exterior. Para este curso, nuestro entorno de ejecucion será el **Navegador Web** (Chrome, Firefox, edge).

2. El Navegador: Interacción y Visualización

Cuando abres una página web, el navegador descarga archivos de JavaScript y los ejecuta. El objetivo principal de este entorno es la **interacción con el usuario**.

El papel de `window` y `document`

En el navegador, JavaScript tiene acceso a objetos globales que no existen en ningún otro lado. Los dos más importantes son:

- **`window`**: Representa la ventana del navegador. Gracias a él, JavaScript puede saber qué tan grande es tu pantalla, mostrar alertas o incluso saber en qué URL estás navegando.
- **`document`**: Es la representación de todo el HTML de tu página. A través del `document`, JavaScript puede crear nuevos elementos, borrar párrafos, cambiar colores o reaccionar cuando haces clic en

un botón. A esto se le llama **DOM (Document Object Model) y lo veremos mas adelante en el curso.**

APIs del Navegador

Además del DOM, el navegador te regala herramientas (APIs) para tareas específicas:

- **Fetch API:** Para pedir datos a otros sitios (como consultar el clima o noticias).
- **Web Storage:** Para que la página "recuerde" cosas incluso si la cierras (como el modo oscuro o los artículos en un carrito).
- **Geolocation:** Para pedir permiso al usuario y saber su ubicación.

Ejemplo real: Cuando entras a **Google Docs**, el JavaScript en tu navegador está usando intensamente el `document` para dibujar cada letra que escribes y `fetch` para guardar esos cambios en la nube en tiempo real.

3. Conectar JavaScript a HTML

Cuando desarrollamos sitios web, es común separar el contenido HTML del código JavaScript para mantener organizado el código y facilitar su mantenimiento, por lo que es muy importante tener un archivo JS externo, dedicado a todo el código.

Para que nuestro proyecto funcione correctamente, hay que vincular el archivo JS al HTML, esto se logra mediante la etiqueta `<script>` en HTML, que permite incluir archivos JavaScript externos al documento HTML.

Uso Básico de la Etiqueta `<script>`

Para vincular un archivo JavaScript externo a un documento HTML, se utiliza la etiqueta

`<script>` con el atributo `src`, que contiene la ruta al archivo JavaScript. Esta etiqueta se puede colocar dentro del `<head>` (lo utilizado actualmente) o antes del cierre del `<body>` (reservado para librerias).

Ejemplo:

CÓDIGO

```
<script src="ruta/al/archivo.js"></script>
```

Atributos async y defer

Para controlar cómo y cuándo se cargan los scripts en el head del navegador, podemos usar los atributos `async` y `defer`, que influyen en el comportamiento de carga y ejecución de los scripts sin bloquear el renderizado del resto de la página.

`async` permite que el script se cargue de manera asincrónica con respecto al resto de la página. Esto significa que el navegador no necesita esperar a que el script se cargue y ejecute para continuar

procesando el resto del contenido HTML. El script se ejecutará tan pronto como esté disponible, lo que puede mejorar la velocidad de carga de la página.

Ejemplo:

CÓDIGO

```
<script async src="ruta/al/archivo.js"></script>
```

`defer`, por otro lado, también permite la carga asincrónica del script, pero retrasa la ejecución del mismo hasta que todo el documento HTML ha sido completamente cargado y analizado. Esto es útil para scripts que necesitan interactuar con elementos del DOM o cuyo orden de ejecución es importante (la practica actual de enrutado).

Ejemplo:

CÓDIGO

```
<script defer src="ruta/al/archivo.js"></script>
```

Conclusión

Utilizar adecuadamente

`async` y `defer` puede mejorar significativamente la experiencia del usuario al reducir los tiempos de carga percibidos y asegurar que el contenido crítico se muestre lo más rápido posible. Al integrar JavaScript en nuestros documentos HTML, estos atributos son herramientas valiosas para optimizar el rendimiento y la interactividad de las páginas web.

Variables modernas: let y const

Aprende a gestionar datos de forma segura usando las palabras reservadas let y const, y entiende por qué var ha quedado en desuso.

En JavaScript moderno, la forma en que guardamos información ha evolucionado. Ya no simplemente "creamos variables"; ahora decidimos qué tan estricto queremos que sea nuestro código usando `let` y `const`.

¿Por qué importa cómo declaramos variables?

Imagina que estás construyendo una aplicación de banca. Tienes el nombre del banco (que nunca cambia) y el saldo del usuario (que cambia constantemente). Si usas la herramienta equivocada para guardar estos datos, podrías permitir que un error en el código sobrescriba el nombre del banco o que el saldo se pierda en una parte del programa donde no debería existir.

Los tres jugadores: const, let y var

1. `const` **(Constante):** Es tu opción por defecto. Úsala para todo lo que **no necesite ser reasignado**. Si intentas cambiar su valor después de definirlo, JavaScript lanzará un error.
2. `let` **:** Úsala cuando sepas que el valor **va a cambiar** (un contador, un total, una entrada de texto).
3. `var` **:** Es la forma antigua. A diferencia de `let` y `const`, `var` ignora los bloques de código (como los `if` o los `for` que lo veremos mas adelante), lo que a menudo causa errores donde las variables "se filtran" hacia afuera. **En el desarrollo actual, evitamos usar** `var` **.**

Errores comunes de principiantes

- **Intentar reasignar un** `const` : `const pi = 3.14; pi = 4;` -> ¡Error!
- **Olvidar inicializar un** `const` : No puedes hacer `const nombre;` sin darle un valor inmediatamente.

Visualizacion de la información

En JavaScript, existen diferentes formas de mostrar información al usuario o al programador, antes de aprender manejo de DOM vamos a implementar dos de las más utilizadas: `console.log()` y `alert()`.

El método `console.log()` se utiliza para mostrar mensajes en la consola del navegador. Es muy útil para probar código, revisar valores de variables y detectar errores mientras programamos.

Ejemplo:

CÓDIGO

```
let nombre = "Ana"
console.log(nombre)
```

Resultado en la consola: `Ana`

¿Dónde se ve?

La información aparece en la **consola del navegador**, que puedes abrir normalmente con `F12`

Luego selecciona la pestaña **Console**.

¿Para qué sirve?

- Verificar valores de variables.
- Comprobar si una función funciona correctamente.
- Depurar errores.

El método `alert()` muestra un mensaje emergente en pantalla.

Ejemplo:

CÓDIGO

```
alert("Bienvenido al curso de JavaScript");
```

Resultado: Aparece una ventana con el mensaje y un botón de “Aceptar”.

¿Para qué sirve?

- Mostrar avisos simples al usuario.
- Confirmar acciones importantes.
- Enseñar resultados rápidos en ejercicios básicos.

Diferencias principales

`console.log()`	`alert()`
Muestra información en la consola	Muestra una ventana emergente
Lo usa principalmente el programador	Lo ve directamente el usuario
No interrumpe la página	Detiene la interacción hasta aceptar

Tipos primitivos y operadores aritméticos básicos

Aprende a diferenciar y utilizar los tipos de datos primitivos en JavaScript y a realizar operaciones matemáticas correctas evitando errores comunes de tipos.

Imagina que estás construyendo una aplicación para un supermercado. Tienes que manejar el nombre de un producto, su precio, cuántas unidades hay en stock y si el producto tiene descuento o no. En programación, estos diferentes tipos de información no se guardan de la misma manera: el nombre es texto, el precio es un número y la disponibilidad es un "sí" o un "no".

1. ¿Qué es un tipo de dato primitivo?

En JavaScript, un **valor primitivo** es un dato que no es un objeto y no tiene métodos (funciones propias) asociados de forma intrínseca. Son los bloques de construcción más simples y básicos del lenguaje. A diferencia de las estructuras complejas que veremos más adelante, los primitivos representan un único valor.

Una característica fundamental es que son **inmutables**: una vez que se crea un valor primitivo, no se puede cambiar. Si tienes el número `5` y quieres que sea `6`, no "cambias" el 5, sino que creas un valor nuevo (el 6) y lo asignas a la variable.

Los 5 tipos de datos primitivos principales

1. **Number**: Para todos los números (enteros y decimales).
2. **String**: Para cadenas de texto.

3. **Boolean**: Para valores lógicos (verdadero o falso).
4. **Undefined**: Para variables declaradas a las que no se les ha asignado un valor.
5. **Null**: Para representar intencionalmente la ausencia de valor.

2. Explorando los tipos a fondo

Number: Un solo tipo para todos los números

A diferencia de otros lenguajes de programación que distinguen entre números enteros y decimales, JavaScript usa el tipo `number` para ambos.

CÓDIGO

```
let edad = 25          // Entero
let precio = 99.99     // Decimal (se usa punto, no coma)
let temperatura = -10   // Negativo
```

Dentro de `number`, existen tres valores "especiales" que debes conocer:

- **NaN (Not a Number):** Aparece cuando intentas hacer una operación matemática matemáticamente imposible (ej. `"hola" * 2`). Curiosamente, ¡el tipo de dato de `NaN` sigue siendo `number`!
- **Infinity / -Infinity:** Resultado de dividir por cero o superar el límite de capacidad de los números en JS.

String: El mundo del texto

Las cadenas de texto se definen encerrando el contenido entre comillas. Puedes usar comillas simples `'` o dobles `".`

CÓDIGO

```
let nombre = "Ana"
let saludo = 'Hola, ¿cómo estás?'
```

Boolean: El interruptor lógico

Solo tiene dos valores posibles: `true` (verdadero) o `false` (falso). Es la base de la toma de decisiones en programación.

CÓDIGO

```
let estaLogueado = true
let tienePermisos = false
```

Undefined vs Null: El arte de la "nada"

Aquí es donde muchos estudiantes se confunden. Aunque ambos parecen significar "nada", se usan en contextos distintos:

- **undefined:** Es el valor que JavaScript asigna automáticamente a una variable que ha sido declarada pero aún no tiene valor. Significa: *"No sé qué es esto aún"*.
- **null:** Es un valor que tú, como programador, asignas a propósito para decir que una variable está vacía. Significa: *"Sé qué es esto, y he decidido que no tiene valor"*.

3. El operador `typeof` y el famoso error de `null`

¿Cómo podemos saber de qué tipo es una variable mientras nuestro código corre? Usamos el operador `typeof`.

CÓDIGO

```
console.log(typeof 42)         // "number"
console.log(typeof "Hola")     // "string"
console.log(typeof true)        // "boolean"
let miVariable
console.log(typeof miVariable)  // "undefined"
```

4. Concatenacion de mensajes

El signo `+` en JavaScript tiene "doble personalidad".

- Si ambos valores son números, los **suma**.
- Si al menos uno es un `string`, los **concatena** (los pega uno al lado del otro).

CÓDIGO

```
console.log(5 + 5)     // 10 (Número)
console.log("5" + 5)   // "55" (String) - ¡Cuidado con esto!
console.log("Hola " + "Mundo") // "Hola Mundo"
```

La concatenación nos ayuda a mostrar mensajes más elaborados, combinando strings con los valores asignados a las variables que deseemos utilizar para mostrar la información de forma más ordenada.

5. Prompt: Comunicación con el usuario

En JavaScript, cuando necesitamos que el usuario escriba información, podemos utilizar `prompt()`. Además, muchas veces es necesario convertir esos datos a números usando `parseInt()` o `parseFloat()`.

El método `prompt()` muestra una ventana emergente donde el usuario puede escribir información:

CÓDIGO

```
let nombre = prompt("Escribe tu nombre")
console.log(nombre)
```

¿Qué sucede?

1. Aparece una ventana con el mensaje: `"Escribe tu nombre"`
2. El usuario escribe un dato.
3. Ese dato se guarda en la variable `nombre`.

Importante: `prompt()` siempre devuelve texto

Aunque el usuario escriba números, JavaScript los guarda como **string** (texto), por eso necesitamos convertir el texto a número.

`parseInt()` convierte un texto en un número entero:

CÓDIGO

```
let edad = prompt("Escribe tu edad")
edad = parseInt(edad)
console.log(edad)
console.log(typeof edad)
```

Resultado: 25 number

¿Qué hace?

- Convierte `"25"` → `25` (tipo numérico)
- Elimina decimales si existen.

`parseFloat()` convierte un texto en un número decimal:

CÓDIGO

```
let precio = prompt("Escribe el precio")
precio = parseFloat(precio)
console.log(precio)
```

Resultado: 19.99

¿Qué hace?

- Convierte `"19.99"` → `19.99`
- Mantiene los decimales.

6. Operadores aritméticos y precedencia

Ahora que sabemos qué tipos de datos existen, veamos cómo los números interactúan entre sí. Los operadores aritméticos básicos son similares a los de una calculadora:

Operador	Operación	Ejemplo	Resultado
`+`	Suma	`10 + 5`	`15`
`-`	Resta	`10 - 5`	`5`
`*`	Multiplicación	`10 * 2`	`20`
`/`	División	`10 / 2`	`5`
`%`	Resto (Remanente)	`10 % 3`	`1` (lo que sobra de la división)
`**`	Exponenciación	`2 ** 3`	`8` (2 al cubo)

Precedencia de operadores

Al igual que en matemáticas, JavaScript sigue un orden de prioridad:

1. Paréntesis `()`
2. Potencias ``
3. Multiplicación `*`, División `/` y Resto `%` (de izquierda a derecha)
4. Suma `+` y Resta `-` (de izquierda a derecha)

Ejemplo importante:

CÓDIGO

```
let total = 10 + 5 * 2;   // Resultado: 20 (primero multiplica 5*2, luego suma 10)
let total2 = (10 + 5) * 2; // Resultado: 30 (los paréntesis obligan a sumar primero)
```

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 1: Primeros pasos con JavaScript

Entregable para crear el primer script funcional en JavaScript, integrando variables, tipos primitivos e interacción con el usuario a través del navegador.

> **Ejemplo**
>
> **Este es el entregable evaluado del Módulo 1.** La *Pre-Entrega 1: Primeros pasos con JavaScript* es el único checkpoint que se corrige y suma al proyecto. Los ejercicios y ejemplos de las unidades teóricas anteriores son **práctica guiada (no evaluable)**.

¡Felicidades! Has llegado al final del primer módulo. Hasta ahora has aprendido la teoría sobre dónde corre JavaScript, cómo declarar variables modernas con `let` y `const`, y cómo funcionan los tipos de datos primitivos. Pero la programación no se trata solo de escribir datos en un archivo; se trata de **interacción**.

Este checkpoint es el puente entre el código estático y una aplicación que "escucha" y "responde" al usuario.

Contexto y posición en el programa

El entregable final de este programa será **una página web interactiva** de la temática que elijas en JavaScript que permitirá simular distintos procesos. **Un "simulador"** es un programa que soluciona ciertas tareas y proporciona al usuario información de valor. **Utilizarás FETCH y JSON** para obtener datos y diversas herramientas de JS como **librerías, promises, async y frameworks** para controlar eventos en la interfaz y producir animaciones en respuesta.

Esta primer entrega es la semilla de la lógica de procesamiento que usarás allá.

Objetivo de aprendizaje

Consolidar el uso de variables (`const` / `let`), tipos de datos y los métodos de interacción del navegador para crear un flujo lógico de programa que solicite, transforme y devuelva información.

¿Qué debes construir?

Debes crear un repositorio que contenga un script de JavaScript vinculado a un archivo HTML. El script debe realizar las siguientes acciones:

1. **Solicitar datos**: Pedir al menos tres datos distintos al usuario (ej. nombre y año de nacimiento).
2. **Procesar información**: Realizar una operación aritmética simple (ej. calcular la edad actual) y una transformación de texto concatenando variables con strings.

3. **Comunicar resultados**: Mostrar el resultado final al usuario mediante la colsola o una alerta.

Criterios de Aceptación

El proyecto debe cumplir con:

- **Uso de Variables**: No utilizar `var`. Debes usar `const` para valores que no cambian y `let` para los que sí.
- **Interacción**: Uso correcto de `prompt()`, `parseInt()/parseFloat()` para capturar datos y `console.log()/alert()` para mostrar resultados.
- **Lógica**: Realizar al menos una conversión de tipo (recordar que `prompt` devuelve strings) para realizar un cálculo matemático.
- **Entorno**: El script debe estar en un archivo `.js` externo, dentro de su carpeta, vinculado correctamente al `index.html` en el head con el atributo `defer`

Método de Entrega

Se deberá enviar el link del repositorio de github en modo público.

Instrucciones del Ejercicio: "Mi Primer Script Interactivo"

En esta práctica consolidarás todo lo aprendido en el Módulo 1. Tu misión es crear un pequeño programa que interactúe con el usuario y realice un cálculo lógico.

Requisitos Técnicos:

1. **Estructura de archivos**: Crea una carpeta para tu proyecto con un archivo `index.html` y un archivo `main.js` en su correspondiente carpeta. Vincula el script en la etiqueta head del HTML con el atributo defer.
2. **Captura de datos**: * Usa `prompt()` para solicitarle mínimo tres datos diferentes al usuario. * Almacenar los datos en variables o constantes según lo solicitado.
3. **Procesamiento**: * Conviertelos numeros solicitados vía prompt a tipo `number`. * Concatena strings con la información solicitada al usuario. * Realiza operaciones matemáticas con los números solicitados.
4. **Salida de datos**: * Usa `console.log()/alert()` para mostrar los diferentes mensajes.

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Rúbrica para evaluar el checkpoint del Módulo 1 en la que el estudiante crea un primer script interactivo en JavaScript usando prompt, alert, console.log, let, const, conversión de tipos y una organización básica de archivos en un repositorio de GitHub. La evaluación se centra en el alcance del ejercicio, la corrección técnica y la claridad del código como evidencia de aprendizaje inicial.

Criterio	Descripción	Peso
Estructura del proyecto y entrega en repositorio	Evalúa que la entrega sea un repositorio público de GitHub con la estructura solicitada y los archivos correctamente organizados. El archivo main.js debe estar enrutado en el head con el atributo defer.	25%
Captura de datos con prompt y uso correcto de const y let	Evalúa si el programa solicita información con prompt, guardando la infomación en variable, respetando las reglas de let y const aprendidas en el módulo.	20%
Procesamiento de la numeros y conversión de tipos	Evalúa que el estudiante identifique el tipo de dato inicial, convierta informaci{on del prompt a número y elabore correctamente operaciones matemáticas sin concatenar los numeros.	35%
Salida de información con alert y console.log	Evalúa que el programa presente comunicación con alert y console.log.	15%
Calidad del código: nombres descriptivos, ausencia de var y comentarios breves	Evalúa la legibilidad del código, el uso correcto de nombres descriptivos, la ausencia de var y la presencia de comentarios breves.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 2

Control de ciclos y lógica de flujo

Implementar estructuras de control condicionales y bucles para gestionar el flujo lógico de una aplicación.

Decisiones con condicionales: if, else if, else y switch

Aprende a controlar el flujo de una aplicación de JavaScript mediante el uso de estructuras condicionales, permitiendo que tu programa tome decisiones inteligentes basadas en datos.

Imagina que estás programando un videojuego. Tu personaje llega a una puerta: si tiene la llave, la puerta se abre; si no la tiene, aparece un mensaje que dice "Cerrado". O piensa en un cajero automático: si el saldo es suficiente, permite retirar dinero; de lo contrario, rechaza la operación. Incluso algo tan simple como un sitio web que te saluda con un "¡Buenos días!" o "¡Buenas noches!" dependiendo de la hora.

Todas estas situaciones tienen algo en común: el programa debe **tomar una decisión**. Hasta ahora, hemos aprendido que el código se ejecuta de arriba hacia abajo, línea por línea. Pero en el mundo real, los programas no siempre siguen un camino recto. Las **estructuras condicionales** son las herramientas que nos permiten crear bifurcaciones en nuestro código, permitiendo que JavaScript ejecute diferentes bloques de instrucciones según se cumplan o no ciertas condiciones.

1. La base de todo: El condicional `if`

El condicional `if` (que en español significa "si", pero en sentido condicional) es la estructura más básica de control. Su función es simple: **si** una condición es verdadera, entonces ejecuta un bloque de código.

Estructura básica

CÓDIGO

```
if (condicion) {
    // Este código solo se ejecuta si la condición es verdadera (true)
}
```

Ejemplo concreto

Imagina que queremos validar si una persona es cliente o empleado para ver cierta informacion de una pagina de un comercio:

CÓDIGO

```
let persona = "empleado"

if (persona == "empleado") {
    console.log("Bienvenido al panel de administrador")
}
```

¿Qué sucede aquí?

1. JavaScript evalúa lo que está dentro de los paréntesis `(persona == "empleado")`.
2. Como la variable tiene asignado el mismo valor esperado, el resultado es `true` (verdadero).
3. Al ser verdadero, se ejecuta el código dentro de las llaves `{ ... }`.
4. Si hubiéramos puesto `persona == "cliente"`, la condición sería `false` y el mensaje nunca aparecería en la consola.

2. El camino alternativo: `else`

A menudo, no solo queremos saber qué hacer si algo es cierto, sino también qué hacer en caso de que sea **falso**. Aquí es donde entra el `else` ("de lo contrario").

Ejemplo: Validación de acceso

CÓDIGO

```
const passwordCorrecta = "codear123"
const passwordIngresada = "123456"

if (passwordIngresada === passwordCorrecta) {
    console.log("Acceso concedido. Iniciando sesión...")
} else {
    console.log("Contraseña incorrecta. Inténtalo de nuevo.")
}
```

El "Porqué" del `else`: Sin el `else`, si la contraseña fuera incorrecta, el usuario no recibiría ninguna respuesta y pensaría que el botón no funciona. El `else` nos permite manejar los casos negativos y mejorar la experiencia del usuario. Su uso es opcional pero recomendado para ciertos casos.

3. Decisiones múltiples con `else if`

¿Qué pasa si hay más de dos caminos posibles? Por ejemplo, un semáforo tiene tres estados: Verde, Amarillo y Rojo. Para estos casos usamos `else if`.

Ejemplo: El clima según la temperatura

CÓDIGO

```
let temperatura = 22

if (temperatura > 30) {
    console.log("Hace mucho calor, busca sombra.")
} else if (temperatura >= 15) {
    console.log("El clima está agradable.")
} else {
    console.log("Hace mucho frío, quédate en casa.")
}
```

Reglas de oro del `else if`:

1. **Orden secuencial**: JavaScript revisa las condiciones de arriba hacia abajo. En cuanto encuentra una que es verdadera, ejecuta ese bloque y **ignora todo lo demás**.
2. **Exclusividad**: Solo se ejecutará un bloque de código, aunque técnicamente más de una condición pueda ser verdadera (por eso el orden importa).
3. **El final**: El `else` al final actúa como una "red de seguridad" para cualquier caso que no haya sido atrapado por las condiciones anteriores.
4. **No colocar** mas de 2 condiciones especificas (if, else-if) ya que una estructura excesivamente larga es contraproducente para el código.

4. Estructura `switch`: Decisiones basadas en valores exactos

A veces, tenemos que comparar una sola variable contra muchos valores posibles. Si usáramos muchos `else if`, el código se volvería difícil de leer. El `switch` es una alternativa más elegante y ordenada para estos casos.

¿Cuándo usar switch?

Úsalo cuando tengas una variable y quieras ejecutar diferentes acciones dependiendo de su **valor exacto**.

Ejemplo: Menú de una cafetería

CÓDIGO

```
let pedido = "Capuchino"

switch (pedido) {
    case "Espresso":
        console.log("El precio es $2.00. Marchando un café fuerte.")
        break
    case "Capuchino":
        console.log("El precio es $3.50. Preparando espuma de leche.")
        break
    case "Latte":
        console.log("El precio es $3.00. Mezclando café y leche.")
        break
    default:
        console.log("Lo siento, no tenemos ese producto en el menú.")
}
```

Componentes clave del `switch`:

- **`case`** : Cada uno de los valores posibles que queremos comparar.
- **`break`** : Le dice a JavaScript: "Ya encontraste lo que buscabas, sal del switch". Si olvidas el `break`, JavaScript seguirá ejecutando todos los casos siguientes, aunque no coincidan.
- **`default`** : Es el equivalente al `else`. Se ejecuta si ninguno de los casos anteriores coincidió.

6. Errores comunes y mejores prácticas

1. Confundir asignación (=) con comparación (==)

Este es el error número uno.

- `if (edad = 18)` -> **MAL**. Estás intentando cambiar el valor de la edad a 18.
- `if (edad == 18)` -> **BIEN**. Estás preguntando si la edad es igual a 18.

2. Olvidar el `break` en un `switch`

Si olvidas el `break`, se producirá lo que llamamos "fall-through". El programa ejecutará el caso correcto y también todos los que vienen después del correcto hasta encontrar un break o terminar el switch.

3. El orden de las condiciones en `else if`

Si pones el caso más general primero, los casos específicos nunca se ejecutarán.

CÓDIGO

```
// ERROR DE LÓGICA
let nota = 9
if (nota > 5) {
    console.log("Aprobado")
} else if (nota > 8) {
    console.log("Excelente") // Esto NUNCA se ejecutará porque 9 ya entró en nota > 5
}
```

Regla: Ve siempre de lo más específico a lo más general.

Comparaciones y condiciones compuestas

Aprende a combinar múltiples condiciones y comparaciones para crear una lógica de flujo robusta en JavaScript utilizando operadores lógicos y de igualdad estricta.

En la unidad anterior aprendimos a usar `if` y `else` para tomar decisiones. Pero, ¿qué pasa cuando esas decisiones dependen de varios factores a la vez? Por ejemplo, para entrar a una aplicación, no solo necesitas un usuario correcto, sino también una contraseña válida y, quizás, no tener la cuenta bloqueada.

Para resolver esto, JavaScript nos ofrece los **operadores de comparación** y los **operadores lógicos**.

1. Operadores de Comparación: Hacer las preguntas correctas

Los operadores de comparación comparan dos valores y devuelven un booleano (`true` o `false`). Son como "preguntas" que le hacemos al código:

- \=== **Igualdad estricta**: ¿Son el valor y el tipo idénticos? (Ej: 5 === 5 es `true`, pero 5 === "5" es `false`).
- !== **Desigualdad estricta**: ¿Son diferentes en valor o tipo?
- `>` y `<` **Mayor y menor que**: Comparaciones numéricas básicas.
- = y <= **Mayor/menor o igual**: Incluyen el valor límite en la comparación.

2. Operadores Lógicos: Combinando respuestas

A veces una sola comparación no basta. Queremos combinar varias condiciones:

OR (||) - "Al menos uno debe servir"

El resultado es `true` si **al menos una** de las condiciones es verdadera.

- *Ejemplo:* `(pagaConEfectivo || pagaConTarjeta)` -> Puedes comprar si tienes efectivo O si tienes tarjeta.

AND (&&) - "Todo debe ser verdad"

Para que el resultado sea `true`, **todas** las condiciones deben cumplirse.

- *Ejemplo:* (edad >= 18 && tieneentra === true) -> Solo entras si eres mayor de edad Y tienes entrada.

NOT (`!`) - "El gran inversor"

Invierte el valor de una condición. Si algo es `true`, lo vuelve `false`, y viceversa.

- *Ejemplo:* `!estaAusente` -> Es verdad si el usuario **no** está ausente.

3. El concepto de Cortocircuito

JavaScript es eficiente. En una cadena de `&&`, si encuentra un `false` al principio, deja de leer el resto porque ya sabe que el resultado final será `false`. Lo mismo pasa con `||`: si el primero es `true`, ya no evalúa los demás.

> **Aviso**
>
> **Un error común:** Olvidar que las comparaciones de texto distinguen entre mayúsculas y minúsculas. "Admin" === "admin" devolverá `false`. Siempre ten en cuenta el tipo de dato que estás comparando.

Repetición con for, while y do...while

Aprende a automatizar tareas repetitivas utilizando bucles for, while y do...while para controlar el flujo de tus programas en JavaScript.

¿Alguna vez has tenido que escribir la misma línea de código 10 veces? ¿O has necesitado procesar una lista de 1,000 productos en un carrito de compras? Hacer esto manualmente es imposible. Para eso existen los **bucles (loops)**.

Un bucle es una estructura que nos permite repetir un bloque de código tantas veces como sea necesario. Es la base de la automatización en programación.

1. El bucle `for`: El contador preciso

El bucle `for` se utiliza cuando **sabes de antemano cuántas veces** quieres que se ejecute el código.

Estructura:

- **Inicialización:** Se crea una variable (generalmente nombrada `let i`).
- **Condición:** El bucle sigue mientras esto sea verdad.
- **Incremento:** Qué pasa al final de cada vuelta (ej. sumar 1 a `i`).

CÓDIGO

```
for (let i = 0; i < 5; i++) {
    console.log("Vuelta número " + i)
}
```

2. El bucle `while`: El vigilante

Se usa cuando **no sabes exactamente cuántas vueltas** dará el bucle, sino que depende de una condición que puede cambiar en cualquier momento.

Piénsalo como: "Mientras (while) la batería sea mayor a 0, sigue encendido".

> **Aviso**
>
> **¡Cuidado!** Si la condición nunca se vuelve falsa, crearás un **bucle infinito** y tu programa se trabará. Asegúrate siempre de modificar la variable de control dentro del bucle.

CÓDIGO

```
let login = false
while (login === false) {
    let password = prompt("Ingrese la contraseña para ingresar")
    if(password == 123456) {
      console.log("Contraseña correcta, ingresando...")
      login == true
    } else {
        console.log("Contraseña incorrecta, reintentar")
    }
}
```

3. El bucle `do...while`: Ejecuta primero, pregunta después

Es casi igual al `while`, con una diferencia clave: el código dentro del `do` se ejecutará **al menos una vez**, sin importar si la condición es verdadera o falsa al inicio.

CÓDIGO

```
let password;
do {
    // Esto se ejecuta al menos una vez aunque password esté vacío
    password = "123"
    console.log("Validando...")
} while (password !== "123")
```

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 2: Lógica de Programación y Control de Flujo

Guía detallada paraentregable, enfocada en consolidar la lógica de programación mediante estructuras de control, bucles y saltos de flujo en JavaScript.

¡Felicidades por llegar al final del segundo módulo! Hasta ahora, has aprendido a declarar variables, interactuar con el usuario y, lo más importante, a tomar decisiones con el código y repetir tareas automáticamente.

Este entregable es un **paso intermedio fundamental**, ya que es la base lógica sobre la que construiremos aplicaciones más complejas. En esta etapa, nos aseguraremos de que dominas la "columna vertebral" de cualquier programa: la capacidad de controlar exactamente qué instrucciones se ejecutan y cuántas veces.

El objetivo

El propósito de esta pre-entrega es consolidar el uso de **bucles (loops)** y **condicionales**. Un programador no solo escribe código que funciona, sino código que sabe cuándo detenerse, cuándo saltar pasos innecesarios y cómo manejar diferentes escenarios lógicos.

¿Qué construirás?

Vas a desarrollar un **Algoritmo de Interacción Lógica**. Este script debe ser capaz de procesar una serie de entradas (datos) del usuario, evaluarlas mediante condiciones y repetir procesos hasta que se cumpla un stop específico.

Instrucciones del Checkpoint

1. Estructura de Decisión (Condicionales)

Debes usar `if`, `else if` y `else` para bifurcar el flujo. Por ejemplo: validar si un número es mayor que otro, si un string coincide con una clave, o si un usuario tiene permisos. *Opcional:* Puedes usar un `switch` si tienes múltiples opciones fijas (como un menú de selección).

2. Estructura de Repetición (Bucles)

Debes implementar al menos un bucle (`for`, `while` o `do...while`).

- Usa `for` si conoces la cantidad de veces que quieres repetir algo (ej: mostrar los números del 1 al 10).
- Usa `while` si la repetición depende de una condición que el usuario puede cambiar (ej: repetir hasta que el usuario escriba "ESC").

- Usá `do...while` si querés que el bloque se ejecute **al menos una vez** antes de chequear la condición (ej: pedir un dato con `prompt` y repetir hasta que sea válido). Si no lo viste en la clase en vivo, repasalo en la unidad **"Repetición con for, while y do...while"** de este mismo módulo: está explicado con ejemplos, errores comunes y quiz.

Criterios de Aceptación

El repositorio debe:

1. **Funcionar sin errores:** No debe haber errores de sintaxis al abrir el archivo.
2. **Combinar estructuras:** Debe existir al menos un bucle que contenga un condicional dentro.
3. **Interactividad:** El script debe interactuar con el usuario mediante `prompt` o `console.log`.
4. **Desarrollo con sentido:** No deben ser herramientas sueltas, el script debe funcionar en un contexto de simulador.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Así se ve el entregable (ejemplo de referencia)

Estructura de archivos esperada:

CÓDIGO

```
mi-simulador/
├── index.html   # enlaza el script: <script src="main.js" defer></script>
└── main.js      # toda tu lógica de control de flujo
```

Ejemplo de `main.js` (adaptalo a tu propio simulador — este es un control de acceso con reintentos):

CÓDIGO

```
// Simulador: control de acceso con reintentos
const CLAVE = "coder";
let intentos = 0;
let acceso = false;

// BUCLE: repite hasta 3 intentos o hasta acertar
while (intentos < 3 && !acceso) {
  const ingreso = prompt("Ingresá la clave:");

  // CONDICIONAL dentro del bucle
  if (ingreso === CLAVE) {
    acceso = true;
    console.log("   Acceso concedido");
  } else {
    intentos++;
    console.log(`   Clave incorrecta. Intentos restantes: ${3 - intentos}`);
  }
}

// Resultado final coherente al terminar el bucle
if (!acceso) {
  console.log("   Cuenta bloqueada por demasiados intentos");
}
```

Fijate que hay un **bucle (`while`) que contiene un condicional (`if/else`)** — exactamente lo que pide el criterio "Combinar estructuras".

Checklist de entrega

- [ ] Repo público de GitHub con `index.html` + `main.js` vinculados con `defer`. *(→ Estructura del repositorio, 25%)*
- [ ] Al menos un bucle (`for` / `while` / `do...while`) con un condicional adentro. *(→ Uso correcto de bucles, 25%)*
- [ ] Variables declaradas con `const` y `let` + condicionales `if/else` (o `switch`). *(→ Variables y condicionales, 25%)*
- [ ] Interacción con `prompt` y salida por `console.log` / `alert` . *(→ Salida final, 10%)*
- [ ] Todo dentro de un contexto de "simulador" con sentido, no instrucciones sueltas. *(→ Planteamiento del problema, 15%)*

Instrucciones para la Pre-entrega

1. **Estructura de archivos: Crea una carpeta para tu proyecto con un archivo index.html y un archivo main.js en su correspondiente carpeta. Vincula el script en la etiqueta head del HTML con el atributo defer.**

2. **Definición del Problema: Situate en una parte del simulador elegido a desarrollar que requiera repetición y decisiones (ej: un contador de puntos de un juego, un validador de stock, o un calculador de promedio de notas).**
3. **Implementación de Lógica:**

- **Declara las variables necesarias (usa `const` y `let`).**
- **Crea mínimo bucle para interactuar con el usuario.**
- **Usar un `if/else` para evaluar esos datos. Si la estructura es extensa debe mutar a switch.**

1. **Salida de Resultados: Muestra por consola o mediante un alert toda comunicación necesaria para el flujo del script.**
2. **Entrega: Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.**

Cómo se evalúa esta entrega

Rúbrica para evaluar una pre-entrega del nivel Beginner centrada en la creación de un repositorio de GitHub con un archivo HTML enlazado a un script JavaScript que implemente lógica de control de flujo con bucles, condicionales, uso de break/continue, salida de resultados y comentarios explicativos. Esta rúbrica valora el avance verificable y la claridad de la solución, sin exigir aún estructuras avanzadas como funciones u objetos.

Criterio	Descripción	Peso
Estructura del repositorio y enlace entre archivos	Evalúa si la entrega está organizada en un repositorio de GitHub y si contiene los archivos solicitados con la vinculación correcta entre HTML y JavaScript, permitiendo ejecutar el script desde el navegador.	25%
Planteamiento del problema y lógica secuencial	Evalúa si el estudiante elige una situación adecuada y construye una lógica coherente con repetición y decisiones, sin herramientas sueltas sin sentido.	15%
Uso de variables y condicionales	Evalúa si se declaran correctamente las variables necesarias con `const` y `let` y si se emplean condicionales (if/else, switch) para la organización del flujo de la informacion.	25%
Uso correcto de bucles	Evalúa la aplicación correcta de bucles (while/do-while/for) en un ambito correcto para la ejecución del simulador.	25%
Salida final y evidencia del resultado	Evalúa si, al terminar el bucle, el programa muestra por consola o mediante `alert` un resultado final coherente con la operación lógica realizada.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 3

Funciones

Modularizar código mediante la creación y ejecución de funciones, comprendiendo el concepto de scope y parámetros.

Declaración y expresión de funciones

Aprende las dos formas fundamentales de crear funciones en JavaScript (declaración y expresión), comprendiendo el concepto de hoisting y las mejores prácticas para elegir entre ellas.

Imagina que estás construyendo una aplicación de cocina. Tienes un libro de recetas físico guardado en el estante y, además, tienes pequeñas notas adhesivas pegadas en la nevera con instrucciones rápidas. En la programación con JavaScript, las funciones funcionan de manera similar: son bloques de código reutilizables, pero la forma en que las "guardas" o las "anuncias" en tu programa determina cómo y cuándo puedes usarlas.

1. ¿Por qué necesitamos funciones?

Antes de entrar en la sintaxis, recordemos el "porqué". Una función es un bloque de código diseñado para realizar una tarea específica. En el Módulo anterior, aprendiste a usar ciclos `for` y condicionales `if`. Si quisieras repetir esa lógica en diferentes partes de tu código, tendrías que copiar y pegar.

Las funciones resuelven esto permitiéndote:

1. **Reutilizar código**: Escribes la lógica una vez y la usas mil veces.
2. **Modularizar**: Divides un problema grande en piezas pequeñas y manejables.
3. **Abstraer**: Puedes usar una función como `enviarEmail()` sin necesidad de recordar cada línea de código interna cada vez que la llamas.

2. Declaración de Función (La forma clásica)

La **función declarada** es la manera más tradicional y directa de crear una función. Es como un anuncio oficial en tu código.

Sintaxis

Para declarar una función, usamos la palabra clave reservada `function`, seguida de un nombre, paréntesis para los parámetros y llaves para el cuerpo del código.

CÓDIGO

```
// Declaración de función
function saludar(nombre) {
  console.log("Hola, " + nombre + "!")
}
```

Características principales:

- **Nombre obligatorio**: Al ser una declaración independiente, debe tener un nombre para poder referenciarla.
- **Hoisting (Elevación)**: Esta es su característica más especial. JavaScript "mueve" virtualmente estas declaraciones al principio del código antes de ejecutarlo.
- **Claridad**: Son fáciles de identificar a simple vista como "herramientas" disponibles en todo el archivo.

3. Expresión de Función (Funciones como valores)

En JavaScript, las funciones son tratadas como **ciudadanos de primera clase**. Esto significa que pueden ser tratadas como cualquier otro valor (como un número o un texto) y, por lo tanto, pueden ser asignadas a una variable.

A esto le llamamos **función expresada**.

Sintaxis

Aquí, creamos la función y la asignamos a una variable (normalmente usamos `const` para que la referencia a la función no cambie).

CÓDIGO

```
// Expresión de función
const despedirse = function(nombre) {
  console.log("Adiós, " + nombre)
};
```

Características principales:

- **Pueden ser anónimas**: Como puedes ver arriba, la función después del signo `=` no tiene nombre propio; se le conoce por el nombre de la variable `despedirse`.
- **No tienen Hoisting de la misma manera**: No puedes llamar a esta función antes de la línea donde la definiste.

4. El concepto de Hoisting: El "Truco de Magia" de JavaScript

Funcion declarada

Puedes llamar a la función *antes* de que aparezca escrita en tu archivo:

CÓDIGO

```
// Esto funciona perfectamente
presentarse()

function presentarse() {
  console.log("Hola, soy una función declarada.")
}
```

Esto **no es recomendado** ya que llamar a una función antes de declararla arruina la sintaxis y el orden de tu código, dificultando la lectura y organización del mismo

Funcion expresada

Si intentas hacer lo mismo con una expresión, el programa lanzará un error:

CÓDIGO

```
// Esto causará un ERROR: "Uncaught ReferenceError: Cannot access 'cocinar' before initializa
cocinar();

const cocinar = function() {
  console.log("Cocinando...");
};
```

Parámetros, argumentos y retorno de valores

Aprende a pasar datos a tus funciones mediante parámetros y a recuperar resultados procesados utilizando la sentencia return.

Para que las funciones sean realmente poderosas, necesitan poder comunicarse con el resto del programa. En esta unidad aprenderemos a enviarles datos y a recibir respuestas de ellas.

1. Parámetros y Argumentos: El "Input"

Imagina una cafetera. La cafetera tiene una ranura (parámetro) donde esperas granos de café. El café real que tú depositas hoy (argumento) es lo que determina el sabor.

- **Parámetros:** Son las variables que definimos en la declaración de la función. Son como "huecos" o etiquetas.
- **Argumentos:** Son los valores reales que le pasamos a la función cuando la llamamos.

CÓDIGO

```
function saludar(nombre) { // 'nombre' es el parámetro
  console.log("Hola, " + nombre)
}

saludar("Julieta") // "Julieta" es el argumento
```

2. El Retorno de Valores: El "Output"

Hasta ahora, nuestras funciones hacían cosas (como imprimir en consola), pero el programa no "recuperaba" nada de ellas. Para que una función nos devuelva un resultado que podamos guardar en una variable, usamos la palabra clave `return`.

CÓDIGO

```
function duplicar(numero) {
  return numero * 2 // Envía el resultado hacia afuera
}

let miResultado = duplicar(5) // miResultado ahora vale 10
```

> **Importante**
>
> **Regla de oro:** Cuando el motor de JavaScript encuentra un `return`, la función se detiene inmediatamente. Nada de lo que escribas debajo de un `return` se ejecutará.

3. Funciones sin retorno explícito

¿Qué pasa si una función no tiene `return`? JavaScript, por defecto, hace que esa función devuelva `undefined`. Estas funciones se usan generalmente para causar un "efecto secundario" (*side effect*), como escribir en la pantalla o modificar una base de datos, en lugar de calcular un valor.

4. Función flecha: la evolución de la función expresada

¿Qué son las Funciones Flecha?

Introducidas en ES6, las funciones flecha ofrecen una sintaxis más concisa para escribir funciones anónimas. Tienen un return implícito, lo que las hace ideales para callbacks y programación funcional.

Ejemplo:

CÓDIGO

```
const suma = (a, b) => a + b
console.log(suma(5, 3)) // Salida: 8
```

Scope: ámbito global y local

Aprende a identificar dónde viven tus variables y quién tiene permiso para verlas, dominando los conceptos de ámbito global y local para escribir código más limpio y seguro.

Imagina que estás escribiendo un diario personal. Tienes una libreta física donde anotas tus pensamientos. Esas notas son **locales** a tu libreta: solo tú puedes verlas y nadie más en la calle sabe qué escribiste. Sin embargo, hay cosas que son **globales**, como el clima: si está lloviendo, está lloviendo para ti, para tu vecino y para cualquier persona en la ciudad. Todos pueden "leer" ese dato.

En JavaScript, el **Scope** (o alcance) funciona exactamente así. Determina la visibilidad y accesibilidad de tus variables. Entender el scope es la diferencia entre un código que funciona de forma predecible y uno lleno de errores extraños donde las variables parecen cambiar de valor "por arte de magia" o simplemente "no existen" cuando intentas usarlas.

1. ¿Qué es el Scope?

El **Scope** es el conjunto de reglas que definen dónde puede ser alcanzada una variable dentro de tu código. Si declaras una variable, el scope dicta en qué partes de tu programa esa variable es "visible" y puede ser utilizada.

2. El Scope Global: El "Clima" de tu Código

El **Scope Global** es el nivel más externo de tu programa. Cualquier variable declarada fuera de una función pertenece al ámbito global.

¿Por qué importa el Scope Global?

Una variable global está disponible para **todo** el script. Puede ser leída y modificada por cualquier función o bloque de código que escribas después de su declaración.

CÓDIGO

```
// Variable en Global Scope
const nombreAplicacion = "Mi Super App"

function mostrarBienvenida() {
    // Podemos acceder a la variable global desde aquí adentro
    console.log("Bienvenido a " + nombreAplicacion)
}

mostrarBienvenida() // Imprime: Bienvenido a Mi Super App
```

El peligro de los Globales

Aunque parece cómodo tener variables que todos pueden ver, el exceso de variables globales es una mala práctica conocida como "contaminación del scope global".

1. **Conflictos de nombres:** Si tienes un programa grande y muchas variables globales, podrías intentar declarar dos cosas con el mismo nombre accidentalmente.
2. **Dificultad de mantenimiento:** Si una variable global cambia su valor, es difícil rastrear qué parte del código la modificó, ya que *cualquiera* pudo haberlo hecho.

3. El Scope Local: Tu "Habitación Privada"

El **Scope Local** se crea cada vez que defines una función o un bloque (llaves `{}`). Las variables declaradas allí solo existen "dentro" de ese espacio. Una vez que la ejecución termina y se sale del bloque, esas variables dejan de existir para el resto del programa.

Scope de Función

Cuando declaras una variable dentro de una función, solo esa función puede verla.

CÓDIGO

```
function calcularImpuesto(monto) {
    let tasa = 0.21 // Variable LOCAL a la función
    return monto * tasa
}

console.log(calcularImpuesto(100)) // Funciona perfectamente: 15
console.log(tasa) // ERROR: Uncaught ReferenceError: tasa is not defined
```

En el ejemplo anterior, `tasa` es una variable local. Intentar usarla fuera de la función es como intentar usar el cepillo de dientes de alguien que vive en otra casa: simplemente no tienes acceso a él.

4. Scope de Bloque: El Poder de `let` y `const`

Desde la llegada de ES6 (la versión moderna de JavaScript), tenemos el **Scope de Bloque**. Un bloque es cualquier cosa que esté rodeada por llaves `{}`, como un `if`, un `for` o un `while`.

Las variables declaradas con `let` o `const` dentro de un par de llaves solo son visibles dentro de esas llaves.

CÓDIGO

```
if (true) {
    const saludo = "Hola, estoy atrapado en el bloque";
    console.log(saludo) // Funciona
}

console.log(saludo) // ERROR: saludo no está definido
```

5. Acceso y Aislamiento de Datos

Una regla de oro en JavaScript es: **El código puede mirar "hacia afuera" (hacia scopes superiores), pero no puede mirar "hacia adentro" (hacia scopes inferiores).**

La Jerarquía del Scope (Lexical Scope)

JavaScript utiliza algo llamado *Lexical Scope*. Esto significa que las funciones anidadas tienen acceso a las variables declaradas en su scope exterior.

CÓDIGO

```
const usuario = "Juan" // Global

function exterior() {
    const mensaje = "Hola" // Local de exterior

    function interior() {
        // La función interior tiene acceso a su propio scope,
        // al scope de 'exterior' y al scope global.
        console.log(mensaje, usuario)
    }

    interior()
}

exterior() // Imprime: "Hola, Juan"
```

El concepto de Shadowing (Sombreado)

¿Qué pasa si declaras una variable local con el mismo nombre que una global? La variable local "sombrea" a la global dentro de ese scope.

CÓDIGO

```
const color = "azul" // Global

function cambiarColor() {
    const color = "rojo" // Local (Shadowing)
    console.log(color) // Imprime "rojo"
}

cambiarColor()
console.log(color) // Imprime "azul" (la global no fue modificada)
```

6. Síntesis y Resumen

- **Scope Global:** Variables declaradas fuera de cualquier función o bloque. Accesibles desde cualquier parte.

- **Scope Local / de Función:** Variables declaradas dentro de una función. Solo existen mientras la función se ejecuta.
- **Scope de Bloque:** Variables (`let` , `const`) declaradas dentro de `{ }` . Solo existen dentro de esas llaves.
- **Transparencia:** Las funciones internas pueden ver variables de sus padres, pero los padres no pueden ver las variables de sus hijos.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 3: Funciones e Integración de Lógica

Aprende a dominar las funciones declaradas, flecha de ES6+ y consolida tus conocimientos de modularización.

¡Felicidades! Has llegado al final del módulo de **Funciones**. Hasta ahora, has aprendido a declarar funciones, a pasarles información mediante parámetros y a obtener resultados con el túnel de `return`. También has descubierto que las variables tienen "hogares" (scopes) y que no siempre podemos acceder a ellas desde cualquier lugar.

El objetivo

El propósito de esta pre-entrega es consolidar el uso defunciones, aplicando todo lo visto en las unidades para una correcta implementación de funciones declaradas, expresadas y funciones flecha. Es muy importante tener presente cuando utilizar el return de una función y un uso correcto de parámetros para generar una interacción completa.

¿Qué construirás?

Vas a continuar desarrollando tu simulador, implementando minimo tres funciones con parámetros para optimizar y modularizar las operaciones en tu script, de manera que se puedan reciclar las estructuras correctamente.

Criterios de Aceptación

El repositorio debe:

1. Crea **al menos 3 funciones** siguiendo el algoritmo básico de todo programa (entradas de datos, procesamiento de datos y mostrar los resultado de salida de los mismos.)
2. **Uso de parámetros:** Ingresar información para procesarla dentro de la funcion mediante el uso de parámetros y argumentos en todas las funciones**.**
3. **Uso de return:** Al menos una funcion debe retornar un valor para utilizarse en el algoritmo**.**
4. **Función flecha:** Al menos una funcion debe ser del tipo flecha, optimizando un proceso simple**.**
5. **Realizar llamadas(invocar)** a las funciones que utilizaste en tu simulador
6. Agrega los **ciclos de iteración y condicionales** necesarios, para que tu proyecto funcione correctamente.
7. La interacción del simulador se hará íntegramente sobre la Consola y Alert, con los datos previamente solicitados mediante prompt.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

El Escenario

Debes continuar el desarrollo de tu simulador, implementando el uso de funciones para mejorar y modularizar procesos en tu script para simplificar la lógica y la reutilización de fragmentos de código sin la necesidad de repetirlos.

Criterios de Aceptación

El repositorio debe:

1. Crea **al menos 3 funciones** siguiendo el algoritmo básico de todo programa (entradas de datos, procesamiento de datos y mostrar los resultado de salida de los mismos.)
2. **Uso de parámetros:** Ingresar información para procesarla dentro de la funcion mediante el uso de parámetros y argumentos en todas las funciones**.**
3. **Uso de return:** Al menos una funcion debe retornar un valor para utilizarse en el algoritmo**.**
4. **Función flecha:** Al menos una funcion debe ser del tipo flecha, optimizando un proceso simple**.**
5. **Realizar llamadas(invocar)** a las funciones que utilizaste en tu simulador
6. Agrega los **ciclos de iteración y condicionales** necesarios, para que tu proyecto funcione correctamente.
7. La interacción del simulador se hará íntegramente sobre la Consola y Alert, con los datos previamente solicitados mediante prompt.

Método de entrega

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Rúbrica para evaluar una pre-entrega del módulo de Funciones en JavaScript desde cero. Se valora la creación de un pequeño simulador de procesamiento de datos que use correctamente una función declarada, una expresión de función y una función flecha, con paso de datos entre funciones, reutilización y salida por consola. Esta rúbrica se centra en el alcance del checkpoint y en criterios observables dentro del repositorio entregado.

Criterio	Descripción	Peso
Uso correcto de funciones	Evalúa si el código identifica y aplica correctamente tres funciones con parámetros, con mínimo una función declarada y una función flecha, respetando la sintaxis y el propósito de cada una.	30%
		20%

Criterio	Descripción	Peso
Integración y flujo de datos entre funciones	Evalúa si el resultado de una función se utiliza como entrada de la siguiente, logrando un pequeño flujo de procesamiento encadenado y coherente utilizando el return.	
Uso de condicionales y bucles	Evalúa si el código presenta tambien uso correcto de condicionales y bucles para mejorar la estructura del script.	15%
Reutilización, ausencia de repetición y claridad del código	Evalúa si la lógica se expresa mediante funciones reutilizables, evitando código repetido, y si la solución mantiene una estructura simple y fácil de entender para el nivel.	15%
Scope, uso de const y let, y ejecución sin errores	Evalúa el uso adecuado de `const` y `let` según el alcance de las variables, así como la ausencia de errores en consola que impidan la ejecución del script.	15%
HTML y JS correctos	Proyecto subido correctamente en un repositorio de github, con el archivo js correctamente enrutado en el head del html mediante el atributo defer.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 4

Arrays

Utilizar arrays para gestionar colecciones de datos y aplicar métodos de manipulación de listas.

Arrays: Lista de elementos

Aprende a gestionar colecciones de datos utilizando Arrays en JavaScript, dominando el sistema de índices

Imagina que estás desarrollando una aplicación para una cafetería. Tienes una lista de pedidos que llegan a la cocina. Cada vez que un cliente pide algo, ese pedido se anota al final de una lista. Cuando el chef termina un plato, lo tacha de la lista. En programación, para gestionar este tipo de "colecciones" de datos de forma ordenada, utilizamos una de las estructuras más potentes y comunes: el **Array**.

1. ¿Qué es un Array?

Hasta ahora, has trabajado con variables que guardan un solo valor: `let nombre = "Juan"` o `let edad = 25`.

Pero, ¿qué pasa si quieres guardar los nombres de 50 estudiantes? Crear 50 variables individuales (`estudiante1`, `estudiante2`, etc.) sería una pesadilla de mantenimiento. Aquí es donde entran los **Arrays**.

Un **Array** es una estructura de datos que nos permite almacenar múltiples valores en una sola variable. Es como una caja con compartimentos numerados donde cada compartimento guarda un dato.

Sintaxis básica

Para crear un array, usamos corchetes `[]` y separamos los elementos por comas:

CÓDIGO

```
// Un array de strings (nombres de frutas)
const frutas = ["Manzana", "Pera", "Banana"]

// Un array de números
const numeros = [10, 20, 30, 40]

// Un array vacío (listo para llenarse después)
const listaDeCompras = []
```

2. Índices y la propiedad `.length`

Para trabajar con arrays, debes entender dos reglas de oro sobre cómo JavaScript organiza estos datos:

A. Los índices empiezan en 0

En el mundo de la programación, empezamos a contar desde cero. El primer elemento de un array no está en la posición 1, sino en la **posición 0**.

CÓDIGO

```
const peliculas = ["Batman", "Superman", "Flash"]
// "Batman" está en el índice 0
// "Superman" está en el índice 1
// "Flash" está en el índice 2
```

B. La propiedad `.length`

Todos los arrays tienen una propiedad incorporada llamada `.length`, que nos dice cuántos elementos tiene el array en total.

CÓDIGO

```
console.log(peliculas.length) // Imprime: 3 (como tipo de dato Number)
```

Un error muy comun es confundir el índice con el conteo total. Si un array tiene 3 elementos (length: 3), el último índice siempre será `2` (es decir, `length - 1`). Intentar acceder a `peliculas[3]` nos devolverá `undefined` porque esa posición no existe.

3. Los Arrays como Estantes Numerados

Imagina que un array es como un estante con cajas numeradas. Para llegar a lo que necesitas, no tienes que vaciar todo el estante; solo necesitas saber el número de la caja. Aquí es donde entra el concepto de **Índice**.

1. Acceso por índice

En JavaScript, los arrays son de **índice cero (zero-indexed)**. Esto significa que empezamos a contar desde el 0:

- Posición 1 → Índice `0`
- Posición 2 → Índice `1`
- Posición 3 → Índice `2`

Para leer un valor, usamos los corchetes `[]` pegados al nombre de la variable:

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]
console.log(frutas[0]) // Imprime "Manzana"
console.log(frutas[1]) // Imprime "Pera"
```

2. Modificación por asignación

Lo más potente de los índices es que nos permiten **sobrescribir** datos. Si te equivocas al cargar un producto o si el precio cambia, puedes simplemente "pisar" el valor anterior usando el operador de asignación (`=`).

CÓDIGO

```
let stock = ["Laptop", "Celular"]
stock[1] = "Tablet" // El "Celular" desaparece y ahora hay una "Tablet"
console.log(stock) // ["Laptop", "Tablet"]
```

Acceso y modificación de elementos por posición

Aprende a localizar y actualizar información específica dentro de un array utilizando índices numéricos.

A. Modificando el final: `push` y `pop`

Casi siempre necesitaremos que nuestras listas cambien mientras el programa corre. Los métodos más utilizados son los que afectan al **final** del array.

`push()`: Agregar al final

El método `push` añade uno o más elementos al final de la lista.

CÓDIGO

```
let tareas = ["Lavar ropa", "Estudiar JS"]

tareas.push("Limpiar cocina")

console.log(tareas) // ["Lavar ropa", "Estudiar JS", "Limpiar cocina"]
console.log(tareas.length) // 3
```

`pop()`: Quitar del final

El método `pop` elimina el **último** elemento del array.

CÓDIGO

```
tareas.pop()

console.log(tareas) // ["Lavar ropa", "Estudiar JS"]
```

B. Modificando el inicio: `unshift` y `shift`

A veces, el orden de prioridad requiere que insertemos o quitemos elementos desde el **principio** del array.

`unshift()`: Agregar al inicio

Agrega uno o más elementos al comienzo de la lista, desplazando todos los demás hacia la derecha (incrementando sus índices).

CÓDIGO

```
let fila = ["Cliente 1", "Cliente 2"]
fila.unshift("Cliente VIP")

console.log(fila) // ["Cliente VIP", "Cliente 1", "Cliente 2"]
```

`shift()`: Quitar del inicio

Elimina el **primer** elemento del array y desplaza todos los demás hacia la izquierda (restando 1 a sus índices).

CÓDIGO

```
fila.shift()

console.log(fila) // ["Cliente 1", "Cliente 2"]
```

Cuadro comparativo para recordar:

Método	Acción	¿Dónde?
`push`	Agregar	Final
`unshift`	Agregar	Inicio
`pop`	Quitar	Final
`shift`	Quitar	Inicio

Interacción con el array

Con un array no solo podemos agregar, quitar o modificar información, también tenemos diferentes métodos que nos ayudan a buscar un elemento en particular, saber si un elemento está o no incluido en el array e inclusive reordenar alfabéticamente los elementos, ya que son herramientas de suma necesidad para interactuar con la información disponible.

`includes()`: Booleano

El método `includes` busca si un elemento existe o no en el array y en base a eso devuelve true (si existe) o false (si no existe)

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

console.log(frutas.includes("Pera")) // true
```

`indexOf()`: Buscador de índice

El método `indexOf` busca si un elemento existe o no en el array y en base a eso arroja el número de índice del mismo.

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

console.log(frutas.indexOf("Banana")) // 2
```

`sort()`: Ordenador de elementos

El método `sort` ordena todos los elementos del array de manera alfabética.

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

frutas.sort()
console.log(frutas) // ["Banana", "Manzana", "Pera"]
```

`reverse()`: Ordenador inverso

El método `reverse` ordena todos los elementos del array de manera inversa a la original.

CÓDIGO

```
console.log(frutas); // ["Banana", "Pera", "Manzana"]

frutas.reverse()
console.reverse(frutas) // ["Banana", "Pera", "Manzana"]
```

`join()`: Mejora la visualizacion

El método `join` añade strings entre los elementos del array, devolviendo todo en un solo string que ayuda a visualizar la información de manera mas amigable

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

console.log(frutas.join(" - ")) // Manzana - Pera - Banana
```

`splice()`: Booleano

El método `splice` se usa para **agregar, eliminar o reemplazar elementos** dentro de un array, modificando el array original.

Sintaxis

CÓDIGO

```
array.splice(inicio, cantidad, elemento1, elemento2, ...)
```

Parámetros

- `inicio` : posición donde comienza el cambio(indice del elemento).
- `cantidad` : cuántos elementos eliminar.
- `elemento1, elemento2...` : elementos opcionales para agregar.

Ejemplos

Eliminar elementos

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

frutas.splice(1, 1)
console.log(frutas) // ["Manzana", "Banana"]
```

Resultado: Elimina 1 elemento desde la posición 1 (`"banana"`).

Agregar elementos

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

frutas.splice(1, 0, "Frutilla")
console.log(frutas) // ["Manzana", "Pera", "Frutilla, "Banana"]
```

Resultado: No elimina nada (`0`) y agrega `"Frutilla"` desde la posición 1.

Reemplazar elementos

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Banana"]

frutas.splice(1, 1, "Frutilla")
console.log(frutas) // ["Manzana", "Frutilla, "Banana"]
```

Resultado: Reemplaza `"Pera"` por `"Frutilla"`.

Recorrer arrays con for y for...of

Aprende a recorrer colecciones de datos de forma eficiente utilizando bucles clásicos y modernos en JavaScript.

Imagina que tienes una lista de 100 productos en una tienda online. No querrías escribir `console.log` cien veces, ¿verdad? Para eso existen los bucles: herramientas que nos permiten ejecutar un bloque de código repetidamente sobre una colección de datos.

¿Qué es iterar?

Iterar es el término técnico para decir "recorrer". Cuando iteramos un array, visitamos cada elemento, uno por uno, desde el primero hasta el último, para realizar alguna acción (como mostrarlo en pantalla o modificarlo).

1. El `for` clásico: El control total

El bucle `for` convencional utiliza un **índice numérico** para acceder a los elementos. Es como tener un dedo que señala la posición en la lista y se mueve un paso a la vez.

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Uva"]

for (let i = 0; i < frutas.length; i++) {
    console.log("Posición " + i + ": " + frutas[i])
}
```

¿Por qué funciona así?

1. `let i = 0`: Empezamos en el índice 0 (el primer elemento).
2. `i < frutas.length`: El bucle sigue mientras `i` sea menor que el total de elementos.
3. `i++`: Después de cada vuelta, sumamos 1 a `i` para pasar al siguiente.

¿Cuándo usarlo? Cuando necesitas saber en qué posición estás o si quieres recorrer el array de forma especial (por ejemplo, saltando de dos en dos). Es un método **muy poco recomendado** en la actualidad ya que veremos diferentes herramientas mas adelante que facilitan este trabajo.

2. El `for...of`: Simplicidad y Legibilidad

Introducido en versiones modernas de JavaScript (ES6), el `for...of` es la forma más sencilla de leer todos los elementos de un array. Aquí no hay índices; JavaScript te entrega el **valor** directamente.

CÓDIGO

```
const frutas = ["Manzana", "Pera", "Uva"]

for (const fruta of frutas) {
    console.log("Hoy comeré: " + fruta)
}
```

Es el método **mas recomendado** para recorrer arrays de strings actualmente, ya que la simplicidad le da mucho mas orden y prolijidad al código que el for clásico con el length.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 4: Uso e interacción con Arrays

Entregable diseñado para integrar el uso de arrays, métodos de manipulación, iteración y búsqueda en tu proyecto.

¡Felicidades por llegar al final del módulo de Arrays! Hasta ahora, has aprendido a declarar variables, controlar el flujo con ciclos y modularizar tu lógica con funciones.Ahora sumamos una pieza fundamental: los **Arrays**, que nos permiten agrupar datos relacionados en una sola estructura.

¿Qué estamos construyendo?

En entregas anteriores, trabajamos en la lógica de decisión y en funciones reutilizables. Ahora, es momento de que esa lógica tenga **datos** sobre los cuales actuar.

Temas clave a validar

1. Manipulación de extremos

Debes ser capaz de gestionar una lista dinámica. No siempre agregamos elementos al final; a veces el orden de entrada importa.

- **`push` / `pop`** : Para trabajar con el final de la lista.
- **`unshift` / `shift`** : Para gestionar prioridades al inicio de la lista.

2. Acceso y Modificación por Índice

El array no es una caja negra. Cada elemento tiene una "dirección" (índice). Debes poder entrar a una posición específica (ej. `miArray[2]`) y cambiar su valor sin afectar al resto de la colección.

3. Iteración Eficiente

Recorrer la lista es la operación más común. Usaremos:

- **`for...of`** : La forma moderna y legible de leer cada elemento.

4. Búsqueda y Validación

Antes de operar con un dato, debemos saber si existe. Algunos métodos útiles pueden ser:

- **`includes()`** : Nos da un `true` o `false` rápido.
- **`indexOf()`** : Nos dice exactamente dónde está, o `-1` si no existe.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Consigna: Checkpoint de Arrays

En esta entrega, integrarás lo aprendido sobre arrays en la estructura de proyecto que venimos desarrollando.

Criterios de Aceptación:

- El código debe ser funcional y no arrojar errores en la consola.
- Se deben utilizar al menos 3 métodos de arrays vistos en el módulo.
- El uso de `for...of` debe estar correctamente implementado para recorrer la lista.
- Se debe continuar trabajando con las herramientas vistas hasta el momento (funciones, condicionales, ciclos) en un entorno de simulador, no pueden ser herramientas sueltas.

Pasos sugeridos:

1. **Preparación de la Colección:** * Crea un array con un nombre semántico asociado a tu proyecto. * Inicialízalo con al menos 5 elementos (strings o números).
2. **Manipulación Dinámica:** * Agrega un nuevo elemento al final del array usando `push`. * Agrega uno al principio usando `unshift`. * Elimina el último elemento y guárdalo en una variable para mostrarlo en un mensaje: `"Se ha eliminado el elemento: [nombre]"`.
3. **Búsqueda y Validación:** * Solicita al usuario (mediante `prompt` o una variable fija) el nombre de un elemento para buscar. * Usa `includes` para verificar si existe. * Si existe, usa `indexOf` para informar en qué posición (índice) se encuentra.
4. **Actualización por Índice:** * Busca un índice específico y cambia su valor por uno nuevo implementando el método `splice`.
5. **Reporte Iterativo:** * Crea una función para recorrer el array. * Dentro de la función, utiliza un bucle `for...of` para recorrer el array y mostrar cada elemento con un formato claro (ej: `"Producto: Manzana"`).

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Rúbrica para evaluar una pre-entrega del módulo de Arrays en un nivel inicial. Se valora la aplicación funcional de métodos básicos de arrays, la organización del repositorio y la claridad del código, priorizando que el estudiante demuestre avance verificable para continuar con el programa.

Criterio	Descripción	Peso
Estructura del proyecto y documentación mínima	Evalúa que el repositorio incluya la estructura solicitada para la pre-entrega: archivo index.html enlazado con main.js,en el head del documento, con el atributo defer.	15%

Criterio	Descripción	Peso
Creación y uso inicial del array	Evalúa que exista un array principal con nombre adecuado, inicializado con al menos 5 elementos. Este criterio verifica la base sobre la que se construyen las demás manipulaciones.	15%
Manipulación dinámica de extremos	Evalúa el uso correcto de mínimo 2 métodos para agregar y quitar elementos en los extremos del array: push, unshift, pop, shift.	20%
Búsqueda y validación con includes e indexOf	Evalúa que el programa solicite o use un valor de búsqueda, implementando includes o indexOf segun corresponda.	15%
Actualización por índice y recorrido con función	Evalúa que se modifique correctamente un elemento en un índice específico utilizando el metodo splice. Se debe implementar una funcion con un recorrido for...of que muestre cada elemento con formato claro. Este criterio integra acceso por posición e iteración de forma útil y legible.	20%
Recorrer el array y calidad de ejecución	Se debe implementar una función con un recorrido for...of que muestre cada elemento con formato claro. El código se debe ejecutar sin errores en consola, que utilice al menos 3 métodos de arrays del módulo y que el comportamiento general responda a lo pedido por la consigna, siempre en un contexto de simulador y no con herramientas sueltas.	15%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 5

Objetos

Crear y manipular objetos literales y constructores para representar entidades de datos complejas.

Objetos literales: propiedades y métodos

Aprende a representar entidades del mundo real en código utilizando objetos literales en JavaScript, organizando datos en propiedades y acciones en métodos.

Hasta ahora, has trabajado con variables para guardar datos sueltos (como un nombre o una edad) y con **Arrays** para guardar listas ordenadas de elementos. Pero, ¿qué sucede cuando necesitas representar algo más complejo, como un usuario de una aplicación, un producto de una tienda o un personaje de un videojuego?

Imagina que quieres representar un auto. Podrías crear variables sueltas: `let marca = "Toyota" let modelo = "Corolla" let year = 2022`

El problema es que estas variables están "sueltas" en tu código. No hay nada que le diga a JavaScript que esa marca y ese año pertenecen al mismo objeto. Aquí es donde entran los **Objetos Literales**.

1. ¿Qué es un Objeto Literal?

Un objeto es una estructura de datos que nos permite agrupar valores relacionados bajo un solo nombre. A diferencia de los arrays, donde accedemos a los elementos por su posición (índice 0, 1, 2...), en los objetos accedemos a los datos mediante **nombres** o **claves**.

Un **objeto literal** es la forma más directa de crear un objeto en JavaScript utilizando llaves `{}`.

La anatomía de un objeto

CÓDIGO

```
const usuario = {
    nombre: "Ana",
    edad: 25,
    ciudad: "Madrid"
}
```

En este ejemplo:

- `usuario` es el nombre de nuestro objeto.
- `nombre`, `edad` y `ciudad` son las **claves** (keys) o etiquetas.
- `"Ana"`, `25` y `"Madrid"` son los **valores** (values).

Cada combinación de clave y valor se denomina **propiedad**.

2. Propiedades: Los datos del objeto

Las propiedades son como variables que viven dentro del objeto. Pueden guardar cualquier tipo de dato que ya conoces: strings, números, booleanos, e incluso otros arrays u otros objetos.

Ejemplo: Un Producto de E-commerce

Imagina que estás programando una tienda. Un producto no es solo un nombre; es un conjunto de datos:

CÓDIGO

```
const producto = {
    id: "A152",
    nombre: "Auriculares Bluetooth",
    precio: 59.90,
    enStock: true,
    coloresDisponibles: ["Negro", "Blanco", "Azul"]
}
```

Aquí, hemos agrupado información heterogénea (un string, un número, un booleano y un array) en una sola entidad llamada `producto`. Esto hace que nuestro código sea mucho más organizado y fácil de entender.

3. Métodos: El comportamiento del objeto

A veces, no solo queremos que un objeto guarde datos, sino que también queremos que **haga algo**.

Cuando guardamos una función dentro de una propiedad de un objeto, esa función pasa a llamarse **método**. Los métodos representan las acciones que el objeto puede realizar.

Ejemplo: Un usuario que saluda

CÓDIGO

```
const persona = {
    nombre: "Carlos",
    saludar: function() {
        console.log("¡Hola! ¿Cómo estás?");
    }
}

// Para ejecutar el método, usamos paréntesis:
persona.saludar() // Imprime: ¡Hola! ¿Cómo estás?
```

La palabra clave `this`

¿Qué pasa si queremos que el objeto use sus propios datos dentro de un método? Por ejemplo, que Carlos diga su propio nombre. Para eso usamos la palabra clave `this`, que hace referencia al objeto mismo.

CÓDIGO

```
const persona = {
    nombre: "Carlos",
    presentarse: function() {
        console.log("Hola, mi nombre es " + this.nombre);
    }
}

persona.presentarse() // Imprime: Hola, mi nombre es Carlos
```

> **Importante**
>
> **Importante:** En este nivel, piensa en `this` como un dedo que señala al objeto en el que estás trabajando en ese momento.

4. Acceso a la información: Punto vs Corchetes

Para leer, modificar o crear propiedades en un objeto, tenemos dos herramientas principales:

A. Notación de punto (`.`)

Es la más común y fácil de leer. La usaremos siempre que sepamos el nombre de la propiedad de antemano.

CÓDIGO

```
console.log(producto.nombre) // "Auriculares Bluetooth"
producto.precio = 55.00      // Modificamos el valor
```

B. Notación de corchetes (`[]`)

Esta opción es más flexible. Es obligatoria en dos casos:

1. Si el nombre de la propiedad tiene espacios o caracteres especiales (lo cual **no** se recomienda, pero puede pasar).
2. **Si el nombre de la propiedad está guardado en una variable.**

CÓDIGO

```
const propiedadABuscar = "nombre";
console.log(producto[propiedadABuscar]); // "Auriculares Bluetooth"
// Nota: producto.propiedadABuscar daría undefined, porque busca literalmente "propiedadABusca
```

5. Manipulación de objetos: Agregar y Eliminar

Los objetos en JavaScript son dinámicos. Puedes cambiar su estructura después de haberlos creado.

Agregar propiedades

Simplemente asignamos un valor a una clave que no existe todavía:

CÓDIGO

```
const auto = { marca: "Fiat" };
auto.modelo = "Cronos"; // Ahora el objeto tiene dos propiedades
```

Eliminar propiedades

Usamos el operador `delete`:

CÓDIGO

```
const heroe = { nombre: "Batman", ciudad: "Gótica", identidadSecreta: "Bruce Wayne" };
delete heroe.identidadSecreta;
// Ahora la identidad secreta ha desaparecido del objeto por seguridad.
```

6. Errores comunes y mejores prácticas

1. **Confundir Objetos con Arrays:** Recuerda que los Arrays usan `[]` y se basan en orden/posición. Los Objetos usan `{}` y se basan en nombres/etiquetas. Usa un Array para una lista de nombres; usa un Objeto para los detalles de una persona.
2. **Olvidar las comas:** Entre cada propiedad de un objeto **debe** haber una coma. El olvido de la coma es una de las causas más frecuentes de errores de sintaxis en principiantes.
3. **Acceder a propiedades inexistentes:** Si intentas leer `usuario.telefono` y esa propiedad no existe, JavaScript no dará un error fatal, sino que te devolverá `undefined`. Verifica siempre tus nombres de propiedades.
4. **Uso de const:** Aunque definas un objeto con `const`, puedes modificar sus propiedades interiores (cambiar el nombre, agregar una edad). Lo que no puedes hacer es reasignar la variable a un objeto completamente nuevo.

Constructores y operador new

Aprende a usar funciones constructoras y el operador 'new' para crear múltiples objetos con la misma estructura pero diferentes datos de forma eficiente.

En la unidad anterior aprendimos a crear objetos de forma "manual" usando llaves `{}`. Pero, ¿qué pasa si en nuestra aplicación necesitamos crear cientos de usuarios, productos o mensajes? Escribirlos uno por uno sería ineficiente y propenso a errores. Aquí es donde entran los **Constructores** y el operador `new`.

¿Qué es un Constructor?

Imagina un constructor como un **molde** o un plano de construcción. El molde define qué forma tendrá el objeto (qué propiedades y métodos tendrá), pero no es el objeto en sí.

En JavaScript, un constructor es técnicamente una función, pero se distingue por dos cosas:

1. Su nombre suele empezar con **Mayúscula** (convención para indicar que es un constructor).
2. Se ejecuta exclusivamente usando la palabra clave `new`.

El papel de `this`

Dentro de un constructor, la palabra clave `this` es fundamental. Representa al **nuevo objeto que se está creando en ese preciso momento**.

CÓDIGO

```
function Auto(marca, modelo) {
  this.marca = marca,   // El objeto que estoy creando tendrá esta marca
  this.modelo = modelo, // El objeto que estoy creando tendrá este modelo
}
```

El operador `new`

El operador `new` es el "botón de encendido" de la fábrica de objetos. Cuando usas `new`, JavaScript realiza cuatro pasos automáticos:

1. Crea un nuevo objeto vacío.
2. Vincula `this` a ese nuevo objeto.
3. Ejecuta el código de la función constructora (llenando el objeto de datos).
4. **Devuelve automáticamente** el nuevo objeto creado.

CÓDIGO

```
const miAuto = new Auto("Toyota", "Corolla")
// miAuto ahora es: { marca: "Toyota", modelo: "Corolla" }
```

Instancias: Los "hijos" del molde

Cada objeto creado a partir de un constructor se llama **instancia**.

- `persona1` es una instancia de `Persona`.
- `persona2` es otra instancia de `Persona`.

Aunque comparten la misma estructura (el mismo molde), los datos dentro de cada instancia son independientes. Si cambias el nombre de `persona1`, `persona2` no se verá afectada.

Aclaración importante:

Conocer como funciona la sentencia `this` y como instanciar un nuevo objeto es fundamental, pero es importante saber que la función constructora **no se utiliza mas** en la actualidad, ya que su función fue absorbida por las clases (Class) que lo veremos en la próxima unidad.

Clases en ES6 y prototipos: la forma moderna de crear objetos

Aprende a utilizar la sintaxis de clases de ES6 para crear plantillas de objetos robustas en JavaScript moderno.

Imagina que estás desarrollando un videojuego. Tienes cientos de enemigos en pantalla. Todos tienen una posición, una cantidad de vida y la capacidad de atacar. Si tuvieras que crear cada uno manualmente usando objetos literales, tu código sería un caos repetitivo. Si usaras funciones constructoras (como vimos en la unidad anterior), el código funcionaría, pero a medida que el juego crece y los enemigos se vuelven más complejos (jefes, arqueros, guerreros), la sintaxis empezaría a volverse difícil de leer.

Aquí es donde entran las **Clases de ES6**. Introducidas en 2015, las clases no cambiaron la forma en que JavaScript funciona por dentro, pero sí cambiaron radicalmente cómo escribimos y organizamos nuestro código.

1. ¿Qué es una Clase en JavaScript?

Piensa en una **Clase** como el **plano de una casa**. El plano define cuántas ventanas tiene, dónde va la puerta y de qué color son las paredes. Sin embargo, no puedes vivir dentro de un plano. Para tener algo real, necesitas construir la casa usando ese plano. Ese proceso se llama **instanciación**, y el resultado (la casa real) es el **objeto** o **instancia**.

2. La Estructura de una Clase: `constructor()` y Métodos

Para declarar una clase, usamos la palabra clave `class`. Veamos cómo se ve una clase simple para representar a un `Usuario` en un sistema.

CÓDIGO

```
class Usuario {
  // El constructor es el primer método que se ejecuta al crear el objeto (la función construc
  constructor(nombre, email, edad) {
    this.nombre = nombre,
    this.email = email,
    this.edad = edad,
  }

  // Método de instancia: lo que el objeto "sabe hacer"
  saludar() {
    console.log("Hola, mi nombre es "+ this.nombre + " y mi correo es " + this.email)
  }
}

// Creando una instancia (un objeto real a partir del plano)
const usuario1 = new Usuario("Ana", "ana@example.com", 25)
const usuario2 = new Usuario("Pedro", "pedro@dev.com", 15)

usuario1.saludar() // "Hola, mi nombre es Ana..."
```

Elementos clave:

1. **`class Usuario`** : Define el nombre de nuestra plantilla. Por convención, siempre empieza con Mayúscula (PascalCase).
2. **`constructor()`** : Es una función especial. Se invoca automáticamente cuando escribes `new Usuario(...)` . Su trabajo es "setear" o inicializar las propiedades del objeto usando `this` (como antiguamente lo realizaba la función constructora).
3. **Métodos**: Son funciones que viven dentro de la clase. Nota que aquí no necesitamos la palabra `function` ni flechas `=>` . Se escriben directamente.
4. **`this`** : Es la referencia al objeto que se está creando en ese momento. Cuando hacemos `usuario1.saludar()` , la palabra `this.nombre` dentro de la clase se traduce mágicamente a "Ana".

3. El error más común: Olvidar el `new`

En las funciones comunes, si olvidas el `new` , JavaScript a veces crea variables globales por accidente. Con las clases, JavaScript es más estricto.

CÓDIGO

```
class Animal {}

// Error frecuente
const miPerro = Animal()
// TypeError: Class constructor Animal cannot be invoked without 'new'
```

Las clases están protegidas: **siempre** debes usar la palabra clave `new` .

4. Aplicaciones en el Mundo Real

¿Dónde verás esto en tu carrera?

1. **Desarrollo de Videojuegos:** Para representar personajes, enemigos, proyectiles y niveles.
2. **Sistemas de E-commerce:** Clases para `Producto`, `CarritoDeCompras`, `OrdenDePago`.
3. **Frameworks modernos:** Aunque React usa mucho funciones, históricamente usó clases. Angular sigue basándose casi totalmente en clases para definir componentes y servicios.
4. **Backend con Node.js:** Para organizar la lógica de acceso a bases de datos o servicios de correo.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 5: Instanciando Objetos

Entregable del módulo de Objetos para consolidar el modelado de entidades, clases y métodos.

¡Felicidades por llegar al final del módulo de Objetos! Has recorrido un camino fundamental. En programación, casi todo lo que ves en una aplicación real es un objeto: desde un usuario en una red social hasta un producto en un carrito de compras.

Hasta ahora, has aprendido a:

1. **Modularizar tu código** con funciones
2. **Gestionar listas** con arrays
3. **Modelar entidades** complejas con objetos literales, constructores y clases

Aquí unirás todo lo anterior para crear una estructura de datos sólida que servirá como el "corazón" de tu proyecto final.

El objetivo

El propósito de esta entrega es **consolidar el modelado de datos**. No buscamos una aplicación visual todavía, sino una lógica de datos robusta. Aprenderás a representar elementos mediante código, asegurando que cada "objeto" tenga sus datos organizados y sepa realizar acciones por sí mismo.

Lo que debe incluir tu entrega:

1. **Clase:** Una Class que definaobjetos en relación a tu simulador(ej: `Producto`, `Usuario`, `Tarea`, `Vehiculo`).
2. **Propiedades esenciales:** Al menos 4 propiedades (ej: `id`, `nombre`, `precio`, `stock`).
3. **Método de instancia:** Al menos un método que modifique o informe sobre el estado del objeto (ej: `vender()`, `aplicarDescuento()`, `completar()`).

Criterios de Aceptación

Tu código debe:

- **Utilizar `class`**: No basta con un objeto literal suelto; necesitamos la "fábrica" de objetos.
- **Correcto uso de `this`**: El constructor dentro de la Class debe referenciar a sus propiedades usando la palabra clave `this`.
- **Instanciación**: Crear al menos tres instancias (objetos reales) usando el operador `new`.
- **Sintaxis moderna**: El código debe estar libre de errores de sintaxis y seguir las convenciones de nombres (CamelCase para variables, PascalCase para Clases).

Pasos sugeridos

1. **Define tu entidad:** Piensa en qué vas a gestionar. Si es una tienda, tu entidad es `Producto`. Si es un gimnasio, es `Socio`.
2. **Escribe la clase:** Define el `constructor` con los parámetros necesarios.
3. **Agrega lógica:** Crea un método que haga un cálculo. Por ejemplo, si es un producto, un método `sumarIva()` que multiplique el precio por 1.21.
4. **Prueba tu código:** Crea variables para tus objetos y usa `console.log` para ver si los métodos funcionan como esperas.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Instrucciones paso a paso:

1. **Creación de la Estructura:** * Define una **Clase** llamada según tu proyecto (ejemplo: `Producto`, `Item`, `Usuario`). * El constructor debe recibir al menos 4 parámetros para inicializar propiedades (ej: `nombre`, `precio`, `categoria`, `stock`).
2. **Añadir Comportamiento:** * Crea un método dentro de la clase que realice una operación lógica. * *Ejemplo:* un método `vender(cantidad)` que reste el número del stock o un método `aplicarDescuento(porcentaje)` que modifique el precio.
3. **Instanciación:** * Crea al menos **tres objetos** diferentes usando la palabra clave `new` y guárdalos en constantes.
4. **Verificación:** * Ejecuta los métodos de tus objetos creados y usa `console.log` para mostrar los resultados finales de las propiedades modificadas.
5. **Contexto:** * El script debe continuar con la logica y herramientas (ciclos, funciones, etc.) del simulador elegido, siendo la Class y los objetos parte del mismo y no una aplicación aislada

Pasos sugeridos

1. **Define tu entidad:** Piensa en qué vas a gestionar. Si es una tienda, tu entidad es Producto. Si es un gimnasio, es Socio.
2. **Escribe la clase:** Define el constructor con los parámetros necesarios.
3. **Agrega lógica:** Crea un método que haga un cálculo. Por ejemplo, si es un producto, un método sumarIva() que multiplique el precio por 1.21.
4. **Prueba tu código:** Crea variables para tus objetos y usa console.log para ver si los métodos funcionan como esperas.

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Esta rúbrica evalúa un checkpoint de pre-entrega del módulo de Objetos en JavaScript desde Cero. Se valora que el estudiante modele una entidad del proyecto con una clase o función constructora, cree instancias con new, incorpore comportamiento mediante métodos y aplique destructuring y spread operator de forma correcta. También se considera la claridad del código y el cumplimiento del formato de entrega, entendiendo que es una instancia de avance y no el proyecto final completo.

Criterio	Descripción	Peso
Modelado de la entidad con class	Evalúa si el archivo define una Class coherente con el proyecto, con un nombre representativo y con al menos cuatro parámetros para inicializar propiedades relevantes de la entidad principal.	35%
Métodos y comportamiento del objeto	Evalúa si la class incorpora al menos un método con lógica útil que modifique o calcule algo sobre la entidad, como vender, aplicar descuento, actualizar stock u otra acción coherente con el proyecto.	20%
Instanciación con new y creación de objetos	Evalúa si se crean al menos tres instancias distintas usando la palabra clave new y si estas se guardan en constantes.	15%
Continuidad del proyecto	Evalúa si el código sigue un hilo conductor, donde todas las herramientas vistas hasta el momento conmutan en un "simulador interactivo" en vez de estar sueltas.	10%
Verificación, consola y comentarios del archivo	Evalúa si el archivo demuestra el funcionamiento del código mediante console.log, ejecuta los métodos creados y presenta comentarios que expliquen cada sección de forma útil y ordenada.	10%
Repositorio y codigo ordenado	El proyecto se presenta en un repositorio de github, con el html en la raíz y el archivo javascript dentro de su correspondiente carpeta, enlazado en el head del documento con el atributo defer.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 6

Funciones de Orden Superior

Aplicar abstracciones de alto nivel sobre colecciones de datos utilizando métodos iterativos avanzados.

Funciones como valores y abstracción

Explora cómo JavaScript trata a las funciones como valores, permitiendo la creación de abstracciones potentes y funciones que generan otras funciones mediante el uso de closures.

Imagina que estás frente a una máquina de café automática. Para obtener un capuchino, solo tienes que presionar un botón. No necesitas saber a qué temperatura exacta se calienta el agua, cuánta presión ejerce la bomba sobre los granos molidos o cómo se vaporiza la leche. Alguien más ya resolvió esos detalles técnicos por ti y te entregó una interfaz simple: un botón.

En programación, esto se llama **abstracción**. En esta unidad, aprenderás cómo JavaScript nos permite tratar a las funciones no solo como acciones, sino como **valores** que podemos guardar, transportar y hasta fabricar mediante otras funciones.

1. El concepto de Abstracción

La **abstracción** consiste en ocultar los detalles complejos de la implementación para concentrarnos en **qué** hace una solución en lugar de **cómo** lo hace.

Hasta ahora, has creado funciones para evitar repetir código. Por ejemplo, una función para sumar dos números. Cuando usas `sumar(10, 5)`, no te detienes a pensar en los operadores aritméticos internos cada vez; confías en el nombre de la función.

¿Por qué es importante?

Si no abstraemos, nuestro código se vuelve una lista interminable de instrucciones minuciosas. La abstracción nos permite:

1. **Reducir la complejidad:** Podemos manejar sistemas grandes pensando en piezas pequeñas.
2. **Facilitar el mantenimiento:** Si el "cómo" cambia (por ejemplo, ahora la suma debe redondearse), solo cambias la lógica en un lugar, y el resto del programa sigue usando el mismo "botón" sin enterarse del cambio interno.

2. Funciones que retornan funciones (Fábricas de código)

Aquí es donde la magia ocurre. Si una función puede retornar un valor (como un número o un string), y hemos establecido que las funciones son valores, entonces **una función puede retornar otra función**.

A esto se le suele llamar **Pattern Factory** (Patrón de Fábrica). Imagina que quieres crear diferentes tipos de multiplicadores. Podrías hacer esto:

CÓDIGO

```
// Esta función "externa" recibe el factor
function crearMultiplicador(factor) {
    // Esta función "interna" hará el trabajo real
    return (numero) =>  numero * factor
}

// Creamos una función especializada en duplicar
const duplicar = crearMultiplicador(2)

// Creamos una función especializada en triplicar
const triplicar = crearMultiplicador(3)

console.log(duplicar(10)) // 20
console.log(triplicar(10)) // 30
```

¿Cómo es posible que funcione? (El concepto de Closure)

Observa que la función interna utiliza la variable `factor`. Lo increíble es que, cuando ejecutamos `duplicar(10)`, la función `crearMultiplicador` ya terminó de ejecutarse.

3. Funciones que reciben otras funciones

Así como una función puede retornar otra función, también puede **recibir una función como parámetro**, a esto se le llama normalmente **callback**.

La idea es simple: una función “delegada” recibe otra función para decidir qué hacer más adelante.

Imagina que quieres ejecutar distintas acciones sobre un número:

CÓDIGO

```
// La función recibe otra función como parámetro
function procesarNumero(numero, accion) {
  return accion(numero)
}
// Funciones que podemos pasar como argumento
const duplicar = (n) => n * 2
const cuadrado = (n) => n * n
console.log(procesarNumero(5, duplicar)) // 10
console.log(procesarNumero(5, cuadrado)) // 25
```

¿Qué está pasando aquí?

La función `procesarNumero` no sabe exactamente qué operación realizará, simplemente recibe una función (`accion`) y la ejecuta usando: accion(numero)

Eso significa que el comportamiento puede cambiar según la función que enviemos.

¿Por qué esto es útil?

Porque permite crear funciones más flexibles y reutilizables, en lugar de escribir: `duplicarNumero()` `cuadradoNumero()` `triplicarNumero()`

podemos tener una sola función genérica que recibe “la acción” como parámetro.

Búsqueda inteligente con find, filter y some

Aprende a buscar objetos en tus arrays de forma declarativa usando find, filter y some, evitando el uso de bucles manuales.

Así como podemos construir nuestras propias funciones de orden superior, también es muy común utilizar algunos de los diferentes métodos disponibles que nos ahorran muchísimo trabajo en el día a día al momento de interactuar con arrays de objetos.

En esta unidad haremos un repaso por los métodos mas comunes de búsqueda y transformación. Para eso, nuestro array de objetos de punto de partida será el siguiente:

CÓDIGO

```
const productos = [
  { nombre: "Laptop", precio: 800, categoria: "Electronics" },
  { nombre: "Teclado", precio: 50, categoria: "Electronics" },
  { nombre: "Silla Gamer", precio: 200, categoria: "Furniture" },
  { nombre: "Monitor", precio: 300, categoria: "Electronics" },
  { nombre: "Escritorio", precio: 150, categoria: "Furniture" }
]
```

1.forEach()

Muy similar al for-of, el método `forEach()` se usa para recorrer cada elemento de un array de objetos y ejecutar una función sobre cada uno de ellos.

Es una forma más moderna y clara de iterar arrays que usar un `for` tradicional.

Sintaxis

CÓDIGO

```
array.forEach((elemento) => {    // código a ejecutar});
```

Parámetros de la función

- `elemento` : valor actual del array.

Ejemplo básico

CÓDIGO

```
productos.forEach((producto) => console.log(producto))
```

Resultado: `forEach()` ejecuta la función una vez por cada elemento del array.

CÓDIGO

```
{ nombre: "Laptop", precio: 800, categoria: "Electronics" },
{ nombre: "Teclado", precio: 50, categoria: "Electronics" },
{ nombre: "Silla Gamer", precio: 200, categoria: "Furniture" },
{ nombre: "Monitor", precio: 300, categoria: "Electronics" },
{ nombre: "Escritorio", precio: 150, categoria: "Furniture" }
```

2.find()

El método `find()` se usa para buscar un elemento dentro de un array y devolver el primero que cumpla una condición.

Sintaxis

CÓDIGO

```
array.find((elemento, ) => { // condición});
```

Ejemplo básico

CÓDIGO

```
const resultado = productos.find((producto) => producto.nombre == "Monitor")
console.log(resultado);
```

Resultado: `find()` devuelve el primer elemento que cumple la condición.

CÓDIGO

```
{ nombre: "Monitor", precio: 300, categoria: "Electronics" }
```

3.filter()

El método `filter()` se usa para crear un nuevo array con todos los elementos que cumplan una condición.

Sintaxis

CÓDIGO

```
array.filter((elemento) => { // condición});
```

Ejemplo básico

CÓDIGO

```
const resultado = productos.filter((producto) => producto.precio >= 300)
console.log(resultado)
```

Resultado: `filter()` devuelve un nuevo array con los elementos que cumplen la condición.

CÓDIGO

```
{ nombre: "Monitor", precio: 300, categoria: "Electronics" },
{ nombre: "Laptop", precio: 800, categoria: "Electronics" }
```

4.some()

El booleano, este método se usa para comprobar si al menos un elemento de un array cumple una condición.

Sintaxis

CÓDIGO

```
array.some((elemento) => { // condición});
```

Ejemplo básico

CÓDIGO

```
const existe= productos.some((producto) => producto.nombre == "Laptop")
console.log(resultado) // true
```

Resultado: `some()` devuelve `true` si al menos un elemento cumple la condición.

Map y reduce para transformar colecciones

Aprende a transformar, filtrar y resumir colecciones de datos de forma declarativa usando map, filter y reduce, evitando el uso de bucles manuales.

Imagina que tienes una lista de 100 productos en un e-commerce. Si quisieras aplicar un descuento a todos, filtrar los que no tienen stock o calcular el total de una venta, usar un bucle `for` manual puede volverse repetitivo y propenso a errores (como el famoso error de "índice fuera de rango").

En JavaScript moderno, utilizamos **Métodos de Orden Superior** para manipular arrays de forma más limpia, legible y segura.

1. El concepto de Inmutabilidad

A diferencia de métodos como `push` o `pop` que modifican el array original, `map`, `filter` y `reduce` siguen el principio de **inmutabilidad**: no cambian tus datos originales, sino que generan un nuevo resultado basado en ellos. Esto hace que tu código sea mucho más predecible.

2. Map: La "Línea de Traducción"

`map()` se usa cuando quieres transformar cada elemento de un array en algo nuevo.

- **Entrada**: Un array de N elementos.
- **Salida**: Un **nuevo** array de exactamente N elementos transformados.

Ejemplo: Convertir precios de dólares a pesos.

CÓDIGO

```
const preciosUSD = [10, 20, 30];
const preciosARS = preciosUSD.map(precio => precio * 900);
// Resultado: [9000, 18000, 27000]
```

3. Reduce: La "Caja de Empaque"

`reduce()` es el más potente. Se usa para tomar todos los elementos de un array y combinarlos en un **único valor** (un número, un string, e incluso un objeto).

Requiere dos cosas:

1. Una función con un **acumulador** (el resultado parcial) y el **valor actual**.
2. Un **valor inicial** para el acumulador.

Ejemplo: Sumar todos los números.

CÓDIGO

```
const numeros = [5, 5, 10]
const total = numeros.reduce((acc, num) => acc + num, 0);
// Proceso: 0+5=5 -> 5+5=10 -> 10+10=20
// Resultado: 20
```

Errores Comunes

- **Olvidar el `return`**: Si usas llaves `{}` en tu función flecha, debes escribir explícitamente `return`. Si no lo haces, `map` devolverá un array lleno de `undefined`.
- **No poner valor inicial en `reduce`**: Aunque es opcional en algunos casos, omitirlo en arrays de objetos causará errores críticos. ¡Acostúmbrate a ponerlo siempre!

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 6: Interactuando con Funciones de Orden Superior

Entregable para afianzar el uso de funciones de orden superior y la interacción con arrays de objetos.

El objetivo

El propósito de esta entrega es **consolidar la interacción con los datos**. Aprenderás a interactuar con tu array de objetos mediante diferentes métodos vistos en el módulo para una correcta implementación e interacción con la información.

> **Nota**
>
> **Recap rápido (leé esto antes de empezar).** * **¿Qué es "el simulador"?** Es el proyecto que venís construyendo durante todo el curso: una página interactiva de la temática que elegiste (una tienda, un gestor de tareas, una reserva…) que resuelve un proceso y le da información de valor al usuario. Acá **no arrancás de cero**: le sumás funciones de orden superior a ese mismo proyecto. * **¿De dónde salen los objetos?** De la **Class que definiste en el Módulo 5** (ej. `Producto`, `Usuario`, `Tarea`): guardás varias instancias en un array y trabajás sobre él. Si no los tenés a mano, podés crear un array de objetos literal que simule tu base de datos.

Así se ve el resultado esperado (mini-ejemplo)

Partís de un array de objetos:

CÓDIGO

```
const productos = [
  { nombre: "Laptop",  precio: 800, stock: 5 },
  { nombre: "Teclado", precio: 50,  stock: 0 },
  { nombre: "Monitor", precio: 300, stock: 3 },
];
```

Y aplicás los métodos del módulo:

CÓDIGO

```
// find → traer un producto puntual
const buscado = productos.find(p => p.nombre === "Monitor");
console.log(buscado); // { nombre: "Monitor", precio: 300, stock: 3 }

// filter → quedarte con los que tienen stock
const disponibles = productos.filter(p => p.stock > 0);
console.log(disponibles); // [ {Laptop...}, {Monitor...} ]

// reduce → total invertido en stock
const total = productos.reduce((acc, p) => acc + p.precio * p.stock, 0);
console.log("Total en stock: $" + total); // Total en stock: $4900
```

Tu entrega tiene que verse así, pero **integrada a tu simulador** (los datos y la interacción salen de tu temática), no como snippets sueltos.

Lo que debe incluir tu entrega:

1. **Array de objetos** Almacena tus objetos creados a partir de una Class en un array para interactuar con los mismos. Como alternativa puedes crear un array de objetos literal para simular tu base de datos.
2. **Métodos:** Mínimo tres funciones de orden superior (métodos) vistas en el módulo para interactuar con tu array de objetos.
3. **Contexto**: Los métodos que se añaden deben ser complementarios a la interacción con el simulador elegido, ejecutándose segun lo que el usuario desee hacer.

Criterios de Aceptación

Tu código debe:

- **Contener un array de objetos**: Partiendo de los objetos instanciados en el modulo anterior o con un array de objetos literal.
- **Métodos de búsqueda** : Mínimo dos métodos de búsqueda para interactuar con la información del array.
- **Métodos de transformación**: Mínimo un método de transformación para modificar la información recibida desde el array.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Cómo se evalúa esta entrega

Criterio	Descripción	Peso
		15%

Criterio	Descripción	Peso
Entrega correcta	El proyecto se entrega en un repositorio de github, con el archivo html en la raiz y el javascript en su correspondiente carpeta, correctamente enrutado en el head con el atributo defer.	
Array de objetos	El proyecto debe tener un array de objetos correctamente nomenclado, creado a partir de objetos instanciados con una clase o bien un array de objetos literal	25%
Metodos de busqueda	El proyecto debe tener al menos dos metodos de busqueda vistos en el modulo (forEach, find, filter, some) coherentes y en contexto con el proyecto, no deben estar asignados sin sentido	25%
Metodo de transformacion	El proyecto debe tener al menos un método de transformación vistos en el modulo (map, reduce) coherente y en contexto con el proyecto, no deben estar asignado sin sentido	25%
Interaccion	El proyecto debe continuar con la logica del simulador planteado, interactuando con el usuario y ejecutando las diferentes herramientas en consecuencia a lo solicitado.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 7

DOM y Eventos: Interactividad en la Web

Manipular el DOM para actualizar contenido dinámicamente y gestionar eventos de usuario para la interactividad.

Nodos y selectores: encontrar elementos en el DOM

Aprende a conectar JavaScript con el HTML mediante el DOM. En esta unidad descubrirás cómo encontrar y seleccionar elementos de una página web utilizando métodos modernos como querySelector y el clásico getElementById.

Imagina que tu página HTML es una mansión y JavaScript es un robot que vive en ella. Si quieres que el robot pinte una pared de rojo, primero debe saber **dónde** está esa pared. En esta unidad, aprenderás a darle ojos a tu código para que pueda encontrar cualquier elemento en la pantalla.

1. El DOM como representación del documento

Cuando el navegador lee tu archivo HTML, no lo ve simplemente como un montón de texto. Lo transforma en una estructura organizada llamada **DOM (Document Object Model)**.

El DOM es una representación interna de la página en forma de **árbol**. Imagina un árbol genealógico:

- El tronco principal es el documento entero (`document`).
- De ahí nacen ramas como el `<head>` y el `<body>` .
- Del `<body>` nacen ramas más pequeñas como `<h1>` , `<p>` , `<div>` , etc.

¿Por qué es importante esto? Porque gracias al DOM, JavaScript puede "ver" el HTML como si fuera un objeto. Esto significa que podemos acceder a sus propiedades y métodos para cambiar el contenido, los colores o el comportamiento de la página en tiempo real.

2. Nodos vs. Elementos: ¿Qué estamos buscando?

Antes de empezar a buscar, debemos entender qué podemos encontrar. En el universo del DOM, casi todo es un **Nodo**.

- **Nodo:** Es el término más amplio. Un comentario en tu código, el texto dentro de un párrafo, o una etiqueta HTML, todos son nodos.
- **Elemento:** Es un tipo específico de nodo que corresponde directamente a una etiqueta HTML (como `<div>` , `<a>` o `<button>`).

3. Selectores CSS: El mapa de búsqueda

Para encontrar elementos, JavaScript utiliza el mismo sistema que ya conoces de CSS: los **selectores**. Si sabes cómo darle estilo a un botón en CSS, ya sabes cómo buscarlo en JavaScript.

Repasemos los tres más importantes:

1. **Selector de Etiqueta:** Busca por el nombre de la etiqueta. Ejemplo: `h1`, `p`, `button`.
2. **Selector de Clase:** Busca elementos que tengan un atributo `class`. Se escribe con un punto inicial. Ejemplo: `.btn-principal`, `.texto-destacado`.
3. **Selector de ID:** Busca un elemento único con un atributo `id`. Se escribe con un numeral. Ejemplo: `#header`, `#user-profile`.

4. getElementById: El tiro de precisión

El método `document.getElementById()` es la forma más directa y rápida de encontrar un elemento. Funciona como un número de seguridad social: solo debería haber un elemento con ese ID en toda la página.

Ejemplo de uso:

Si tienes este HTML:

CÓDIGO

```
<h1 id="titulo-principal">¡Hola, Mundo!</h1>
```

Puedes "atraparlo" en JavaScript así:

CÓDIGO

```
const titulo = document.getElementById("titulo-principal")
console.log(titulo) // Muestra el elemento <h1> en la consola
```

5. querySelector: La navaja suiza

`document.querySelector()` es el método más moderno y versátil. A diferencia de `getElementById`, este método acepta **cualquier selector CSS válido**.

Es como un buscador inteligente: "Búscame el primer elemento que cumpla esta regla".

Escenarios de uso:

A. Buscar por clase:

CÓDIGO

```
const boton = document.querySelector(".btn-cancelar")
```

B. Buscar por etiqueta:

CÓDIGO

```
const parrafo = document.querySelector("p")
```

C. Búsquedas complejas (Combinadores): Imagina que quieres el enlace que está dentro de un menú:

CÓDIGO

```
const linkNav = document.querySelector("nav > ul > li > a")
```

¿Qué pasa si hay muchos?Es vital recordar que `querySelector` solo te devolverá el **primer** elemento que coincida. Si tienes 10 párrafos y usas `document.querySelector('p')`, solo obtendrás el primero de la lista.

6. getElementsByClassName: Agrupación clásica

El método `document.getElementsByClassName()` agrupa todos los elementos que tengan la misma clase. Funciona como un array, aunque técnicamente su nombre es **colección de elementos HTML**.

Ejemplo de uso:

Si tienes este HTML:

CÓDIGO

```
<p className="descripcion-principal">¡Vamos a aprender HTML!</p>
<p className="descripcion-principal">¡Vamos a aprender CSS!</p>
<p className="descripcion-principal">¡Vamos a aprender JavaScript!</p>
```

Puedes "atraparlos" en JavaScript así:

CÓDIGO

```
const parrafos = document.getElementsByClassName("descripcion-principal")
console.log(titulo) // Muestra todos los elementos <p> con la class "descripcion-principal" en
```

7. querySelectorAll: Agrupando con modernidad

`document.querySelectorAll()` es el método más moderno para agrupar elementos. A diferencia de `getElementsByClassName` e igual que con `querySelector` , este método acepta **cualquier selector CSS válido**. Tambien funciona como un array, aunque técnicamente su nombre es **lista de nodos**.

CÓDIGO

```
const parrafos = document.querySelectorAll("descripcion-principal")
console.log(titulo) // Muestra todos los elementos <p> con la class "descripcion-principal" en
```

El valor de `null`

Si intentas buscar un elemento que no existe (por ejemplo, escribiste mal el ID o la clase), ambos métodos te devolverán `null` .

> **Nota**
>
> **Consejo:** Siempre que selecciones algo, es una buena práctica hacer un `console.log` para asegurarte de que no obtuviste un `null`. Si recibes `null`, tu código fallará en el siguiente paso cuando intentes cambiarle algo al elemento.

8. Errores comunes y mejores prácticas

1. **Repetir IDs:** Aunque el navegador a veces te lo permita, los IDs **deben ser únicos**. Si repites un ID, `getElementById` solo encontrará el primero, y tu HTML dejará de ser válido.
2. **Confundir `querySelector` con `querySelectorAll`:** Si necesitas atrapar todos los elementos de una lista (por ejemplo, todos los `<li>`), `querySelector` no te servirá; para eso existe `querySelectorAll`.

Cambiar texto y elementos para actualizar la interfaz

Aprende a modificar el contenido de texto y a gestionar elementos dinámicamente para crear interfaces interactivas y seguras.

En la unidad anterior aprendiste a **encontrar** elementos en el DOM. Ahora aprenderás a **hacer algo con ellos**. Modificar el contenido y la apariencia de una página en respuesta a lo que hace el usuario es la esencia del desarrollo Frontend.

1. Modificar contenido con `textContent`

Cuando quieres cambiar las palabras que aparecen dentro de un elemento (como un título o un párrafo), la herramienta más segura y eficiente es `innerText`.

¿Por qué usar `textContent`?

- **Simplicidad**: Solo maneja texto plano.
- **Seguridad**: Si intentas insertar etiquetas HTML (como `<script>`), `textContent` las mostrará literalmente como texto en lugar de ejecutarlas.

CÓDIGO

```
const titulo = document.querySelector('h1')
titulo.textContent = "¡Hola, mundo dinámico!"
```

> **Importante**
>
> **Diferencia clave**: A diferencia de `innerHTML`, `textContent` no "entiende" etiquetas. Si pones `titulo.textContent = "<b>Hola</b>"`, el usuario verá literalmente las etiquetas <b> en la pantalla.

2. Modificar elementos con `textContent`

Cuando quieres cambiar el contenido de un elemento incluyendo etiquetas HTML (como `<strong>`, `<img />` o `<p>`), la herramienta más utilizada es `innerHTML`.

¿Por qué usar `innerHTML`?

Permite insertar HTML: Puedes agregar etiquetas HTML directamente dentro de un elemento.

Más flexible: Sirve para crear estructuras dinámicas como listas, tarjetas o botones.

CÓDIGO

```
const titulo = document.querySelector('h1')
titulo.innerHTML = "<strong>¡Hola, mundo dinámico!</strong>"
```

Resultado: El texto aparecerá en negrita porque la etiqueta `<strong>` sí es interpretada como HTML.

3. Crear elementos con `createElement`

Cuando quieres crear nuevos elementos HTML desde JavaScript (como un `<p>`, un `<div>` o un `<button>`), la herramienta más utilizada es `createElement`.

¿Por qué usar `createElement()`?

Permite crear elementos dinámicamente: Puedes generar elementos HTML desde JavaScript cuando la página ya está cargada.

Muy útil para aplicaciones dinámicas: Se usa mucho para agregar tarjetas, listas, botones o contenido generado automáticamente.

CÓDIGO

```
const parrafo = document.createElement('p')
parrafo.textContent= "Este párrafo fue creado con JavaScript"
document.body.appendChild(parrafo) //se añade el elemento <p> al final del HTML, antes del ci
```

Resultado

Se creará un nuevo párrafo dentro de la página con el texto:

CÓDIGO

```
Este párrafo fue creado con JavaScript
```

4. Eliminar elementos con `remove`

Cuando quieres eliminar un elemento HTML de la página usando JavaScript, la herramienta más utilizada es `remove()`.

¿Por qué usar `remove()`?

Simplicidad: Permite eliminar un elemento directamente sin necesidad de acceder al elemento padre.

Código más limpio: Es más fácil de leer que métodos antiguos como `removeChild()`.

Muy útil para interfaces dinámicas: Se usa mucho para eliminar mensajes, tarjetas, tareas o elementos creados dinámicamente.

CÓDIGO

```
const titulo = document.querySelector('h1')
titulo.remove() // Se eliminara del DOM el elemento
```

Resultado: El elemento `<h1>` desaparecerá de la página.

Interactividad: addEventListener y objeto Event

Aprende a transformar páginas web estáticas en interfaces interactivas mediante el uso de eventos en JavaScript.

Un evento es, simplemente, **algo que sucede** en el navegador. El navegador es un espía constante: está vigilando si mueves el mouse, si presionas una tecla, si la página terminó de cargar o si un video se detuvo.

Existen cientos de eventos, pero los podemos agrupar en categorías lógicas:

1. **Eventos de Mouse:** `click` (clic), `dblclick` (doble clic), `mousemove` (mover el ratón).
2. **Eventos de Teclado:** `keydown` (presionar tecla), `keyup` (soltar tecla).
3. **Eventos de Formulario:** `submit` (enviar formulario), `change` (cambiar el valor de un input), `focus` (hacer clic dentro de un campo).

Concepto Clave: El "Escuchador" (Event Listener)

Para que JavaScript reaccione a un evento, necesitamos instalar un **Event Listener**. Imagina que pones a un guardia de seguridad junto a un botón con una orden específica: *"En cuanto alguien presione este botón, activa la alarma"*.

3. La Anatomía de la Interactividad: `addEventListener`

La forma moderna y recomendada de manejar interactividad es el método `addEventListener`. Su estructura es la siguiente:

CÓDIGO

```
elemento.addEventListener('tipoDeEvento', funcionAejecutar)
```

Evento `click`

El evento `click` se utiliza para ejecutar una acción cuando el usuario hace clic sobre un elemento, s uno de los eventos más usados en JavaScript.

CÓDIGO

```
const boton = document.querySelector('button')
boton.addEventListener('click', () => {console.log('Hiciste clic')})
```

Cuando el usuario haga clic en el botón, aparecerá el mensaje: "Hiciste clic"

Ejemplo Real: El botón de "Me Gusta"

Imagina un botón que cambia de texto cuando le haces clic.

HTML:

CÓDIGO

```
<button id="btn-like">Dar Like</button>
```

JavaScript:

CÓDIGO

```
// 1. Seleccionamos el elemento
const botonLike = document.querySelector('#btn-like')

// 2. Definimos qué pasará
function manejarClick() {
    botonLike.textContent= "¡Te gusta esto!"
}

// 3. Conectamos el evento
botonLike.addEventListener('click', manejarClick);
```

¿Por qué hacerlo así?

Es vital notar que no escribimos `manejarClick()`. Pasamos el nombre de la función **sin paréntesis**.

- **Con paréntesis `()`**: La función se ejecuta *ahora mismo*, en cuanto carga la página.
- **Sin paréntesis**: Le das al navegador la "receta" para que la ejecute *solo cuando ocurra el clic*.

Evento de teclado: `keydown`

El evento `keydown` se utiliza para ejecutar una acción cuando el usuario presiona una tecla del teclado, es uno de los eventos más usados en JavaScript para detectar escritura e interacción con el teclado.

CÓDIGO

```
const input = document.querySelector('input')
input.addEventListener('keydown', () => {console.log('Tecla presionada')})
```

Cuando el usuario presione una tecla dentro del input, aparecerá el mensaje: `"Tecla presionada"`.

4. El Objeto `Event`: El mensajero con información

Cuando ocurre un evento, el navegador no solo ejecuta tu función, sino que le envía un regalo: un **objeto `Event`**. Este objeto contiene muchísima información sobre lo que acaba de pasar.

CÓDIGO

```
botonLike.addEventListener('click', function(event) {
    console.log(event); // Aquí verás todas las propiedades del evento
    console.log(event.target); // Te dice EXACTAMENTE qué elemento recibió el clic
});
```

Ejemplo de uso: ¿Qué tecla presionó el usuario?

Si estás creando un juego donde el personaje se mueve con las flechas, necesitas saber *cuál* tecla se presionó:

CÓDIGO

```
document.addEventListener('keydown', function(event) {
    if (event.key === 'ArrowUp') {
        console.log("Moviendo al personaje hacia arriba...");
    }
});
```

En este caso, `event.key` es la propiedad que nos dice el nombre de la tecla. Sin el objeto `event`, solo sabríamos que "se presionó algo", pero no el qué.

5. Errores Comunes y Confusiones

Confusión 1: "El código no encuentra mi elemento"

Error: Intentar seleccionar un elemento del DOM antes de que la página haya terminado de cargar. Si tu script está en el `<head>` del HTML, JavaScript intentará buscar el botón antes de que el navegador lo haya dibujado. **Solución:** Usa el atributo `defer` en la etiqueta script al momento de enrutar el archivo JS.

Confusión 2: El evento vs. El manejador

Error: Confundir la acción con la respuesta.

- **Evento:** El `click`. Es la causa.

- **Manejador (Handler):** La función que se ejecuta. Es la consecuencia.

Confusión 3: Olvidar que el DOM es una interfaz de objetos

Muchos principiantes intentan manipular el DOM como si fuera una cadena de texto. No olvides que `event.target` es un **objeto**. Puedes acceder a sus propiedades como `.style`, `.id`, `.className` o `.value`.

6. Aplicaciones en el Mundo Real

¿Dónde ves esto cada día?

1. **Validación de Formularios:** Cuando escribes una contraseña y el sitio te dice "es demasiado corta" instantáneamente. Eso es un evento de tipo `input`.
2. **Menús Desplegables:** Cuando pasas el ratón sobre un menú y se despliega una lista. Eso es un evento `mouseenter`.
3. **Scroll Infinito:** Cuando llegas al final de una página de Facebook o Instagram y carga más contenido. Eso se logra escuchando el evento `scroll` en la ventana (`window`).
4. **Arrastrar y Soltar (Drag & Drop):** En aplicaciones como Trello, donde mueves tarjetas de una columna a otra, se usan eventos específicos como `dragstart` y `drop`.

Hay muchísimos eventos, lo importante es consolidar la base de como "escucharlos" para poder generar interactividad, a probarlo en tu código!

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 7: Interfaz dinámica con DOM y eventos

Entregable para integrar la lógica de programación con la manipulación del DOM y eventos, construyendo una interfaz web interactiva.

Objetivo de aprendizaje

Consolidar la capacidad de manipular el árbol jerárquico del HTML (DOM) y gestionar el flujo de eventos para crear una experiencia de usuario fluida y dinámica.

Qué construir

Debes desarrollar una **interfaz interactiva** para tu proyecto, pasando de los prompts, console y alert a un desarrollo integro mediante el DOM. Esta interfaz debe permitir al usuario:

1. **Ver una colección de objetos renderizada en pantalla** (no por consola), generada dinámicamente desde un array.
2. **Agregar un nuevo ítem** completando los inputs de un formulario y presionando un botón, **sin recargar la página**: la lista se actualiza sola.
3. **Recibir feedback visual** de cada acción (ej. un mensaje en pantalla o un resaltado al agregar/eliminar).
4. **Interactuar con cada ítem** (ej. botón "Eliminar" o "Logrado") y/o filtrar la lista con un evento de teclado.

Punto de partida (array de ejemplo)

Podés usar los objetos que definiste en la pre-entrega anterior, o arrancar desde este array:

CÓDIGO

```
const productos = [
  { id: 1, nombre: "Teclado mecánico", precio: 45000 },
  { id: 2, nombre: "Mouse inalámbrico", precio: 22000 },
  { id: 3, nombre: "Monitor 24''", precio: 180000 }
];
```

Así se ve el resultado esperado

CÓDIGO

```
┌──────────────────────────────────────────┐
│  Mi Simulador                            │
│  Nombre: [____________]  Precio: [_____] │
│                           [ + Agregar ]  │
├──────────────────────────────────────────┤
│  Teclado mecánico — $45.000    [Eliminar]│
│  Mouse inalámbrico — $22.000   [Eliminar]│
│  Monitor 24'' — $180.000       [Eliminar]│
└──────────────────────────────────────────┘
Al agregar un producto, aparece en la lista al instante (sin recargar la página).
```

> *Producción: reemplazar este mock ASCII por un GIF corto del formulario agregando un ítem y actualizando la lista en vivo.*

Criterios de aceptación

Tu código debe cumplir con:

1. **Selección Precisa**: Uso de `querySelector` o `getElementById/ClassName` para referenciar elementos.
2. **Renderizado Dinámico**: Una función que recorra un array de objetos y genere elementos HTML (usando `innerHTML` junto con backticks) para mostrarlos en pantalla.
3. **Feedback Visual**: Uso de DOM para comunicarle al usuario cuando realiza una accion y su correspondiente consecuencia.
4. **Uso de eventos**: Acceso a eventos para plantear la interacción con el usuario (detectar un click, una tecla, etc.).

Pasos sugeridos

1. **Prepara tu HTML**: Crea una estructura básica que tenga inputs claros y un `<div>` o `<ul>` vacío donde se inyectará el contenido.
2. **Vincula tus Objetos**: Toma los objetos que definiste en la pre-entrega anterior. Crea una función que limpie el contenedor y dibuje cada objeto.
3. **Actualiza la Vista**: Cada vez que el array cambie, llama de nuevo a tu función de renderizado.
4. **Añade Interactividad**: Agrega botones dentro de cada elemento renderizado (ej: un botón "Eliminar" o "Logrado") y asígnales eventos para que realicen acciones según lo que desee el usuario.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

1. Preparación del Entorno

- El repositorio debe contar con tres archivos: `index.html` , `style.css` y `main.js` , cada uno en su correspondiente carpeta, solo el index en la raíz.
- En el HTML, prepara inputs para ingresar datos (ej: nombre de producto, precio) y un contenedores vacío (ej: `<div id="contenedor-items"></div>`) para renderizar estructuras dinámicas.

2. Modelado y Selección

- En `main.js` , selecciona los elementos necesarios del DOM utilizando los metodos vistos según el caso (ej: `getElement..... querySelector.....`)

3. Renderizado Dinámico

- Crea una función para recorrer tu array de objetos y, por cada uno, crear un elemento HTML (puedes usar un template string con `innerHTML`).
- Asegúrate de que cada elemento visual refleje las propiedades de tu objeto (ej: el nombre aparezca en un `<h3>` y el precio en un `<p>`).

4. Gestión de Eventos

- Captura los valores de los inputs, crea un nuevo objeto y agrégalo a tu array mediante botones que "escuchen" el evento click.
- Integrar un evento de teclado para plantear un comportamiento (ej. una barra de búsqueda)

5. Estilos y Estados Visuales

- Crea un archivo CSS para un estilado basico, el proyecto no puede quedar en blanco y negro o sin un layout claro.

6. Interacción

- El proyecto no debe contar mas con prompts, alert ni console.log ya que debe migrarse todo a manejo de DOM.
- Se debe continuar con el modelo de simulador planteado, realizando avances significativos

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Esta rúbrica evalúa una pre-entrega del módulo de DOM y eventos enfocada en construir una interfaz dinámica en el navegador: seleccionar elementos del DOM, renderizar una colección de objetos, manejar el envío de un formulario sin recarga y aplicar interacción visual con clases CSS. La evaluación

prioriza que el proyecto funcione correctamente, que el código esté organizado de forma comprensible y que el repositorio refleje claramente el avance esperado para continuar con el programa.

Criterio	Descripción	Peso
Estructura del proyecto y vinculación de archivos	Verifica que el repositorio incluya la estructura mínima solicitada, con HTML, CSS y JavaScript correctamente enlazados y cada archivo en su carpeta respetando toda la estructura de entregas anteriores.	15%
Selección de elementos del DOM	Evalúa si en el archivo javascript se utilizan correctamente selectores como querySelector o getElement para capturar nodos y generar contenido dinámico.	15%
Renderizado dinámico de la lista de objetos	El proyecto debe contener una función que renderice contenido dinámico desde un array de objetos o similar, planteando plantillas con backticks e información variable.	30%
Manejo de eventos	El proyecto debe contar con eventos como click y algún evento de teclado para la interacción con el usuario, planteando acciones en base a lo que desee el usuario.	30%
Estilos basicos en el proyecto	El proyecto debe contener un estilado básico dentro del archivo css de estilos, planteando una estructura simple que haga al simulador mas comprensible y fácil de interactuar.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 8

Storage, Operadores Avanzados y Workshop Integrador

Implementar persistencia de datos local y optimizar código mediante operadores lógicos avanzados de ES6+.

Web Storage: persistencia local con localStorage y sessionStorage

Aprende a persistir datos en el navegador utilizando Web Storage. Descubre las diferencias entre localStorage y sessionStorage, y domina la transformación de datos con JSON para crear aplicaciones que "recuerdan" información.

¿Alguna vez te has preguntado cómo es que Facebook recuerda tu modo oscuro incluso después de cerrar el navegador, o cómo un carrito de compras mantiene tus productos aunque refresques la página? En las unidades anteriores, aprendimos que cada vez que refrescamos el navegador, JavaScript "nace de nuevo": todas nuestras variables vuelven a su estado inicial o se pierden.

En esta unidad, aprenderemos a darle **memoria** a nuestras aplicaciones. Descubriremos el **Web Storage**, una herramienta integrada en el navegador que nos permite guardar datos de forma sencilla, permitiendo que nuestras aplicaciones web se sientan persistentes.

1. ¿Qué es el Web Storage? El armario del navegador

El **Web Storage** es un mecanismo que los navegadores modernos nos ofrecen para almacenar información directamente en el dispositivo del usuario (computadora o celular). Imagina que tu aplicación es una oficina: las variables representadas por `let` o `const` son papeles sobre el escritorio que se limpian cada noche; el Web Storage es como un **armario de etiquetas** donde puedes guardar cajas con información y encontrarlas exactamente donde las dejaste la próxima vez que abras la oficina.

Almacenamiento de Clave y Valor

El Web Storage funciona mediante un sistema de **clave y valor** (key/value).

- La **Clave** es el nombre que le das al dato (como la etiqueta de una caja).
- El **Valor** es la información que guardas dentro.

> **Importante**
>
> **Regla de oro:** En el Web Storage, tanto la clave como el valor deben ser **cadenas de texto (strings)**. Más adelante veremos cómo guardar objetos complejos, pero por ahora, recuerda que el navegador solo entiende texto en este armario.

Existen dos tipos principales de almacenamiento que usaremos: `localStorage` y `sessionStorage`.

2. Diferencias entre localStorage y sessionStorage

Aunque ambos comparten la misma forma de uso, se diferencian radicalmente en **cuánto tiempo vive la información**.

localStorage: La memoria a largo plazo

Los datos guardados en `localStorage` no tienen fecha de expiración. Se quedan guardados aunque:

- Refresques la página.
- Cierres la pestaña.
- Cierres el navegador y apagues la computadora.

Solo desaparecen si el usuario limpia manualmente su historial de navegación o si nosotros, mediante código, decidimos borrarlos.

- **Caso de uso ideal:** Preferencias de tema (oscuro/claro), el nombre del usuario, o un carrito de compras.

sessionStorage: La memoria de la pestaña actual

Los datos en `sessionStorage` son efímeros. Solo existen mientras la pestaña del navegador esté abierta.

- Si refrescas la página, los datos **se mantienen**.
- Si cierras la pestaña o el navegador, los datos **se borran**.

Es importante notar que `sessionStorage` es único por pestaña. Si abres la misma página en dos pestañas diferentes, cada una tendrá su propio "armario" separado.

- **Caso de uso ideal:** Datos temporales de un formulario de varios pasos o información que no quieres que quede guardada permanentemente.

Característica	localStorage	sessionStorage
Persistencia	Permanente (hasta que se borre)	Mientras la pestaña esté abierta
Alcance	Por dominio (compartido entre pestañas)	Por pestaña individual
Capacidad	Mayor (\~ 10MB)	Menor (Varía por navegador)

3. Comandos básicos: El set de herramientas

Para interactuar con el almacenamiento, JavaScript nos provee de métodos muy sencillos. Usaremos `localStorage` en los ejemplos, pero recuerda que funcionan exactamente igual para `sessionStorage`.

Guardar datos: `setItem(key, value)`

Para guardar algo, necesitamos decirle el nombre de la "etiqueta" y el "contenido".

CÓDIGO

```
// Guardar el nombre del usuario
localStorage.setItem('nombreUsuario', 'Ana García')

// Guardar la preferencia de tema
localStorage.setItem('tema', 'oscuro')
```

Recuperar datos: `getItem(key)`

Para leer lo que guardamos, usamos la clave. Si la clave no existe, nos devolverá `null`.

CÓDIGO

```
const nombre = localStorage.getItem('nombreUsuario');
console.log(nombre) // Imprime: 'Ana García'

const edad = localStorage.getItem('edad');
console.log(edad) // Imprime: null (porque no existe la clave 'edad')
```

Eliminar datos: `removeItem(key)` y `clear()`

A veces necesitamos limpiar el armario.

CÓDIGO

```
// Eliminar un item específico
localStorage.removeItem('tema')

// Borrar ABSOLUTAMENTE TODO el almacenamiento del sitio
localStorage.clear()
```

4. El problema de los objetos: JSON al rescate

Como mencionamos, el Web Storage solo guarda **strings**. Si intentas guardar un objeto directamente, ocurrirá algo frustrante:

CÓDIGO

```
const usuario = { nombre: 'Luis', puntos: 100 }
localStorage.setItem('datosUsuario', usuario)

console.log(localStorage.getItem('datosUsuario'))
// Imprime: "[object Object]" -> ¡Los datos se perdieron!
```

¿Qué pasó? JavaScript intentó convertir el objeto a texto de forma automática y el resultado es esa frase genérica `[object Object]` que no nos sirve de nada. Para solucionar esto, usamos **JSON (JavaScript Object Notation)**.

Serialización con `JSON.stringify()`

"Serializar" significa convertir un objeto o array en texto plana paro enviarla a algún lugar (como el almacenamiento).

CÓDIGO

```
const usuario = { nombre: 'Luis', puntos: 100 }

// Convertimos el objeto en un string de texto
const usuarioJSON = JSON.stringify(usuario)

localStorage.setItem('datosUsuario', usuarioJSON);
// Ahora en el storage se guarda: '{"nombre":"Luis","puntos":100}'
```

Deserialización con `JSON.parse()`

Cuando recuperamos ese texto, necesitamos convertirlo de nuevo en un objeto de JavaScript para poder acceder a sus propiedades.

CÓDIGO

```
const datosDesdeStorage = localStorage.getItem('datosUsuario')

// Convertimos el texto de vuelta a un objeto real
const usuarioCargado = JSON.parse(datosDesdeStorage)

console.log(usuarioCargado.nombre); // Imprime: 'Luis'
```

5. Aplicaciones en el mundo real

Caso 1: Recordar el tema del sitio

Imagina un botón que cambia entre "Modo Claro" y "Modo Oscuro". Sin Web Storage, al cambiar de página o refrescar, el sitio siempre volvería al modo claro. Con `localStorage`, guardamos la elección y la aplicamos apenas carga la página.

Caso 3: Carrito de compras (E-commerce)

Plataformas como tiendas virtuales usan almacenamiento para que, si cierras el navegador y vuelves mañana, tus productos sigan ahí esperándote.

6.Mejores prácticas y recomendaciones

Límites de seguridad

Nunca guardes contraseñas, números de tarjeta de crédito o información sensible en Web Storage. Cualquier script malicioso (o cualquier persona con acceso físico a la computadora y conocimientos mínimos de la consola) puede leer estos datos fácilmente. El Web Storage no es una caja fuerte, es un armario abierto.

Capacidad limitada

Recuerda que tienes unos 5MB - 10MB aproximadamente. Es mucho para texto, pero si intentas guardar imágenes convertidas a texto o bases de datos gigantes, el navegador arrojará un error de `QuotaExceededError`.

Operadores modernos para escribir menos y evitar errores

Aprende a escribir código JavaScript más limpio y resistente a errores usando sugar syntax moderno.

En el mundo de la programación, existe un concepto llamado **Sugar Syntax**. No cambia lo que el código hace, pero lo hace "más dulce" de leer y escribir. En JavaScript moderno, esto es vital para evitar errores al tratar con datos que vienen de fuentes externas (como una API o una base de datos) que podrían estar incompletos.

1. Operador Ternario (?): Acceso sin miedo

El operador ternario es una simplificación de la estructura condicional *if...else*. Cuando nos encontramos frente a una estructura del tipo *condición - caso 1 - caso 2*, podemos utilizar el operador ternario para simplificarlo. Es decir, el *ternario* significa que es un condicional que consta *sí o sí* de tres partes: la condición, el caso de ejecución en caso que se cumpla, y el caso *else* si no se cumple.

Cuenta con la siguiente estructura:

CÓDIGO

```
condicion  ?  caso1  :  caso2
```

Ejemplo sin ternario:

CÓDIGO

```
let temperatura = 31
if (temperatura > 30) {
  console.log("Dia caluroso!")
} else {
   console.los("Dia agradable")
}
```

Con ternario:

CÓDIGO

```
temperatura > 30 ? console.log("Dia caluroso!") : console.los("Dia agradable")
//El signo ? evalua la condicion y en caso de no ser verdadera, el signo : ejecuta el "else"
```

Lo que está a la *izquierda* del signo **?** es la *condición*, la cual resuelve true o false; a la derecha se escribe la instrucción a ejecutar si la condición es verdadera, y luego del **:** se escribe la instrucción si la condición es falsa.

Esta es una sintaxis ideal para estructuras simples, donde la condición es generalmente pequeña y los casos a resolver también.

2. Hablemos de return

Algo interesante que ofrecen el operador ternario y los operadores lógicos, es un *return implícito* para cada caso. Es decir, no sólo ejecuta la instrucción resuelta, sino que hace ***return*** de la misma. Esto es muy útil cuando queremos retornar valores de forma condicional, lo cual con una estructura tradicional sería más extenso.

Ejemplo de carrito vacío:

CÓDIGO

```
// sin operador logico
const carrito = []

if (carrito.length === 0) {
  console.log("El carrito está vacío!")
}

// con operador AND
carrito.length === 0 && console.log("El carrito está vacío!")
```

Ejemplo de carrito y storage:

CÓDIGO

```
//sin operador logico
let carrito
let carritoLocalStorage = JSON.parse(localStorage.getItem('carrito'))

if (carritoLocalStorage) {
  carrito = carritoLocalStorage
} else {
  carrito = []
}

//con operador OR
const carrito = JSON.parse(localStorage.getItem('carrito')) || []
```

3. Spread

El operador spread (**...**) es una herramienta que nos permite "desparramar" un array u objeto. Esto significa cambiar la forma en la que presentamos la información.

Por ejemplo, si paso un array por parámetro a alguna función, ésta recibe el array entero como tal:

CÓDIGO

```
const nombres = ["Juan", "Julieta", "Carlos", "Mariela"]

console.log(nombres) // ["Juan", "Julieta", "Carlos", "Mariela"]
```

Vemos que se hace console.log del array entero porque eso enviamos por parámetro. Pero si en cambio enviamos un *spread* del array, veremos lo siguiente:

CÓDIGO

```
const nombres = ["Juan", "Julieta", "Carlos", "Mariela"]

// spread ... del array
console.log(...nombres) // Juan Julieta Carlos Mariela

// equivalente a:
console.log("Juan", "Julieta", "Carlo", "Mariela")
```

Vemos que el resultado del spread es equivalente a enviar los cuatro elementos del array por separado como parámetros, ya que ésto es exactamente lo que hace el spread: al aplicarse sobre un array, en vez de enviarlo como array_,_ envía todos sus elementos como parámetros individuales.

Así, si el array "nombres" tiene 4 elementos, al hacer console.log(nombres) estamos enviando 1 parámetro, pero al ejecutar console.log(**...nombres**)_ _estamos enviando **cuatro** parámetros a la función.

Spread de objetos

Se puede hacer spread de objetos también, pero debe hacerse **dentro de una estructura que lo permita**, como suele ser otro objeto. Un spread aplicado sobre un objeto presentaría cada par de *clave-valor* separado por comas, y ésto en una función no sería admisible, pero sí puedo serlo dentro de otro objeto.

Esto suele ser útil cuando queremos replicar o modificar estructuras de objetos, ya que nos permite primero listar todas sus propiedades y valores y luego modificar/agregar las que queramos:

CÓDIGO

```
const usuario1 = {
    nombre: "Juan",
    edad: 24,
    curso: "Javascript"
}

// lista todas las propiedades y valores de usuario1 dentro de otro objeto
const usuario2 = {
    ...usuario1
}
console.log(usuario2) // { nombre: 'Juan', edad: 24, curso: 'Javascript' }

const usuario3 = {
    ...usuario1,
    curso: "ReactJS",
    email: "juan@doe.com"
}
console.log(usuario3)
// { nombre: 'Juan', edad: 24, curso: 'ReactJS', email: 'juan@doe.com' }
```

En este ejemplo, agregamos una propiedad (email) y modificamos la propiedad curso. Recordemos que no podemos tener dos propiedades con el mismo nombre, y en tal caso prevalece la última declarada.

Rest Parameters

El operador spread también puede utilizarse dentro de la declaración de una función para indicar que queremos recibir una **cantidad indeterminada de parámetros**. Supongamos que quiero tener una función para sumar cualquier cantidad de números que reciba por parámetro.

Puedo hacer esto con el operador spread definiendo rest parameters, lo que significa que mi función va a recibir una cantidad indeterminada de parámetros, pero los va a agrupar dentro de un array con el nombre que defina.

Esto suele ser útil cuando queremos replicar o modificar estructuras de objetos, ya que nos permite primero listar todas sus propiedades y valores y luego modificar/agregar las que queramos:

CÓDIGO

```
function sumar(...numeros) {
    return numeros.reduce((acc, n) => acc + n, 0)
}

console.log( sumar(4, 2) ) // 6
console.log( sumar(10, 15, 30, 5) ) // 60
console.log( sumar(100, 300, 50) ) // 450
```

El parámetro _..._numeros se define como un **array** donde se guardan todos los argumentos enviados que coincidan con esa posición.

De esta forma podemos escribir funciones que reciban múltiples parámetros, sin saber con precisión cuántos serán, pudiendo trabajarlos luego como un array dentro de la función.

En este caso, tomamos "numeros" e hicimos que retorne la suma de todos los elementos que reciba con un reduce.

Desestructuración

Aprende una forma rápida y ordenada de extraer valores de arrays u objetos y guardarlos en variables.

Muchas veces queremos acceder a propiedades de objetos y almacenarlas en variables diferentes para un posterior uso, para eso la desestructuración es una forma rápida y moderna de extraer valores de arrays u objetos y guardarlos en variables, ya que hace el código mas limpio y fácil de leer

1.Desestructuración de Arrays

Permite guardar elementos de un array en variables individuales.

CÓDIGO

```
const frutas = ['manzana', 'banana', 'pera']
const [fruta1, fruta2, fruta3] = frutas
console.log(fruta1)
console.log(fruta2)
```

Resultado: "manzanabanana"

Cada variable toma la posición correspondiente del array.

2.Desestructuración de Objetos

Es una técnica que nos permite declarar variables donde guardar propiedades de un objeto de forma rápida y directa. La sintaxis es la siguiente:

CÓDIGO

```
let { prop1, prop2 } = objeto
```

Nótese las llaves a la izquierda del operador =, esto significa que estamos **desestructurando un objeto**. Significa que estamos creando dos variables, *prop1* y *prop2*, donde se almacenan las propiedades con el mismo nombre del objeto que referenciamos a la derecha.

Es decir, los nombres de las variables deben coincidir **exactamente** con los nombres de las propiedades que queremos obtener del objeto.

CÓDIGO

```
const usuario = {
    nombre: "John Doe",
    edad: 32
}
const { nombre, edad} = usuario

console.log(nombre) // "John Doe"
console.log(edad) // 32
```

Si queremos acceder a propiedades más internas dentro de un objeto, es decir desestructurar alguna propiedad que sea a la vez un objeto, es posible hacerlo siguiendo el mismo patrón.

CÓDIGO

```
const usuario = {
    nombre: "John Doe",
    edad: 32,
    telefono: {
        cel: 113334444,
        casa: null,
        trabajo: 113325555
    }
}
const {nombre, telefono: {trabajo}} = usuario

console.log(nombre) // "John Doe"
console.log(trabajo) // 113325555
```

En este caso, como telefono es un objeto, desestructuramos la propiedad trabajo de éste, dentro de la desestructuración de usuario.

3.Alias

Para que la desestructuración funcione debe haber coincidencia con los nombres de las propiedades del objeto. Sin embargo a veces puede que los nombres de las propiedades no sean muy descriptivos para el uso que queremos darle, y por ello podemos desestructurarlas con un *alias*, es decir declarar la variable con un nombre alternativo tras haber desestructurado el objeto.

CÓDIGO

```
const item = {
    item_id: 432,
    product_name: "Some product",
    price_per_unit: 5600
}

const {
    item_id: id,
    product_name: nombre,
    price_per_unit: precio
} = item

console.log(id) // 432
console.log(nombre) // "Some product"
console.log(precio) // 5600
```

En este caso desestructuramos todas las propiedades de *item*, pero lo almacenamos en variables denominadas ***id, nombre, precio***, a través del alias que indicamos para cada una.

4.Desestructuración en parámetros

Si en una función recibimos objetos por parámetros, también es posible desestructurarlos directamente en el llamado, definiendo esto al declarar la función.

Esta función recibe un objeto *producto* por parámetro y debe trabajar con sus propiedades *id* y *nombre*.

CÓDIGO

```
const producto = {
    id: 10,
    nombre: "Curso Javascript",
    precio: 12500
}

const desestructurar = (item) => {
    // desestructurando dentro del bloque
    const {id, nombre} = item
    console.log(id, nombre)
}

desestructurar(producto) // 10 Curso Javascript
```

Sabiendo qué es lo que vamos a recibir y qué necesitamos desestructurar, podemos traducir esto con la siguiente lógica:

CÓDIGO

```
// desestructurando lo que reciba por parámetro
const desestructurar = ({id, nombre}) => {
    console.log(id, nombre)
}

desestructurar(producto) // 10 Curso Javascript
```

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 8: Sincronización de estado entre DOM y Storage

Entregable centrado en la integración de persistencia de datos (localStorage) con la interfaz dinámica (DOM), asegurando que el estado de la aplicación se mantenga entre recargas del navegador.

Objetivo de aprendizaje

Consolidar el flujo de **lectura, actualización y persistencia**. El objetivo es que logres que el estado de tu aplicación sea independiente de la sesión actual, utilizando operadores modernos (`??` , `?.`) para manejar datos que podrían no existir aún en el almacenamiento.

Qué construir

Debes ajustar tu proyecto actual para que realice las siguientes tareas:

1. **Persistencia de datos:** El proyecto no debe perder los datos al refrescar la pagina (ej. los productos en un carrito, usuarios guardados, stock actualizado).
2. **Sincronización en Tiempo Real:** Cada vez que el usuario agregue o elimine un elemento (un evento que ya programaste antes), la aplicación debe: * Actualizar el array de datos en JavaScript. * Guardar la versión actualizada en el `localStorage` (usando `JSON.stringify`). * Volver a renderizar la vista para reflejar el cambio.
3. **Operadores avanzados:** El proyecto se debe optimizar utilizando operadores avanzados para mejorar la estructura (ternarios, or, etc.).
4. **Destructuring:** El proyecto debe contar con algún objeto desestructurado para acceder y manipular sus propiedades mas fácilmente.

Criterios de aceptación

- Al recargar la página (`F5`), los elementos que el usuario agregó previamente **siguen apareciendo** en pantalla.
- Al eliminar un elemento, este desaparece tanto de la pantalla como del `localStorage` .
- El código utiliza `JSON.parse()` y `JSON.stringify()` correctamente para manejar arrays de objetos.
- Se utiliza el operador `?` (ternario) para reemplazar estructuras if-else simples.
- Se aplica destructuring para manipular con precision propiedades de objetos.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Tarea de Sincronización de Estado

1. **Persistencia de datos**: * Se debe aplicar `localStorage` para almacenar, recuperar, borrar, modificar y vaciar datos del simulador. Esto es fundamental para la persistencia de datos y que no se pierdan selecciones del usuario. * **Importante**: Usa `JSON.parse() - JSON.Stringify()` para manipular array de objetos en el storage.
2. **Renderizado dinámico**: * El proyecto debe estar íntegramente planteado con el uso de DOM y Eventos, es fundamental para esta ultima parte del desarrollo del simulador.
3. **Uso de Operadores Modernos**: * Dentro de tu proyecto, intenta usar `?` para reemplazar condicionales simples.
4. **Destructuring**: * Desestructurar un objeto al que le desees manipular propiedades, optimizando y modernizando el desarrollo de tu simulador.

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Esta rúbrica evalúa un checkpoint intermedio del módulo de Storage y operadores avanzados. Se centra en verificar que la aplicación recupere, renderice, actualice y persista el estado correctamente entre el DOM y localStorage, usando buenas prácticas básicas de JSON y operadores modernos, con limpieza del DOM para evitar duplicados.

Criterio	Descripción	Peso
Persistencia y recuperación del estado	Evalúa si la aplicación carga correctamente los datos guardados en el storage, usa JSON.parse sobre localStorage y mantiene la información tras recargar la página.	35%
Actualización del estado en eventos	Evalúa si, al interactuar con el formulario o botón, se crea el nuevo dato, se agrega al array de estado, se guarda en storage y se vuelve a renderizar la interfaz.	30%
Renderizado del DOM y Eventos	Evalúa si la parte dinámica del proyecyo se renderiza desde JavaScript manipulando el DOM y capturando eventos. No debe haber uso de prompt, console ni alert.	25%
Estructura del repositorio y entregable	Evalúa si el repositorio de GitHub contiene los archivos esperados y si la aplicación puede entenderse y ejecutarse como una entrega funcional del checkpoint.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 9

Asincronismo, Promesas y Flujo No Bloqueante

Gestionar procesos asíncronos utilizando Promesas y la sintaxis Async/Await para evitar bloqueos en la interfaz.

El Event Loop y por qué JavaScript no se bloquea

Aprende cómo funciona el motor interno de JavaScript, el Call Stack y el Event Loop para entender cómo se gestionan múltiples tareas sin congelar la interfaz de usuario.

Imagina que estás en un restaurante muy popular. Solo hay un cocinero (JavaScript), pero cientos de clientes (tareas) esperando sus pedidos. Si el cocinero tuviera que esperar a que el horno terminara de hornear una pizza antes de empezar a cortar los vegetales para el siguiente plato, el restaurante colapsaría en cuestión de minutos. Los clientes se irían y el negocio fallaría.

En el mundo de la programación, este "colapso" es lo que sucede cuando una página web se congela: no puedes hacer clic, no puedes hacer scroll y parece que la aplicación ha muerto. JavaScript, a pesar de tener "un solo cocinero", es famoso por ser extremadamente eficiente y fluido. ¿Cómo lo logra? La respuesta es el **Event Loop** (Bucle de Eventos).

1. El modelo de un solo hilo (Single Threaded)

Antes de entender la magia, debemos entender la limitación: **JavaScript es un lenguaje de un solo hilo**. Esto significa que solo puede ejecutar una instrucción a la vez. No puede procesar dos funciones simultáneamente en el hilo principal.

Si JavaScript es tan "limitado", ¿cómo es posible que pueda cargar un video, escuchar tus clics y enviar un formulario al mismo tiempo? Aquí es donde entra el modelo **no bloqueante**.

2. El Call Stack (La Pila de Ejecución)

El **Call Stack** es como una lista de tareas mental que tiene el cocinero. Es una estructura de datos tipo LIFO (*Last In, First Out* - el último en entrar es el primero en salir). Cuando llamas a una función, se pone en la parte superior de la pila. Cuando la función termina, se saca de la pila.

Ejemplo de Call Stack:

CÓDIGO

```
function saludar() {
    console.log("Hola");
}

function iniciar() {
    saludar();
}

iniciar();
```

1. `iniciar()` se añade a la pila.
2. `iniciar` llama a `saludar()`, que se añade encima.
3. Se ejecuta `console.log("Hola")`.
4. `saludar()` termina y sale de la pila.
5. `iniciar()` termina y sale de la pila.

El problema ocurre cuando una función tarda mucho (como procesar una imagen pesada). Mientras esa función esté en el Call Stack, el "cocinero" está ocupado y no puede hacer nada más. Esto es el **bloqueo**.

3. Síncrono vs. Asíncrono

Para evitar que el usuario se quede mirando una pantalla congelada, JavaScript divide el trabajo en dos tipos de mensajería:

- **Código Síncrono:** Es secuencial. "Hago esto, y hasta que no termine, no paso a lo siguiente". Es predecible pero peligroso si la tarea es pesada.
- **Código Asíncrono:** Es delegable. "Empiezo esto, pero como sé que va a tardar, te aviso cuando termine. Mientras tanto, sigo con otra cosa".

4. El Event Loop y la Task Queue

Aquí es donde todas las piezas encajan. El **Event Loop** es un mecanismo que vigila constantemente dos cosas:

1. **El Call Stack:** ¿Está vacío?
2. **La Task Queue (Cola de tareas):** ¿Hay algún aviso de que una tarea asíncrona terminó?

Si el Call Stack está vacío y hay una tarea esperando en la cola, el Event Loop la toma y la pone en el stack para que se ejecute.

5. Bloqueante vs. No Bloqueante

Un error común es pensar que "asíncrono" es lo mismo que "paralelo". No lo es. Todo el código de JavaScript eventualmente corre en el mismo hilo. La diferencia es cómo gestionamos el tiempo de espera.

- **Bloqueante:** Una operación que impide que el hilo principal continúe hasta que se completa. Por ejemplo, un bucle `for` de 10 billones de iteraciones. El navegador se congelará.
- **No Bloqueante:** Una operación que inicia un proceso y permite que JavaScript siga con lo siguiente. La respuesta se procesará más tarde.

No bloqueante: setTimeout vs setInterval

En JavaScript tenemos dos herramientas muy utilizadas para crear retrasos, mensajes temporales o acciones automáticas. Si queremos que se ejecute una unica vez, se utilizara `setTimeout()` mientas que si queremos que se repita, se utilizará `setInterval()`

Ambas herramientas comparten una estructura similar:

CÓDIGO

```
setTimeout(() => {
    // código a ejecutar
}, tiempo)

El tiempo en setTimeout representa el delay hasta ejecutar el codigo, mientras que en setInte
```

Ejemplo:

CÓDIGO

```
console.log("1. Inicio del script");

setTimeout(() => {
    console.log("2. Tarea asíncrona (2 segundos después)");
}, 2000);

console.log("3. Fin del script");
```

Lo que verás en consola:

1. `1. Inicio del script`
2. `3. Fin del script`
3. (Pausa de 2 segundos)
4. `2. Tarea asíncrona`

¿Por qué el 3 sale antes que el 2?Porque `setTimeout` es no bloqueante. JavaScript delega la espera al navegador y sigue inmediatamente con el siguiente `console.log`.

Reglas de Oro:

1. **No bloquees el stack:** Nunca pongas cálculos extremadamente pesados directamente en el hilo principal de una web interactiva.
2. **Usa asincronismo para I/O:** Siempre que necesites leer un archivo, consultar una base de datos o llamar a una API, usa métodos asíncronos.
3. **Entiende el orden:** El código síncrono siempre termina antes de que empiece cualquier callback asíncrono.

Callbacks y Promesas: de una función diferida a un objeto con estado

Aprende a diferenciar y construir callbacks y promesas, entendiendo los tres estados fundamentales de una promesa (pending, fulfilled, rejected).

En la unidad anterior, vimos que JavaScript usa el **Event Loop** para no quedarse "congelado" mientras espera tareas largas (como una descarga o un temporizador). Pero, ¿cómo le decimos a nuestro código qué hacer *exactamente* cuando esa tarea termine?

Aquí es donde entran los **Callbacks** y las **Promesas**.

1. El Callback: "Llámame cuando termines"

Un callback es simplemente una función que pasas como argumento a otra función. Es la forma más antigua de manejar asincronismo.

- **Problema:** Si tienes muchas tareas seguidas (ej: loguear -> buscar perfil -> cargar fotos), terminas con funciones dentro de funciones dentro de funciones. A esto se le conoce como **Callback Hell** (el infierno de los callbacks).

2. La Promesa: Un objeto con futuro

Una **Promise** (Promesa) es un objeto que representa un valor que estará disponible ahora, en el futuro o nunca.

Imagina que pides una pizza:

1. **Pending (Pendiente):** Haces el pedido. Tienes el ticket (el objeto Promesa), pero la pizza no ha llegado.
2. **Fulfilled (Cumplida):** La pizza llega caliente. La promesa se resolvió con éxito.
3. **Rejected (Rechazada):** Llaman para decir que no hay ingredientes. La promesa falló por un error.

Estados de una Promesa

Es crucial entender los tres estados posibles:

- `pending`: El estado inicial. La operación asíncrona aún no ha terminado.
- `fulfilled`: La operación terminó con éxito. Se ejecuta la función `resolve()`.

- **`rejected`**: La operación falló. Se ejecuta la función `reject()`.

¿Por qué usar Promesas en lugar de Callbacks?

Las promesas nos permiten escribir código más limpio y manejar errores de forma centralizada. Mientras que un callback te obliga a pasar funciones por todos lados, la promesa te devuelve un objeto que puedes "escuchar" usando métodos como `.then()` y `.catch()`.

Ejemplo rápido de sintaxis

CÓDIGO

```
const miPromesa = new Promise((resolve, reject) => {
  const exito = true;
  if (exito) {
    resolve("¡Todo salió bien!"); // Pasa a fulfilled
  } else {
    reject("Algo falló"); // Pasa a rejected
  }
});
```

Explorando el método .then()

La promesa necesita tiempo para resolverse ya que depende de factores externos (resolve, reject) y como no queremos bloquear el flujo de ejecución de javascript, tenemos que combinar el uso de setTimeout con el método .then() para un resultado preciso.

Imagina una función que simula obtener datos de un servidor.

CÓDIGO

```
function obtenerUsuario() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve('Usuario cargado')
      }, 3000)
   })
 }
 obtenerUsuario().then((respuesta) => {console.log(respuesta)})
```

Resultado:

Después de 3 segundos aparecerá:

CÓDIGO

```
Usuario cargado
```

Las promesas permiten esperar resultados asíncronos de forma organizada y fácil de leer. El método then "espera" a que se resuelva la asincronía planteada por el setTimeout ya que sino seria imposible poder ejecutar el console.log, estaría esperando una respuesta que todavía no le llego (atención con los 3 segundos del temporizador)

Async/Await: El Flujo Secuencial Asíncrono

Domina la sintaxis moderna de Async/Await para escribir código asíncrono limpio, legible y con manejo de errores robusto usando try/catch.

Hasta ahora, hemos aprendido a gestionar procesos asíncronos usando `.then()` . Aunque funciona perfectamente, cuando tenemos muchas operaciones seguidas, el código puede volverse difícil de leer.

En esta unidad final del módulo, aprenderás a usar **Async/Await**, una forma de escribir código asíncrono que se ve y se siente como código síncrono (secuencial).

El problema: ¿Por qué Async/Await?

Imagina que necesitas:

1. Validar un usuario.
2. Obtener sus publicaciones.
3. Cargar los comentarios de esas publicaciones.

Con `.then()`, terminarías con una cadena larga de promesas anidadas. **Async/Await** resuelve esto permitiéndote "esperar" a que una promesa termine antes de pasar a la siguiente línea de código, sin bloquear el resto de tu aplicación.

1. La palabra clave `async`

Para usar esta sintaxis, primero debes declarar una función como asíncrona usando la palabra `async`.

CÓDIGO

```
async function saludar() {
  return "Hola Mundo";
}
```

Dato clave: Una función `async` *siempre* devuelve una Promesa, incluso si devuelves un simple texto.

2. La palabra clave `await`

Dentro de una función `async`, puedes usar `await` antes de una promesa. Esto le dice a JavaScript: *"Pausa la ejecución de ESTA función aquí hasta que la promesa se resuelva, y luego dame el valor"*.

CÓDIGO

```
const datos = await [{id: 1, name: 'Juan Pérez'}, {id: 2, name: 'Ana García'}]
```

Nota: `await` solo funciona dentro de funciones marcadas con `async`.

3. Manejo de errores con `try/catch`

Con `async/await`, volvemos a la estructura clásica `try/catch` de JavaScript:

- **`try`**: Aquí ponemos el código "peligroso" que podría fallar (como una petición de red).
- **`catch`**: Si la promesa es rechazada (`rejected`), el control pasa inmediatamente a este bloque con el error resultante.
- **`finally`**: (Opcional) Código que se ejecuta siempre, haya habido éxito o error. Ideal para ocultar animaciones de "Cargando...".

Ejemplo de flujo con manejo de errores:

CÓDIGO

```
async function obtenerUsuario() {
  try {
    console.log("Cargando...")
    // El código se detiene aquí hasta que la promesa se cumpla
    const usuarios = await [{id: 1, name: 'Juan Pérez'}, {id: 2, name: 'Ana García'}]

    const usuario = usuarios[0]
    console.log(`Usuario encontrado: ${usuario.name}`)
  } catch (error) {
    console.error("Algo salió mal:", error.message)
  } finally {
    console.log("Intento de conexión terminado.")
  }
}
```

Un error común: Olvidar el `await`

Si llamas a una función asíncrona sin `await`, obtendrás el objeto `Promise { <pending> }` en lugar de los datos reales. Recuerda: `await` es el que "abre el sobre" de la promesa para sacar el contenido.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 9: Asincronismo y Promesas

Entregable para afianzar los conceptos vistos sobre asincronismo, promesas y manejo de errores.

El objetivo

El propósito de esta entrega es **consolidar el concepto de asincronía**. Es muy importante afianzar este conocimiento para encarar el próximo modulo de manera mas eficiente.

Lo que debe incluir tu entrega:

1. **Temporizador:** Uso de `setTimeout()` para generar un comportamiento asincrono en tu proyecto, mostrando informacion valiosa para el usuario segundos despues de ingresar al simulador.
2. **Manejo de errores:** Aplicar una estructura try-catch-finally para manejo de errores en algun bloque de codigo pequeño que pueda ser propenso a los mismos (puede ser el comportamiento de un evento por ejemplo).
3. **Contexto**: Las herramientas que se añaden deben ser complementarias a la interacción con el simulador elegido, ejecutándose según lo que el usuario desee hacer.

Así se ve el resultado esperado

CÓDIGO

```
[Al entrar al simulador]
… (unos segundos después, sin bloquear la interfaz)
   "Cotización del dólar hoy: $1.200"      ← mostrado con setTimeout()

[Si una operación falla]
   "No se pudo procesar la operación, intentá de nuevo"   ← capturado en el catch
   (el bloque finally se ejecuta siempre)
```

Criterios de Aceptación

Tu código debe:

- **Contener un temporizador**: Partiendo de la idea general del simulador, mostrar una información complementaria para el usuario.
- **Manejo de errores** : Evaluación de un fragmento de código con try y su posterior manejo de errores con catch y finally.
- **Avance en el proyecto**: Avances significativos en el desarrollo del simulador, acordes a la instancia actual del curso.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Asincronismo

Para esta instancia, se deben cumplir los siguientes puntos

1. **Temporizador**:

- Se debe plantear un temporizador `setTimeout()` para generar una pequeña asincronía y mostrarle al usuario información unos segundos después de ingresar al simulador (ej. la cotización del dólar del día, un cupón de descuento, un mensaje a modo de recordatorio de una tarea importante, etc.)

1. **Manejo de errores**:

- Plantear una estructura try-catch-finally sobre un fragmento de código que pueda llegar a generar un error a futuro y capturarlo en el catch.

1. **Contexto:**

- El proyecto debe mantener el contexto actual y las herramientas ser una adición al mismo, no se deben incorporar de manera aislada. Es importante presentar avances en el proyecto acordes a la instancia del curso.

Cómo se evalúa esta entrega

Criterio	Descripción	Peso
Implementacion de temporizadores	El proyecto cuenta con la implementacion de un temporizador setTimeout() para mostrarle al usuario un mensaje o informacion relevante unos segundos despues de iniciado el simulador.	30%
Manejo de errores	El proyecto debe contar con un fragmento de codigo dentro de un try para su analisis y si genera un error debe ser manejado desde el catch con un finally final.	30%
Avances en el proyecto	El simulador debe estar en las etapas finales de desarrollo, implementando todo lo visto a lo largo del curso hasta esta instancia	30%
Repositorio correcto	El repositorio de github debe contener todo el proyecto correctamente ordenado, con el archivo javascript en su correspondiente carpeta y enrutado en el head del documento. El archivo css debe estar tambien en su correspondiente carpeta.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 10

APIs, Peticiones y Librerías

Consumir APIs externas mediante Fetch API e integrar librerías de terceros para potenciar funcionalidades.

Cómo funciona Fetch API y cómo leer respuestas JSON

Aprende a realizar peticiones asíncronas con Fetch API, interpretar el objeto Response y transformar datos JSON para su uso en aplicaciones web modernas.

Imagina que estás en un restaurante. Tú eres el cliente (el navegador), el camarero es la **Fetch API** y la cocina es el **Servidor** donde está la base de datos.

Cuando quieres comer algo, no vas directamente a la cocina a buscar los ingredientes. En lugar de eso, llamas al camarero y le haces un pedido (una **petición HTTP**). El camarero va a la cocina, espera a que preparen tu plato y regresa con una bandeja. Pero atención: la bandeja tapada no es la comida todavía; es el **objeto Response**. Para poder comer, tienes que destapar la bandeja y servirte el plato. En JavaScript, ese proceso de "destapar y servir" es lo que hacemos cuando extraemos los datos **JSON**.

1. ¿Qué es Fetch?

Históricamente, realizar peticiones de datos a un servidor desde JavaScript era un proceso complejo y tedioso que requería una tecnología llamada `XMLHttpRequest`. Afortunadamente, los navegadores modernos introdujeron **Fetch API**, una interfaz mucho más sencilla, limpia y basada en **Promesas**.

La función `fetch()` nos permite pedir información a una URL externa. Es una operación **asíncrona**, lo que significa que el código no se detiene mientras espera la respuesta; simplemente continúa y nos avisa cuando la información ha llegado.

La anatomía de un `fetch` básico:

CÓDIGO

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => {
        // Aquí llega la "bandeja" (el objeto Response)
        console.log(response)
    })
```

Como vimos en el módulo anterior, al ser una promesa, usamos `.then()` para capturar el resultado. Sin embargo, si imprimes ese `response` en la consola, notarás algo extraño: **no verás la lista de usuarios de inmediato**. Verás un objeto con propiedades como `status`, `ok` y `headers`. Esto es porque la respuesta es un "contenedor" que aún no ha procesado el cuerpo de los datos.

2. El Objeto Response: La "Bandeja" de entrada

Cuando el servidor nos contesta, JavaScript nos entrega un objeto de la clase `Response`. Este objeto representa la respuesta HTTP completa y tiene herramientas esenciales para saber si todo salió bien antes de intentar leer los datos.

Propiedades clave que debes conocer:

1. **`response.ok`**: Es un valor booleano (`true` o `false`). Es la forma más rápida de saber si la petición fue exitosa (códigos de estado entre 200 y 299).
2. **`response.status`**: El código numérico de la respuesta (ej. `200` para éxito, `404` para "no encontrado", `500` para error de servidor).
3. **`response.headers`**: Información adicional sobre la respuesta, como el tipo de contenido que nos están enviando.

3. Lectura de JSON con `response.json()`

La mayoría de las APIs modernas (como las de clima, películas o redes sociales) envían la información en un formato llamado **JSON** (JavaScript Object Notation). Aunque se parece mucho a un objeto de JavaScript, técnicamente es solo una cadena de texto larga con un formato específico.

Para convertir ese texto en un objeto o array de JavaScript que podamos manipular (usar `forEach`, `filter`, acceder a propiedades, etc.), debemos usar el método `.json()`.

Punto crucial: El método `.json()` también devuelve una **Promesa**. Por lo tanto, necesitamos un segundo `.then()` o un segundo `await` para obtener los datos finales.

Ejemplo con encadenamiento de promesas (.then):

CÓDIGO

```
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json()) // Paso 1: Convertir la respuesta a JSON
    .then(data => {
        // Paso 2: Usar los datos
        console.log(data) // Aquí sí verás el array de usuarios
    })
```

4. Verificación de éxito con `response.ok`

¿Qué pasa si escribes mal la URL o si el servidor está caído? `fetch()` tiene un comportamiento particular: **solo se "rechaza" (va al `.catch`) si hay un error de red** (como quedarte sin internet). Si el servidor responde con un error 404 (No encontrado), `fetch()` lo considera una respuesta válida y no dispara el error automáticamente.

Por eso, una opción es verificar siempre la propiedad `ok` antes de procesar los datos.

El flujo profesional con Async/Await:

Esta es la sintaxis más moderna y recomendada por su legibilidad.

CÓDIGO

```
async function obtenerUsuarios() {
    try {
        const response = await fetch("https://jsonplaceholder.typicode.com/users");

        // Verificamos si la respuesta fue exitosa
        if (!response.ok) {
            // Si el estado no es 200-299, lanzamos un error manual
            throw new Error(`Error en la petición: ${response.status}`);
        }

        const data = await response.json();
        console.log("Datos recibidos:", data);

    } catch (error) {
        // Capturamos errores de red o errores lanzados manualmente arriba
        console.error("Hubo un problema:", error.message);
    }
}

obtenerUsuarios();
```

5. Simulación desde un archivo JSON local

Este proceso se puede replicar también simulando una base de datos local, la única diferencia es que en vez de colocar la ruta de la API dentro del fetch, esta debería cambiar por la ruta local de tu archivo JSON desde donde estás simulando la base de datos.

Es un método muy útil para elaborar proyectos personales sin tener que crear una api propia con nuestros datos simulados.

6. Errores comunes y mejores prácticas

Error: Olvidar el `await` en el `.json()`

CÓDIGO

```
const response = await fetch(url);
const data = response.json(); // ¡ERROR! data será una Promesa pendiente, no los datos.
```

Correcto: Siempre debes esperar a que el JSON se procese: `const data = await respon-se.json();`.

Error: Confundir JSON con un Objeto de JS

Aunque se ven iguales, el JSON que viaja por internet es puro texto. No puedes ejecutar funciones dentro de un JSON ni usar tipos de datos especiales como `undefined`. Es por eso que el paso de "parsing" (análisis) con `.json()` es obligatorio.

Librerías externas para mejorar la experiencia de usuario

Aprende a potenciar tus aplicaciones JavaScript integrando librerías externas. Descubre cómo mejorar la interfaz con SweetAlert2 y Toastify, y cómo dominar el manejo de fechas con Luxon para ofrecer una experiencia de usuario profesional.

Imagina que estás construyendo una tienda online. El usuario hace clic en "Eliminar producto" del carrito. Tienes dos opciones: usar el comando `confirm()` nativo del navegador —ese cuadro gris y aburrido que parece un error del sistema operativo— o mostrar un elegante cuadro de diálogo con iconos, colores de tu marca y animaciones suaves que pregunten: "¿Estás seguro de que quieres quitar este artículo?".

Durante el desarrollo, rara vez escribimos cada línea de código desde cero. Así como un arquitecto no fabrica sus propios ladrillos, los programadores utilizamos **librerías externas** para resolver problemas comunes de interfaz y lógica, permitiéndonos centrar nuestra energía en lo que hace única a nuestra aplicación.

En esta unidad, aprenderás a identificar qué es una librería, cómo integrarla en tus proyectos y cómo utilizar tres de las herramientas más queridas por la comunidad: **SweetAlert2** y **Toastify.**

1. ¿Qué es una librería externa y por qué usarla?

Una **librería** es una colección de código preescrito que otros desarrolladores han creado y compartido para que cualquiera pueda usarlo. Imagina que es un "paquete de soluciones" que puedes importar a tu proyecto.

¿Por qué no hacerlo todo nosotros mismos?

A medida que avanzas como desarrollador, notarás que ciertos problemas se repiten constantemente:

1. **Estética de la interfaz:** Hacer que un mensaje de error se vea bien en Chrome, Safari y Firefox consume mucho tiempo de CSS.
2. **Retroalimentación al usuario:** Necesitamos notificaciones que no bloqueen la navegación del usuario (como los "toasts").

Ventajas de usar librerías:

- **Ahorro de tiempo:** Lo que te tomaría días programar, lo resuelves en minutos.
- **Consistencia:** Ofreces una experiencia visual uniforme.
- **Pruebas de la comunidad:** Estas librerías son utilizadas por millones de personas, lo que significa que sus errores ya han sido detectados y corregidos.

> **Importante**
>
> **Importante:** No debes confundir una *librería* con un *framework*. Una librería es como una herramienta que añades a tu caja (tú tienes el control total de cuándo usarla), mientras que un framework (como React o Angular) es como un molde donde tú encajas tu código.

2. Cómo integrar librerías en tu proyecto

Existen dos formas principales de traer estas herramientas a tu mundo de desarrollo:

A. Mediante un CDN (Content Delivery Network)

Es la forma más rápida. Consiste en añadir una etiqueta `<script>` en tu HTML que apunta a un servidor donde vive el código de la librería.

- **Pros:** Configuración instantánea.
- **Contras:** Dependes de una conexión a internet para que funcione mientras desarrollas.

B. Mediante un gestor de paquetes (npm)

Usas la terminal para descargar el código a tu carpeta local. `npm` (Node Package Manager) es la herramienta que nos permite decir: `npm install sweetalert2`.

- **Pros:** El código vive en tu proyecto, es más seguro y profesional.
- **Contras:** Requiere un poco más de configuración inicial.

3. SweetAlert2: Transformando las alertas

El objeto `window.alert()` que aprendimos al principio de este curso cumple su función, pero tiene un gran problema: **detiene la ejecución de todo el navegador**. Hasta que el usuario no hace clic en "Aceptar", nada más sucede.

SweetAlert2 reemplaza estos mensajes con modales elegantes que no bloquean el hilo principal y son totalmente personalizables.

Link a SweetAlert2: https://sweetalert2.github.io/

Ejemplo de uso básico

Para disparar una alerta de éxito, en lugar del aburrido `alert("¡Guardado!")`, usamos:

CÓDIGO

```
// Swal es el objeto global que nos da la librería
Swal.fire({
  title: '¡Éxito!',
  text: 'El usuario se ha registrado correctamente',
  icon: 'success',
  confirmButtonText: 'Genial'
});
```

El poder de las Promesas en las alertas

Como vimos en la unidad de Asincronismo, SweetAlert2 utiliza promesas. Esto nos permite esperar a que el usuario tome una decisión:

CÓDIGO

```
Swal.fire({
  title: '¿Estás seguro?',
  text: "¡No podrás revertir esto!",
  icon: 'warning',
  showCancelButton: true,
  confirmButtonColor: '#3085d6',
  cancelButtonColor: '#d33',
  confirmButtonText: 'Sí, borrar'
}).then((result) => {
  if (result.isConfirmed) {
    // Aquí ejecutamos la lógica para borrar, por ejemplo, llamar a una API
    Swal.fire('¡Borrado!', 'Tu archivo ha sido eliminado.', 'success');
  }
});
```

Análisis: Observa cómo el método `.then()` maneja la respuesta del usuario. Si el usuario hace clic en confirmar, `result.isConfirmed` será verdadero.

4. Toastify: Notificaciones no intrusivas

A veces no queremos interrumpir al usuario con un cuadro en medio de la pantalla. Queremos informarle que "se ha añadido un producto al carrito" o que "hay un nuevo mensaje". Para esto usamos los **Toasts**.

Un **Toast** es como una pequeña nota adhesiva que aparece en la esquina de la pantalla, da la información y desaparece sola después de unos segundos.

Link a Toastify: https://apvarun.github.io/toastify-js/

Configuración con Toastify-js

Para usarlo, necesitas importar tanto el JS como el CSS de la librería.

CÓDIGO

```
Toastify({
  text: "Producto añadido al carrito",
  duration: 3000, // 3 segundos
  close: true,
  gravity: "top", // top o bottom
  position: "right", // left, center o right
  style: {
    background: "linear-gradient(to right, #00b09b, #96c93d)",
  },
  onClick: function(){} // Callback después de hacer clic
}).showToast();
```

Diferencia clave: Usa SweetAlert2 para **decisiones críticas** (borrar, confirmar compra) y Toastify para **confirmaciones informativas** que no necesitan una acción del usuario.

5. Errores comunes y mejores prácticas

El error de la "Libreritis"

Muchosdesarrolladores se emocionan e instalan 20 librerías para un proyecto pequeño. Esto hace que tu aplicación cargue muy lento (aumento del *bundle size*).

- **Regla de oro:** Antes de instalar, pregúntate: "¿Puedo resolver esto con 5 líneas de JavaScript puro?". Si la respuesta es no, o si la librería aporta una mejora de UX masiva, instálala.

Olvidar el CSS

Librerías como SweetAlert2 o Toastify necesitan sus estilos para verse bien. Si instalas la librería pero ves un texto feo al final de tu página, probablemente olvidaste importar el archivo `.css`.

No leer la documentación

La documentación es tu mejor amiga. No intentes adivinar cómo se llama una propiedad; ve a la página oficial.

Síntesis y conexiones

En esta unidad aprendimos que las librerías externas son nuestras aliadas para crear experiencias profesionales.

- **SweetAlert2** nos da modales interactivos y elegantes.
- **Toastify** nos permite comunicar eventos sin interrumpir al usuario.

Todas estas herramientas tienen algo en común: se integran con lo que ya sabemos. Usan **objetos** para configurarse, **promesas** para manejar respuestas y se activan mediante **eventos** del DOM.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega 10: APIs, Peticiones y Librerías

Entregable para integrar el consumo de APIs reales con Fetch API y mejorar la interfaz mediante librerías externas de notificaciones.

Objetivo de aprendizaje

Consolidar la capacidad de integrar servicios externos (`fetch`), gestionar el flujo de datos asíncronos (`async/await`) y enriquecer la interfaz con librerías (como SweetAlert2 o Toastify) para informar al usuario sobre el estado de sus peticiones.

Qué construir

Debes integrar a tu proyecto el uso de fetch hacia una api y/o archivo json local, asi como tambien implementar al menos una libreria javascript de las vistas.

El proyecto debe:

1. **Solicitar datos**: Al cargar la página, debe traerla informacion desde la api y/o el archivo json local.
2. **Mostrar feedback visual**: Mientras los datos cargan, o si ocurre un error, el usuario debe saberlo (implementar async/await).
3. **Integrar una Librería**: Usa una librería de las abordadas(Toastify o SweetAlert2) para comunicarse de manera mas eficiente con el usuario y reemplazar las alertas nativas de javascript.
4. **Renderizar en el DOM**: Los datos recibidos deben transformarse en elementos HTML visibles y prolijos.

Punto de partida: fuente de datos

Si no vas a usar una API pública, arrancá desde un archivo `data.json` local con esta forma (adaptalo a tu simulador) y consumilo con `fetch("./data.json")` :

CÓDIGO

```
[
  { "id": 1, "nombre": "Teclado mecánico", "precio": 45000, "stock": 12 },
  { "id": 2, "nombre": "Mouse inalámbrico", "precio": 22000, "stock": 30 },
  { "id": 3, "nombre": "Monitor 24''", "precio": 180000, "stock": 5 }
]
```

Si preferís una API real, sirven opciones abiertas y sin autenticación como Fake Store API, JSONPlaceholder o PokeAPI.

Así se ve el resultado esperado

CÓDIGO

```
Cargando productos…           ← mientras resuelve el fetch (async/await)
┌──────────────────────────────────────────┐
│  Teclado mecánico — $45.000   (stock 12) │
│  Mouse inalámbrico — $22.000  (stock 30) │
│  Monitor 24'' — $180.000      (stock 5)  │
└──────────────────────────────────────────┘
   Toastify/SweetAlert: "Productos cargados con éxito"
   Si el fetch falla: notificación de error con la librería (no un alert nativo)
```

> **Nota**
> Producción: reemplazar este mock ASCII por un GIF del fetch cargando datos y la notificación de la librería.

Criterios de Aceptación

- **Consumo de API**: Se utiliza `fetch` con `async/await` para obtener datos de un endpoint real y/o de un archivo json local.
- **Manejo de Errores**: Implementas un bloque `try/catch` que capture fallos de red o respuestas no exitosas para notificar al usuario.
- **Uso de Librería**: Se evidencia la integración de al menos una librería externa (vía CDN) para mostrar notificaciones.
- **Manipulación del DOM**: Los datos de la API se visualizan dinámicamente en el navegador.

Método de entrega

Se deberá enviar el link del repositorio de github en modo público.

Objetivo

Planificar la conexión técnica entre tu lógica de JavaScript, una API externa/JSON local y una librería de apoyo, documentando el flujo de datos.

Instrucciones

1. **Define tu Fuente de Datos**: Captura los datos desde un api y/o un archivo JSON local que simule ser tu base de datos.
2. **Captura de la información**: Se debe consumir la información mediante un fetch, trabajando con la estructura async/await y try-catch.finally para el manejo de errores. Es fundamental que el usuario tenga comunicación en todo momento, el manejo de errores es muy importante.
3. **Implementación de librerías**: Implementar una librería de las vistas en el modulo para mejorar la comunicación de acciones con el usuario.

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Esta rúbrica evalúa una pre-entrega en la que el estudiante planifica la integración entre una API externa, la lógica de JavaScript y una librería de apoyo mediante un archivo JSON. Se valora especialmente que el archivo sea válido, que el esquema de datos esté bien pensado y que la hoja de ruta técnica sea clara, coherente y útil para la siguiente etapa de implementación.

Criterio	Descripción	Peso
Manejo de errores	Evalua si se implemento correctamente la estructura try-catch-finally en el fetch para capturar eventuales errores en la peticion.	30%
Implementacion correcta del fetch	Evalua si el proyecto implementa correctamente un fetch para consumir una api o un archivo json local con su correspondiente estructura async/await	25%
Elección de librería de apoyo y pertinencia técnica	Evalúa si la librería seleccionada está nombrada correctamente y si su elección tiene sentido para mejorar la experiencia de usuario o apoyar la lógica del proyecto según el contexto del ejercicio.	25%
Renderizado en el DOM	Evalua si el fetch luego renderiza la informacion recibida dinamicamente en el dom para su posterior interaccion con el usuario.	20%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 11

Ecosistema Moderno: Node.js, Frameworks y Cierre

Comprender el rol de JavaScript en el servidor y las bases de los frameworks modernos antes de la entrega final.

Node.js y npm: ejecutar JavaScript fuera del navegador

Descubre qué es Node.js, cómo permite ejecutar JavaScript en servidores y computadoras personales, y por qué npm es el corazón del ecosistema de desarrollo moderno.

Hasta este punto de tu camino como desarrollador, has visto a JavaScript como el "motor" de la interactividad en las páginas web. Has aprendido a manipular el DOM, a responder a clics de usuarios y a guardar datos en el navegador. Pero, ¿qué pasaría si pudieras usar esa misma lógica de programación para crear herramientas en tu propia computadora, procesar archivos o incluso construir la base de datos de una aplicación completa?

Aquí es donde entra **Node.js**, vamos a descubrir cómo JavaScript rompió las barreras del navegador para convertirse en una herramienta todoterreno y cómo **npm** se transformó en la biblioteca de piezas de código más grande del mundo.

1. Node.js como entorno de ejecución fuera del navegador

Para entender qué es Node.js, primero debemos recordar dónde vive normalmente JavaScript. Tradicionalmente, JavaScript necesitaba un "padre" para correr: el navegador web (Chrome, Firefox, Safari). El navegador le daba las herramientas para que JS pudiera hablar con la pantalla (el DOM).

Node.js es JavaScript que no necesita un navegador.

El motor debajo del capó

Node.js utiliza el motor **V8**, que es el mismo componente ultrarrápido que usa Google Chrome para procesar JavaScript. Lo que hizo el creador de Node.js fue tomar ese motor, sacarlo del navegador y añadirle "superpoderes" para interactuar directamente con el sistema operativo de tu computadora.

¿Por qué existe? (El "Por qué" antes del "Qué")

Antes de la llegada de Node.js en 2009, si querías programar el servidor de una web (donde se guardan los usuarios o se procesan pagos), tenías que aprender lenguajes distintos como PHP, Java o Ruby. Node.js permitió que los desarrolladores usaran **un solo lenguaje** (JavaScript) para todo el proyecto.

Diferencia clave entre Entornos:

- **En el Navegador:** Tienes acceso al objeto `window` y `document`. Puedes cambiar el color de un botón o detectar un scroll. No puedes leer archivos del disco duro del usuario por seguridad.
- **En Node.js:** No existe el DOM ni el `document` (porque no hay una "página" que mostrar). En su lugar, tienes acceso al sistema de archivos (`fs`), a la red y a procesos de la computadora. Puedes crear carpetas, leer archivos de texto y montar servidores.

2. Casos de uso de Node.js: Servidores, scripts y herramientas

Node.js no es solo para "hacer sitios web". Su versatilidad lo ha llevado a casi todas las áreas del desarrollo de software.

A. Servidores Web (Backend)

Es el uso más famoso. Node.js es excelente para aplicaciones que necesitan manejar muchas conexiones al mismo tiempo de forma rápida, como un chat en tiempo real o una red social.

- **Ejemplo Real: Netflix** utiliza Node.js para gran parte de su interfaz y servicios de backend para asegurar que la carga sea casi instantánea cuando navegas entre películas.

B. Scripts de Automatización

Imagina que tienes 1,000 imágenes que necesitas renombrar o cambiar de tamaño. En lugar de hacerlo a mano, puedes escribir un pequeño "script" (un archivo de JavaScript) y ejecutarlo con Node.js para que haga el trabajo pesado por ti en segundos.

3. Errores comunes y mejores prácticas

El error de la "Caja Negra" (node_modules)

Un error muy común de los principiantes es intentar editar los archivos que están dentro de la carpeta `node_modules`.

- **Por qué es un problema:** Esa carpeta es generada automáticamente por npm. Si borras algo o lo cambias, tus cambios desaparecerán la próxima vez que instales algo o si compartes el código con alguien más. **Nunca toques `node_modules` directamente.**

Confundir Node.js con un lenguaje nuevo

A veces los estudiantes dicen: *"Estoy aprendiendo Node"*. Es importante recordar que **Node no es un lenguaje**, es JavaScript. Si sabes hacer un `for`, un `if` o una función en JS, ya sabes programar en Node. Lo único que cambia son las herramientas (APIs) que tienes a mano.

Ignorar el archivo `package.json`

Este archivo es el "Documento de Identidad" de tu proyecto. Muchos principiantes lo ven como algo que se crea solo y no lo miran.

- **Mejor práctica:** Lee siempre tu `package.json`. Ahí verás qué librerías está usando tu proyecto y qué comandos (scripts) puedes ejecutar. Es el mapa de tu aplicación.

React, Vue y Angular: cómo pensar en frameworks modernos

Exploración de la filosofía detrás de los frameworks modernos (React, Vue, Angular), comparando el enfoque de componentes y estado frente al JavaScript tradicional para escalar aplicaciones web.

A lo largo de este programa, has aprendido a dominar los cimientos de JavaScript. Has manipulado el DOM, has gestionado eventos, has realizado peticiones a servidores y has almacenado datos en el navegador. Con esas herramientas, técnicamente puedes construir cualquier aplicación web.

Sin embargo, si intentaras construir Facebook, Spotify o un panel de administración complejo usando solo `document.querySelector` y manipulando el HTML manualmente paso a paso, pronto te encontrarías en una pesadilla de mantenimiento. Este es el punto donde el JavaScript "puro" (o Vanilla JS) se vuelve difícil de escalar y donde entran en juego los **frameworks modernos**.

1. El problema de escalar interfaces con JavaScript manual

Imagina que estás construyendo una lista de tareas (To-Do List). Con lo que sabes hasta ahora, el flujo sería algo así:

1. Escuchas el evento `click` en el botón "Agregar".
2. Obtienes el valor del `input`.
3. Creas un nuevo elemento `<li>` con `document.createElement`.
4. Le asignas el texto al `<li>`.
5. Buscas el `<ul>` en el DOM y le haces un `appendChild`.
6. Si quieres borrar la tarea, debes buscar ese elemento específico y eliminarlo del DOM.

Este enfoque se llama **Imperativo**: le estás dando al navegador instrucciones detalladas, paso a paso, de *cómo* cambiar la interfaz.

El "Espagueti" del DOM

Cuando la aplicación crece, surgen problemas graves:

- **Sincronización de datos:** Si tienes el nombre del usuario en la barra de navegación y también en el perfil, y el usuario lo cambia, tienes que acordarte de buscar ambos elementos en el DOM y actualizarlos manualmente. Si te olvidas de uno, la interfaz queda inconsistente.
- **Complejidad visual:** Rastrear qué parte del código cambió qué parte del HTML se vuelve una tarea detectivesca.

- **Fragilidad:** Si cambias una clase en tu HTML, es probable que tus selectores de JavaScript dejen de funcionar y la aplicación se rompa silenciosamente.

Los frameworks modernos proponen un cambio de paradigma: el enfoque **Declarativo**. En lugar de decirle al navegador *cómo* cambiar, le dices *qué* quieres ver según los datos que tienes.

2. Componentes: La unidad de interfaz reutilizable

La idea central de los frameworks modernos es la **Componentización**.

Imagina que tu aplicación es un juego de LEGO. En lugar de ver una página web como un único documento gigante de HTML, la ves como una colección de piezas independientes y reutilizables llamadas **Componentes**.

¿Qué es un componente?

Un componente es una pieza de la interfaz que encapsula tres cosas:

1. **Estructura (HTML):** ¿Qué etiquetas tiene?
2. **Lógica (JavaScript):** ¿Cómo se comporta?
3. **Estilo (CSS):** ¿Cómo se ve?

Por ejemplo, una "Tarjeta de Producto" en una tienda online es un componente perfecto. Tiene una imagen, un título, un precio y un botón de "Comprar".

- En Vanilla JS, copiarías y pegarías el HTML de la tarjeta muchas veces.
- En un framework, defines el componente una sola vez y lo "invocas" pasando diferentes datos (nombre, precio, foto).

La ventaja competitiva: Si decides cambiar el color del botón de compra, lo cambias en el archivo del componente y se actualiza automáticamente en toda la aplicación, sin importar si aparece en la home, en el carrito o en los favoritos.

3. Estado y Reactividad: El motor de la interfaz

Este es el concepto más importante para entender los frameworks.

El **Estado** (State) son los datos que definen lo que se ve en la pantalla en un momento dado. Por ejemplo:

- ¿Está el modal abierto? `true` o `false`.
- ¿Qué hay en el carrito? Un array de productos.
- ¿Se están cargando los datos? `loading: true`.

El concepto de Reactividad

En los frameworks modernos, la interfaz es una "función" del estado.

> *Interfaz = Framework (Estado)*

Cuando el **Estado cambia**, el framework detecta automáticamente ese cambio y actualiza la interfaz por ti. Tú ya no tocas el DOM directamente; tú solo actualizas los datos (el estado) y dejas que el framework se encargue del trabajo sucio de mover etiquetas.

Ejemplo concreto: Si tienes una variable `contador = 5` mostrada en un componente, y haces un `contador++`, el framework verá que el dato cambió y refrescará el pedacito de HTML exacto que muestra ese número. No tienes que hacer `document.innerText = contador`. Eso es **Reactividad**.

4. Panorama general: React, Vue y Angular

Aunque todos comparten la idea de componentes y estado, cada uno tiene una "personalidad" diferente.

React: La biblioteca flexible

Creado y mantenido por **Meta (Facebook)**, React es actualmente la tecnología más popular. Oficialmente se define como una "biblioteca" y no como un framework, porque React solo se encarga de la vista (los componentes).

- **Filosofía:** "Todo es JavaScript". Usa una sintaxis llamada **JSX** que permite escribir algo muy parecido a HTML directamente dentro de tus funciones de JavaScript.
- **Punto fuerte:** Ecosistema gigantesco. Hay una librería de React para casi cualquier cosa que imagines.
- **Ideal para:** Proyectos donde quieres elegir tus propias herramientas y donde la flexibilidad es clave.

Vue: El framework progresivo

Creado por Evan You, Vue nació con el objetivo de tomar lo mejor de Angular y React para hacerlo más sencillo. Es famoso por su curva de aprendizaje muy suave.

- **Filosofía:** Separación de preocupaciones. Un componente de Vue suele tener tres secciones claras: `<template>` (HTML), `<script>` (JS) y `<style>` (CSS). Esto se siente muy natural para quien viene del desarrollo web tradicional.
- **Punto fuerte:** Documentación excelente y facilidad para integrarse gradualmente en proyectos existentes.
- **Ideal para:** Equipos que quieren empezar rápido sin sacrificar potencia, o proyectos que necesitan una estructura clara pero sencilla.

Angular: El framework robusto

Mantenido por **Google**, Angular es un framework "con todo incluido" (batteries included). A diferencia de React, Angular te da todas las herramientas necesarias para routing, formularios y peticiones HTTP desde el primer segundo.

- **Filosofía:** Estructura y escalabilidad profesional. Usa **TypeScript** obligatoriamente, lo que ayuda a evitar muchos errores en aplicaciones muy grandes.
- **Punto fuerte:** Consistencia. Si entras a un proyecto de Angular, la estructura siempre será muy similar, facilitando el trabajo en equipos de cientos de desarrolladores.
- **Ideal para:** Aplicaciones empresariales de gran escala (bancos, sistemas internos complejos) donde la estabilidad y las reglas estrictas son una ventaja.

5. ¿Cuándo usar un framework?

No siempre necesitas un framework. Es importante no caer en la "sobreingeniería".

- **Usa un framework si:**
- Tu aplicación tiene muchos datos que cambian constantemente (dashboards, redes sociales).
- Necesitas reutilizar muchas partes de la interfaz.
- El equipo de desarrollo va a crecer.
- Quieres crear una **SPA (Single Page Application)** donde la navegación sea instantánea.
- **Usa Vanilla JS (o solo HTML/CSS) si:**
- Es una página estática (como un blog simple o una landing page).
- Necesitas un rendimiento extremo y cada kilobyte del bundle importa.
- Es un script muy pequeño para una funcionalidad específica.

MÓDULO 12

Proyecto Final

Consolidar todas las competencias del programa en un proyecto integrador final.

ENTREGA FINAL DEL PROYECTO

Actividad evaluable · proyecto integrador final

Entrega final: aplicación web interactiva

Instrucciones detalladas y criterios de aceptación para la entrega final del curso de JavaScript, integrando todos los conocimientos de aprendidos a lo largo de los 11 módulos.

Objetivo

Desarrollar un simulador de proceso comercial o profesional completo (compra, reserva, cotización, gestión, etc.) que integre todas las herramientas JavaScript del curso.

Criterios de aceptación

Estructura de archivos

- `index.html` en la raíz como único archivo suelto; resto en carpetas organizadas
- Carpeta `assets/` con recursos multimedia
- Entre 2 y 3 archivos `.js` correctamente referenciados
- Al menos un archivo `.json` usado como base de datos simulada
- CSS propio o framework (Bootstrap, Tailwind, etc.); interfaz clara e intuitiva

DOM e interacción

- Toda interacción con el usuario es mediante la interfaz (prohibido `prompt`, `alert`, `confirm`)
- Manipulación dinámica del DOM en respuesta a acciones del usuario

Arrays y objetos

- Todos los arrays de objetos están en `.json` y se consumen con `fetch`
- Se usan al menos 2 funciones de orden superior distintas (`map`, `filter`, `reduce`, `find`, `some`, `forEach`, etc.) para buscar, filtrar, transformar o calcular datos

Storage y sintaxis avanzada

- `localStorage` o `sessionStorage` con operaciones de guardar, modificar, borrar y vaciar
- Código usa al menos: operador ternario, operador OR y destructuring

Asincronismo

- `fetch` con `async` / `await`, `try` / `catch` y `finally` cuando aplique
- La app maneja errores de carga de datos

Librería externa

- Al menos una librería JS integrada (SweetAlert, Toastify u otra) que reemplace `alert` / `confirm` / `prompt`

Lógica del simulador

- El circuito principal está completo de punta a punta (ej. para e-commerce: ver productos → seleccionar → carrito → modificar cantidades → calcular total → confirmar)
- Sin registro ni login (no es parte del alcance)

Código

- Nombres de variables, funciones y objetos son semánticos (sin `x`, `a`, `funcA`, etc.)
- Sin código comentado en desuso ni `console.log` innecesarios

Cómo entregar

Repositorio público de GitHub con todos los archivos. Verificar antes de entregar que las rutas de archivos y la carga del `.json` funcionen correctamente. Incluir `readme.md` si el proyecto requiere configuración previa.

Qué evitar

- `prompt`, `alert` o `confirm` nativos en cualquier parte del código
- Arrays de objetos hardcodeados en el `.js` (deben vivir en `.json`)
- Circuito del simulador incompleto (ej. carrito sin confirmación final)
- Más de 3 archivos `.js`
- Archivos sueltos fuera de la raíz salvo `index.html`

Proyecto Final: Aplicación Web Interactiva

Objetivo: Entregar el simulador final que demuestre el dominio de JavaScript, manipulación del DOM, asincronismo y persistencia de datos.

1. Requisitos Técnicos del Proyecto (Checklist)

A. Uso de DOM:

Tu aplicación JavaScript debe interactuar integramente con HTML utilizando DOM y los eventos necesarios. No se debe tener en ningun caso prompt ni alertas nativas del navegador.

B. Arrays y Objetos:

Todos los arrays de objetos que utilices dentro de tu aplicación deben ser convertidos **obligatoriamente** a un archivo en formato .JSON y accedidos mediante la tecnología Fetch.

C. Funciones de Orden Superior:

El proyecto debe contar con mínimo dos funciones de orden superior diferentes (ej. un forEach y un reduce) para interactuar con el simulador y realizar tareas de búsqueda o transformación de la información según sea necesario.

D. Storage y Operadores Avanzados:

El proyecto debe tener correctamente implementado el uso de storage, permitiendo que se pueda guardar, borrar, modificar informacion o vaciar el storage para una correcta experiencia del usuario.

Se deben plantear mejoras en el script con operadores avanzados (ternario, or) y destructuring.

E. Asincronismo y Peticiones:

El proyecto debe tener correctamente implementado fetch para consumir una api/json local, planteando la estructura con async-await y un correcto manejo de errores con try-catch-finally.

F. Uso de Librerias:

Debes incluir al menos una librería JS externa(sweetalert/toastify) y debes eliminar el uso de herramientas más limitadas del lenguaje, como ser los cuadros de diálogo Prompt, Confirm y Alert.

G. Lógica del simulador:

El simulador interactivo debe completar todo el circuito o proceso de negocio, de acuerdo a la temática del mismo. Debes obviar temas más complejos como ser un registro de usuario y login, pero no puedes obviar armar un circuito completo de una compra online, o de la cotización de productos o servicios.

H. Código claro:

Todo tu código debe ser claro y estar correctamente estructurado para su lectura. Puedes dejar comentarios breves en el código, pero no puedes dejar código en desuso/comentado que complique la lectura y análisis del mismo. Toda la nomenclatura de variables, arrays, objetos, funciones debe ser semántica (evitar usar nombres genericos como "x", "a", "funcA")

Entrega:

Sube tus archivos a un repositorio de GitHub y comparte la URL. Asegúrate de que el repositorio sea público para que pueda ser revisado.

Cómo se evalúa esta entrega

Esta rúbrica evalúa la entrega final integradora del programa JavaScript desde Cero. Se valora que la aplicación web sea funcional, que integre DOM, eventos, almacenamiento, asincronismo, API externa y librería. El foco está en la consolidación del trabajo final, la claridad de la evidencia entregada y la calidad de la defensa técnica.

Criterio	Descripción	Peso
Uso de DOM	El simulado debe interactuar integramente con HTML utilizando DOM y los eventos necesarios. No se debe tener en ningun caso prompt ni alertas nativas del navegador.	20%
Uso de arrays y objetos	Se debe trabajar con arrays de objetos para formar la estructura de la base de datos, estos deben ser convertidos obligatoriamente a un archivo en formato .JSON y accedidos mediante Fetch.	15%
Funciones de orden superior	El proyecto debe contar con mínimo dos funciones de orden superior diferentes (ej. un forEach y un reduce) para interactuar con el simulador y realizar tareas de búsqueda o transformación de la información según sea necesario.	15%
Storage y operadores avanzados	El proyecto debe tener correctamente implementado el uso de storage, permitiendo que se pueda guardar, borrar, modificar informacion o vaciar el storage para una correcta experiencia del usuario. Se deben plantear mejoras en el script con operadores avanzados (ternario, or) y destructuring.	8%
Asincronismo y peticiones	El proyecto debe tener correctamente implementado fetch para consumir una api/json local, planteando la estructura con async-await y un correcto manejo de errores con try-catch-finally.	15%
Librerias	Se debe incluir al menos una librería JS externa(sweetalert/toastify) para reemplazar todo uso de Prompt, Confirm y Alert.	8%
Logica completa del simulador	El simulador interactivo debe completar todo el circuito o proceso de negocio, de acuerdo a la temática del mismo. Se debe obviar temas más complejos como un registro de usuario y login, pero no obviar armar un circuito completo de una compra online, o de la cotización de productos o servicios.	15%
Codigo claro y estructura completa	Se debe presentar el proyecto en el repositorio de github con la estructura de archivos y carpetas mencionada a lo largo del curso. El archivo javascript enrutado en el head del documento con el atributo defer obligatoriamente. Toda la nomenclatura de variables, arrays, objetos, funciones debe ser semántica, no se pueden utilizar nombres genericos (a, x, func1, numero2, etc.)	4%

Total: 100 pts | Aprobación: 70 pts

ANEXO

Material complementario

Glosario, análisis en profundidad, preguntas frecuentes y conclusiones del curso.

Palabras clave

- **Variable:** Un contenedor con un nombre específico donde guardamos un dato que nuestro programa necesitará usar más tarde. Imagínalo como una caja etiquetada donde puedes guardar un número o un texto para cambiarlo o consultarlo en cualquier momento del código.
- **Tipo de Dato Primitivo:** Son los valores más básicos y simples que JavaScript puede procesar, como los números, las cadenas de texto (strings) y los booleanos (true/false). A diferencia de los objetos complejos, estos representan un único valor que no puede ser descompuesto en partes más pequeñas.
- **Estructura de Control:** Instrucciones que permiten decidir qué camino sigue el programa dependiendo de si se cumple una condición o no. Por ejemplo, un bloque 'if' permite que una acción ocurra solo si el usuario ha iniciado sesión, de lo contrario, el programa toma una ruta distinta.
- **Bucle (Loop):** Una estructura que permite repetir un bloque de código varias veces de forma automática mientras se cumpla una condición. Es muy útil para tareas repetitivas, como mostrar en pantalla todos los elementos de una lista de compras sin escribir código para cada uno.
- **Función:** Un bloque de código reutilizable diseñado para realizar una tarea específica cuando se le solicita. En lugar de escribir la misma lógica muchas veces, creas una función una vez y la 'llamas' cada vez que necesites ejecutar esa acción en tu aplicación.
- **Scope (Ámbito):** Define la visibilidad y el alcance que tiene una variable dentro del código. Por ejemplo, una variable declarada dentro de una función tiene un ámbito local y no puede ser accedida desde afuera, lo que ayuda a evitar errores y conflictos de nombres.
- **Array:** Una estructura de datos que permite almacenar una lista ordenada de elementos bajo un solo nombre de variable. Es similar a una lista de contactos en tu teléfono, donde cada contacto ocupa una posición numerada empezando desde el cero.
- **Objeto Literal:** Una colección de datos relacionados que se guardan mediante pares de 'clave' y 'valor'. Se usa para representar entidades del mundo real; por ejemplo, un objeto 'Auto' podría tener propiedades como marca, modelo y color agrupadas en un solo lugar.
- **DOM (Document Object Model):** Es la representación interna que hace el navegador de una página web, convirtiendo el HTML en una estructura de objetos manipulables. Gracias al DOM, JavaScript puede cambiar textos, colores o eliminar elementos de la página en tiempo real sin recargar el navegador.
- **Evento:** Cualquier acción que ocurre en el navegador, ya sea iniciada por el usuario o por el propio sistema, como hacer clic en un botón, mover el ratón o terminar de cargar una imagen. JavaScript permite 'escuchar' estos eventos para ejecutar código como respuesta inmediata.

- **Asincronismo:** Una forma de ejecutar tareas largas (como descargar datos de internet) sin detener el resto del programa. Esto permite que la página web siga respondiendo a los clics del usuario mientras los datos se cargan de fondo.
- **Promesa:** Un objeto que representa el resultado futuro de una operación que aún no ha terminado. Imagínala como un ticket de pedido en un restaurante: el ticket garantiza que eventualmente recibirás tu comida (éxito) o un aviso de que se agotó (error).
- **API (Application Programming Interface):** Un conjunto de reglas que permite que dos aplicaciones se comuniquen entre sí para compartir datos. Por ejemplo, tu aplicación de JavaScript puede usar una API de Google Maps para mostrar un mapa interactivo sin tener que programar el mapa desde cero.
- **JSON (JavaScript Object Notation):** Un formato de texto ligero y fácil de leer que se utiliza para intercambiar datos entre servidores y aplicaciones web. Aunque parece un objeto de JavaScript, es en realidad texto plano que cualquier lenguaje de programación puede entender y procesar.
- **Fetch API:** Una herramienta moderna de JavaScript que permite realizar peticiones de datos a servidores de forma asincrónica. Se utiliza principalmente para obtener información de una API externa e integrarla dinámicamente en tu sitio web sin interrumpir la experiencia del usuario.

Análisis profundo

El Corazón de la Reactividad: El DOM y el Event Loop

Para entender por qué JavaScript es el lenguaje rey de la web, debemos mirar más allá de la sintaxis y comprender su motor invisible. Imagina que el navegador es una orquesta. El DOM (Document Object Model) es la partitura, y el Event Loop es el director que decide cuándo entra cada músico. Si no comprendemos esta relación, nuestras aplicaciones se sentirán 'pesadas' o 'congeladas'. El DOM no es JavaScript; es una representación jerárquica de tu HTML que JavaScript puede 'tocar'. El gran problema surge cuando intentamos manipularlo miles de veces por segundo sin entender que cada cambio es costoso en términos de rendimiento. Aquí entra el Event Loop: JS es monohilo, lo que significa que solo puede hacer una cosa a la vez. Si una tarea tarda mucho (como una petición a una API), el Event Loop se asegura de que no se bloquee la interfaz, mandando esa tarea a la 'sala de espera' y siguiendo con el resto. Empresas como Netflix optimizan al máximo estas interacciones para que, mientras se carga la carátula de una película, tú puedas seguir haciendo scroll sin retrasos. El error más común es pensar que un 'setTimeout' de cero milisegundos se ejecuta inmediatamente; en realidad, solo lo pone al final de la cola, permitiendo que el navegador respire antes de actuar. Dominar esto separa a quien 'copia código' de quien 'diseña experiencias'.

La Revolución del Asincronismo: De Callbacks a Async/Await

Históricamente, manejar procesos que tomaban tiempo (como leer una base de datos) era un caos conocido como 'Callback Hell'. Imagina pedir una pizza y tener que quedarte parado frente al teléfono sin hacer nada más hasta que llegue. Eso es el código síncrono. La asincronía nos permite 'hacer el pedido' y recibir un 'avisador' (una Promesa) que vibrará cuando la pizza esté lista, permitiéndonos ver

una serie mientras tanto. Las Promesas fueron un gran avance, pero la sintaxis de 'Async/Await' introducida en ES6+ es lo que realmente cambió el juego, permitiéndonos escribir código asíncrono que se lee casi como si fuera síncrono. Esto es vital porque en el desarrollo moderno, casi todo es asíncrono: desde autenticar a un usuario con Firebase hasta consultar el clima en una API externa. Un error frecuente en principiantes es olvidar el bloque 'try/catch' al usar Async/Await, asumiendo que la red nunca fallará. En la vida real, las conexiones fallan y los servidores caen. Aprender a gestionar el estado de 'rechazado' de una promesa es tan importante como gestionar el éxito, ya que garantiza que tu aplicación no se rompa silenciosamente frente al usuario, sino que informe elegantemente del error.

Programación Funcional y Métodos de Orden Superior

En los módulos intermedios del programa, nos alejamos de los bucles 'for' tradicionales para dar paso a métodos como map, filter y reduce. ¿Por qué complicarnos con esto? Porque el código declarativo es más fácil de mantener y menos propenso a errores que el imperativo. En un bucle trivial, tú controlas el índice y el incremento, lo que deja mucho margen para el error del 'fuera de rango'. Con las funciones de orden superior, tú le dices a JavaScript QUÉ quieres hacer, no CÓMO hacerlo paso a paso. Piensa en una aplicación de e-commerce como Amazon: para mostrar el carrito, necesitas filtrar (filter) los productos con stock, transformar (map) sus precios a la moneda local y calcular el total (reduce). Si intentas hacer todo esto con bucles anidados, el código se vuelve una maraña ilegible. El concepto clave aquí es la 'inmutabilidad': estos métodos no modifican el array original, sino que crean uno nuevo. Esta es la base de frameworks modernos como React. Muchos estudiantes confunden 'map' con 'forEach'; la diferencia crucial es que 'map' devuelve un valor, lo que permite encadenar operaciones de forma elegante y poderosa, transformando datos complejos en interfaces simples.

Preguntas frecuentes

P: ¿Cuál es la diferencia real entre var, let y const y por qué se dice que 'var' ya no debe usarse?

R: Esta es la duda número uno al comenzar. Históricamente, JavaScript solo tenía 'var', pero presentaba problemas de 'scope' (alcance) que causaban errores difíciles de encontrar. 'var' tiene un alcance de función, lo que significa que si declaras una variable dentro de un if, aún puedes acceder a ella fuera de ese bloque. En cambio, 'let' y 'const' tienen alcance de bloque (todo lo que esté entre llaves {}), lo cual es mucho más seguro y predecible. La regla de oro moderna es: usa 'const' por defecto para todo lo que no planees reasignar (como una configuración o un selector del DOM) y 'let' solo cuando sepas que el valor cambiará (como un contador en un bucle). Al usar 'const', proteges tu código de cambios accidentales, lo que hace que tu lógica sea más robusta y fácil de seguir para otros desarrolladores.

P: ¿Por qué necesito aprender métodos como map, filter y reduce si puedo hacer lo mismo con un bucle 'for'?

R: Muchos estudiantes asumen que las Funciones de Orden Superior son solo una forma 'elegante' de escribir código, pero su propósito es la legibilidad y la reducción de efectos secundarios. Un bucle 'for'

es imperativo: le dices a la computadora paso a paso cómo hacer las cosas (crear un array vacío, iterar, empujar elementos). En cambio, map() y filter() son declarativos: le dices qué quieres obtener. Por ejemplo, con filter() el código describe tu intención: 'quiero los productos que cuesten más de 100'. Esto reduce drásticamente la cantidad de código y los errores humanos. En la industria moderna, escribir código limpio es tan importante como que el código funcione, y dominar estos métodos es lo que separa a un principiante de un desarrollador profesional.

P: Me confunde el asincronismo: ¿Cómo es que JavaScript puede hacer varias cosas a la vez si es de un solo hilo?

R: Es una gran pregunta que toca el corazón de cómo funciona el navegador. Imagina JavaScript como un cocinero en una cocina pequeña. Él solo tiene dos manos (un solo hilo), pero puede poner agua a hervir (petición API) y, mientras espera, picar cebolla (ejecutar lógica). No está haciendo las dos cosas al mismo instante, sino que delega la espera al sistema (el Event Loop) y continúa con lo siguiente. Cuando el agua hierve, suena un temporizador y el cocinero vuelve a esa tarea. Las Promesas y el Async/Await son las herramientas que usamos para organizar ese 'temporizador'. Muchos fallan aquí porque intentan usar el resultado de una API antes de que llegue; por eso aprendemos a usar 'await', que básicamente le dice al código: 'espera aquí hasta que la respuesta llegue antes de pasar a la siguiente línea'.

P: ¿Es estrictamente necesario saber HTML y CSS antes de profundizar en el DOM con JavaScript?

R: No necesitas ser un experto diseñador, pero sí entender la estructura. JavaScript en el desarrollo web tiene un propósito principal: dar vida a la estructura (HTML) y al diseño (CSS). Si no sabes qué es un 'id', una 'clase' o un 'input', te costará entender por qué usamos querySelector o por qué escuchamos un evento de 'submit'. Piensa en el HTML como el esqueleto, el CSS como la piel y JavaScript como los músculos y el sistema nervioso. Para este programa, con que sepas crear etiquetas básicas y entiendas cómo se jerarquizan los elementos (padres e hijos), será suficiente para empezar a manipular el DOM con éxito. A medida que avances, verás que ambos conocimientos se retroalimentan.

P: ¿Cuándo debería usar localStorage en lugar de solo guardar mis datos en variables?

R: La confusión común es creer que las variables mantienen la información siempre. Sin embargo, en cuanto el usuario presiona F5 o cierra la pestaña, todas tus variables vuelven a su estado inicial o desaparecen. Aquí es donde entra el Web Storage. Debes usar localStorage cuando quieras que la experiencia del usuario sea persistente: por ejemplo, guardar los productos de un carrito de compras o el nombre del perfil. Si solo necesitas que el dato viva mientras el usuario realiza un cálculo específico, una variable es suficiente. Un error típico es intentar guardar objetos directamente en el storage; recuerda que localStorage solo entiende texto, por lo que siempre usamos JSON.stringify() para 'empaquetar' nuestros datos antes de guardarlos y JSON.parse() al recuperarlos.

P: ¿Por qué el curso termina con Node.js y Frameworks si el objetivo es aprender JavaScript?

R: JavaScript ha dejado de ser solo un lenguaje para navegadores. Aprender las bases de Node.js es vital porque hoy en día casi todas las herramientas de desarrollo (librerías, linters, optimizadores)

corren sobre él. Entender Node y npm (el gestor de paquetes) te abre la puerta a instalar utilidades profesionales como SweetAlert2 o Toastify. Por otro lado, los Frameworks como React o Vue son el estándar de la industria actual. No los vemos a profundidad en este curso inicial, pero te damos la base porque sería irresponsable dejarte solo con JavaScript 'puro' sin mostrarte hacia dónde va el mundo laboral. Al terminar, no solo sabrás programar, sino que sabrás cómo se construyen las aplicaciones reales en 2024.

Conclusiones

Del Script Aislado a la Arquitectura Funcional

Has trascendido la escritura de líneas de código sueltas para construir sistemas cohesivos. Al integrar la lógica de control del Módulo 2 con la modularización de funciones del Módulo 3 y la programación orientada a objetos del Módulo 5, ahora comprendes que el desarrollo moderno no se trata de 'escribir por escribir', sino de estructurar entidades y procesos escalables. Esta mentalidad arquitectónica es lo que diferencia a un programador de un implementador; ahora puedes diseñar soluciones donde los datos (objetos/arrays) y el comportamiento (funciones) coexisten de manera organizada, facilitando el mantenimiento y la colaboración en equipos profesionales.

La Dinámica entre el Estado y la Interfaz de Usuario

Una de las integraciones más críticas que has logrado es la conexión entre la lógica de datos y la representación visual. Combinando el manejo de colecciones (Arrays, Módulo 4), métodos de orden superior (Módulo 6) y la manipulación del DOM (Módulo 7), has aprendido a crear interfaces reactivas. Ya no ves un sitio web como un documento estático, sino como una representación visual de un 'estado' de datos. Dominar esta relación te permite construir aplicaciones profesionales donde cada interacción del usuario transforma la información y actualiza la vista de forma coherente, transformando sitios simples en aplicaciones web interactivas.

Persistencia y Asincronía: La Web que Nunca se Detiene

Has dominado el arte de manejar el tiempo y la memoria en el navegador. Al unir los conceptos de Web Storage (Módulo 8) con la gestión de asincronía y APIs (Módulos 9 y 10), has superado la barrera de la ejecución secuencial. Ahora puedes diseñar experiencias que no se bloquean mientras esperan datos del servidor y que 'recuerdan' al usuario incluso después de recargar la página. En el mundo laboral, esta capacidad es fundamental para consumir microservicios, manejar tiempos de carga y garantizar que la experiencia de usuario sea fluida y profesional, independientemente de la velocidad de la conexión.

Hacia el Futuro: Professionalismo y Ecosistema Moderno

Al finalizar con la introducción a Node.js y frameworks (Módulo 11), has situado tu conocimiento dentro del gran ecosistema de la industria. Ya no eres un principiante que solo conoce sintaxis; eres un desarrollador capaz de usar npm para gestionar dependencias, implementar buenas prácticas de ES6+

y entender cómo tu código encaja en infraestructuras mayores como React o Vue. Estás listo para saltar de los ejercicios académicos a proyectos reales, aplicando un razonamiento basado en la eficiencia, el uso de librerías externas para potenciar tu trabajo y el rigor técnico necesario para desplegar aplicaciones que resuelvan problemas reales.
